BUKU KERJA PRAKTIK MAHASISWA (BKPM)

WORKSHOP APLIKASI MOBILE BERBASIS FLUTTER

TKK30708

SEMESTER 3



OLEH :

1. Yogiswara, S.T., M.T.
2. Arvita Agus Kurniasari, S.ST., M.Tr.Kom.
3. Agus Hariyanto, S.T., M.Kom.

PROGRAM STUDI TEKNIK KOMPUTER

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI
POLITEKNIK NEGERI JEMBER

LEMBAR PENGESAHAN

BKPM WORKSHOP APLIKASI MOBILE BERBASIS FLUTTER

Mengetahui,

Koord. Program Studi,	Koord./Tim Mata Kuliah,	Penulis,
I Gede Wiryawan, S.Kom., M.Kom.	**Yogiswara, S.T., M.T.**	**Yogiswara, S.T., M.T.**
NIP. 198801172019031008	NIP. 197009292003121001	NIP. 197009292003121001

Menyetujui,
Ketua Jurusan Teknologi Informasi,

Hendra Yufit Riskiawan, S.Kom., M.Cs.
NIP. 198302032006041003

KATA PENGANTAR

BKPM mata kuliah Workshop Aplikasi Mobile ini merupakan pegangan dalam pelaksanaan pembelajaran untuk pemenuhan Capaian Pembelajaran Lulusan (CPL) dan Capaian Pembelajaran Mata Kuliah (CPMK). Referensi yang digunakan dalam penyusunan BKPM ini bersumber dari buku-buku dan dokumentasi resmi terkait pengembangan aplikasi mobile. Mata kuliah Workshop Aplikasi Mobile merupakan salah satu mata kuliah rumpun pemrograman di Program Studi Teknik Komputer.

Pada edisi ini, seluruh praktikum disusun menggunakan **framework Flutter** dengan bahasa pemrograman **Dart**, menggantikan pemrograman Android native (Java/XML) pada edisi sebelumnya. Jumlah acara dan pokok bahasan tetap sama, yaitu 56 acara yang meliputi Git, instalasi dan pengenalan lingkungan pengembangan, layout dan Material Design, list (RecyclerView), fragment, intent, manajemen file dan SQLite, shared preferences, GPS dan sensor, JSON, komunikasi HTTP (GET/POST), serta project final. Setiap konsep Android native yang dibahas tetap dijelaskan beserta padanannya di Flutter, sehingga mahasiswa memahami keduanya.

Dengan Flutter, satu basis kode dapat dijalankan pada Android, iOS, web, dan desktop. Diharapkan BKPM ini dapat membantu mahasiswa menguasai pengembangan aplikasi mobile modern dan siap diterapkan pada project akhir maupun dunia kerja.

Jember, ... 2026

Penulis

DAFTAR ISI

Acara 1 : PENGENALAN DAN WORKFLOW PADA GIT 13
a. Capaian Pembelajaran Mata Kuliah (CPMK) 13
b. Indikator Penilaian 13
c. Dasar Teori 13
d. Alat dan Bahan 14
e. Prosedur Kerja 14
f. Hasil dan Pembahasan 14
g. Kesimpulan 14
h. Rubrik Penilaian 14
Acara 2 : PENGENALAN DAN WORKFLOW PADA GIT 15
a. Capaian Pembelajaran Mata Kuliah (CPMK) 15
b. Indikator Penilaian 15
c. Dasar Teori 15
d. Alat dan Bahan 15
e. Prosedur Kerja 15
f. Hasil dan Pembahasan 16
g. Kesimpulan 16
h. Rubrik Penilaian 16
Acara 3 : PENGENALAN DAN WORKFLOW PADA GIT 17
a. Capaian Pembelajaran Mata Kuliah (CPMK) 17
b. Indikator Penilaian 17
c. Dasar Teori 17
d. Alat dan Bahan 17
e. Prosedur Kerja 17
f. Hasil dan Pembahasan 18
g. Kesimpulan 18
h. Rubrik Penilaian 18
Acara 4 : PENGENALAN DAN WORKFLOW PADA GIT 19
a. Capaian Pembelajaran Mata Kuliah (CPMK) 19
b. Indikator Penilaian 19
c. Dasar Teori 19
d. Alat dan Bahan 19
e. Prosedur Kerja 19
f. Hasil dan Pembahasan 20
g. Kesimpulan 20
h. Rubrik Penilaian 20
Acara 5 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO 21
a. Capaian Pembelajaran Mata Kuliah (CPMK) 21
b. Indikator Penilaian 21
c. Dasar Teori 21
d. Alat dan Bahan 21
e. Prosedur Kerja 21
f. Hasil dan Pembahasan 23
g. Kesimpulan 23
h. Rubrik Penilaian 23
Acara 6 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO 24
a. Capaian Pembelajaran Mata Kuliah (CPMK) 24
b. Indikator Penilaian 24
c. Dasar Teori 24
d. Alat dan Bahan 24
e. Prosedur Kerja 24
f. Hasil dan Pembahasan 25
g. Kesimpulan 25
h. Rubrik Penilaian 25

Acara 7 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO 26
a. Capaian Pembelajaran Mata Kuliah (CPMK) 26
b. Indikator Penilaian 26
c. Dasar Teori 26
d. Alat dan Bahan 27
e. Prosedur Kerja 27
f. Hasil dan Pembahasan 29
g. Kesimpulan 29
h. Rubrik Penilaian 29
Acara 8 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO 30
a. Capaian Pembelajaran Mata Kuliah (CPMK) 30
b. Indikator Penilaian 30
c. Dasar Teori 30
d. Alat dan Bahan 30
e. Prosedur Kerja 30
f. Hasil dan Pembahasan 30
g. Kesimpulan 30
h. Rubrik Penilaian 30
Acara 9 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW 32
a. Capaian Pembelajaran Mata Kuliah (CPMK) 32
b. Indikator Penilaian 32
c. Dasar Teori 32
d. Alat dan Bahan 33
e. Prosedur Kerja 33
f. Hasil dan Pembahasan 37
g. Kesimpulan 37
h. Rubrik Penilaian 37
Acara 10 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW 39
a. Capaian Pembelajaran Mata Kuliah (CPMK) 39
b. Indikator Penilaian 39
c. Dasar Teori 39
d. Alat dan Bahan 40
e. Prosedur Kerja 40
f. Hasil dan Pembahasan 43
g. Kesimpulan 43
h. Rubrik Penilaian 43
Acara 11 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW 44
a. Capaian Pembelajaran Mata Kuliah (CPMK) 44
b. Indikator Penilaian 44
c. Dasar Teori 44
d. Alat dan Bahan 44
e. Prosedur Kerja 44
f. Hasil dan Pembahasan 48
g. Kesimpulan 48
h. Rubrik Penilaian 48
Acara 12 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW 50
a. Capaian Pembelajaran Mata Kuliah (CPMK) 50
b. Indikator Penilaian 50
c. Dasar Teori 50
d. Alat dan Bahan 50
e. Prosedur Kerja 50
f. Hasil dan Pembahasan 53
g. Kesimpulan 53
h. Rubrik Penilaian 54
Acara 13 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW) 55
a. Capaian Pembelajaran Mata Kuliah (CPMK) 55
b. Indikator Penilaian 55

c. Dasar Teori 55
d. Alat dan Bahan 55
e. Prosedur Kerja 55
f. Hasil dan Pembahasan 56
g. Kesimpulan 56
h. Rubrik Penilaian 56

Acara 14 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW) 57
a. Capaian Pembelajaran Mata Kuliah (CPMK) 57
b. Indikator Penilaian 57
c. Dasar Teori 57
d. Alat dan Bahan 57
e. Prosedur Kerja 57
f. Hasil dan Pembahasan 58
g. Kesimpulan 59
h. Rubrik Penilaian 59

Acara 15 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW) 60
a. Capaian Pembelajaran Mata Kuliah (CPMK) 60
b. Indikator Penilaian 60
c. Dasar Teori 60
d. Alat dan Bahan 60
e. Prosedur Kerja 61
f. Hasil dan Pembahasan 63
g. Kesimpulan 63
h. Rubrik Penilaian 63

Acara 16 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW) 65
a. Capaian Pembelajaran Mata Kuliah (CPMK) 65
b. Indikator Penilaian 65
c. Dasar Teori 65
d. Alat dan Bahan 65
e. Prosedur Kerja 65
f. Hasil dan Pembahasan 65
g. Kesimpulan 65
h. Rubrik Penilaian 66

Acara 17 : FRAGMENT (WIDGET MODULAR) 67
a. Capaian Pembelajaran Mata Kuliah (CPMK) 67
b. Indikator 67
c. Dasar Teori 67
d. Alat dan Bahan 69
e. Prosedur Kerja 69
f. Hasil dan Pembahasan 69
g. Kesimpulan 69
h. Rubrik Penilaian 69

Acara 18 : FRAGMENT (WIDGET MODULAR) 71
a. Capaian Pembelajaran Mata Kuliah (CPMK) 71
b. Indikator 71
c. Dasar Teori 71
d. Alat dan Bahan 71
e. Prosedur Kerja 71
f. Hasil dan Pembahasan 74
g. Kesimpulan 75
h. Rubrik Penilaian 75

Acara 19 : FRAGMENT (WIDGET MODULAR) 76
a. Capaian Pembelajaran Mata Kuliah (CPMK) 76
b. Indikator 76
c. Dasar Teori 76
d. Alat dan Bahan 76
e. Prosedur Kerja 76

f. Hasil dan Pembahasan 77
g. Kesimpulan 77
h. Rubrik Penilaian 78

Acara 20 : FRAGMENT (WIDGET MODULAR) 79
a. Capaian Pembelajaran Mata Kuliah (CPMK) 79
b. Indikator 79
c. Dasar Teori 79
d. Alat dan Bahan 79
e. Prosedur Kerja 79
f. Hasil dan Pembahasan 79
g. Kesimpulan 79
h. Rubrik Penilaian 79

Acara 21 : INTENT (NAVIGATOR DAN URL LAUNCHER) 81
a. Capaian Pembelajaran Mata Kuliah (CPMK) 81
b. Indikator 81
c. Dasar Teori 81
d. Alat dan Bahan 82
e. Prosedur Kerja 82
f. Hasil dan Pembahasan 82
g. Kesimpulan 82
h. Rubrik Penilaian 82

Acara 22 : INTENT (NAVIGATOR DAN URL LAUNCHER) 84
a. Capaian Pembelajaran Mata Kuliah (CPMK) 84
b. Indikator 84
c. Dasar Teori 84
d. Alat dan Bahan 84
e. Prosedur Kerja 84
f. Hasil dan Pembahasan 86
g. Kesimpulan 86
h. Rubrik Penilaian 86

Acara 23 : INTENT (NAVIGATOR DAN URL LAUNCHER) 87
a. Capaian Pembelajaran Mata Kuliah (CPMK) 87
b. Indikator 87
c. Dasar Teori 87
d. Alat dan Bahan 87
e. Prosedur Kerja 87
f. Hasil dan Pembahasan 90
g. Kesimpulan 90
h. Rubrik Penilaian 90

Acara 24 : INTENT (NAVIGATOR DAN URL LAUNCHER) 91
a. Capaian Pembelajaran Mata Kuliah (CPMK) 91
b. Indikator 91
c. Dasar Teori 91
d. Alat dan Bahan 91
e. Prosedur Kerja 91
f. Hasil dan Pembahasan 91
g. Kesimpulan 91
h. Rubrik Penilaian 91

Acara 25 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE 93
a. Capaian Pembelajaran Mata Kuliah (CPMK) 93
b. Indikator 93
c. Dasar Teori 93
d. Alat dan Bahan 94
e. Prosedur Kerja 94
f. Hasil dan Pembahasan 94
g. Kesimpulan 94
h. Rubrik Penilaian 94

Acara 26 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE 95
a. Capaian Pembelajaran Mata Kuliah (CPMK) 95
b. Indikator 95
c. Dasar Teori 95
d. Alat dan Bahan 95
e. Prosedur Kerja 95
f. Hasil dan Pembahasan 100
g. Kesimpulan 100
h. Rubrik Penilaian 100

Acara 27 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE 102
a. Capaian Pembelajaran Mata Kuliah (CPMK) 102
b. Indikator 102
c. Dasar Teori 102
d. Alat dan Bahan 103
e. Prosedur Kerja 103
f. Hasil dan Pembahasan 103
g. Kesimpulan 104
h. Rubrik Penilaian 104

Acara 28 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE 105
a. Capaian Pembelajaran Mata Kuliah (CPMK) 105
b. Indikator 105
c. Dasar Teori 105
d. Alat dan Bahan 105
e. Prosedur Kerja 105
f. Hasil dan Pembahasan 115
g. Kesimpulan 115
h. Rubrik Penilaian 115

Acara 29 : SHARED PREFERENCES 116
a. Capaian Pembelajaran Mata Kuliah (CPMK) 116
b. Indikator Penilaian 116
c. Dasar Teori 116
d. Alat dan Bahan 116
e. Prosedur Kerja 116
f. Hasil dan Pembahasan 117
g. Kesimpulan 117
h. Rubrik Penilaian 117

Acara 30 : SHARED PREFERENCES 118
a. Capaian Pembelajaran Mata Kuliah (CPMK) 118
b. Indikator Penilaian 118
c. Dasar Teori 118
d. Alat dan Bahan 118
e. Prosedur Kerja 118
f. Hasil dan Pembahasan 120
g. Kesimpulan 120
h. Rubrik Penilaian 120

Acara 31 : SHARED PREFERENCES 121
a. Capaian Pembelajaran Mata Kuliah (CPMK) 121
b. Indikator Penilaian 121
c. Dasar Teori 121
d. Alat dan Bahan 121
e. Prosedur Kerja 121
f. Hasil dan Pembahasan 127
g. Kesimpulan 127
h. Rubrik Penilaian 127

Acara 32 : SHARED PREFERENCES 128
a. Capaian Pembelajaran Mata Kuliah (CPMK) 128
b. Indikator Penilaian 128

c. Dasar Teori 128
d. Alat dan Bahan 128
e. Prosedur Kerja 128
f. Hasil dan Pembahasan 128
g. Kesimpulan 128
h. Rubrik Penilaian 129

Acara 33 : GPS DAN AKSES SENSOR 130
a. Capaian Pembelajaran Mata Kuliah (CPMK) 130
b. Indikator Penilaian 130
c. Dasar Teori 130
d. Alat dan Bahan 130
e. Prosedur Kerja 130
f. Hasil dan Pembahasan 134
g. Kesimpulan 134
h. Rubrik Penilaian 135

Acara 34 : GPS DAN AKSES SENSOR 136
a. Capaian Pembelajaran Mata Kuliah (CPMK) 136
b. Indikator Penilaian 136
c. Dasar Teori 136
d. Alat dan Bahan 137
e. Prosedur Kerja 137
f. Hasil dan Pembahasan 141
g. Kesimpulan 141
h. Rubrik Penilaian 141

Acara 35 : GPS DAN AKSES SENSOR 142
a. Capaian Pembelajaran Mata Kuliah (CPMK) 142
b. Indikator Penilaian 142
c. Dasar Teori 142
d. Alat dan Bahan 143
e. Prosedur Kerja 143
f. Hasil dan Pembahasan 146
g. Kesimpulan 146
h. Rubrik Penilaian 146

Acara 36 : GPS DAN AKSES SENSOR 147
a. Capaian Pembelajaran Mata Kuliah (CPMK) 147
b. Indikator Penilaian 147
c. Dasar Teori 147
d. Alat dan Bahan 147
e. Prosedur Kerja 148
f. Hasil dan Pembahasan 155
g. Kesimpulan 155
h. Rubrik Penilaian 155

Acara 37 : PENGENALAN JSON 156
a. Capaian Pembelajaran Mata Kuliah (CPMK) 156
b. Indikator Penilaian 156
c. Dasar Teori 156
d. Alat dan Bahan 157
e. Prosedur Kerja 157
f. Hasil dan Pembahasan 159
g. Kesimpulan 159
h. Rubrik Penilaian 159

Acara 38 : PENGENALAN JSON 160
a. Capaian Pembelajaran Mata Kuliah (CPMK) 160
b. Indikator Penilaian 160
c. Dasar Teori 160
d. Alat dan Bahan 160
e. Prosedur Kerja 160

f. Hasil dan Pembahasan 164
g. Kesimpulan 164
h. Rubrik Penilaian 165

Acara 39 : PENGENALAN JSON 166
a. Capaian Pembelajaran Mata Kuliah (CPMK) 166
b. Indikator Penilaian 166
c. Dasar Teori 166
d. Alat dan Bahan 166
e. Prosedur Kerja 166
f. Hasil dan Pembahasan 170
g. Kesimpulan 170
h. Rubrik Penilaian 170

Acara 40 : PENGENALAN JSON 172
a. Capaian Pembelajaran Mata Kuliah (CPMK) 172
b. Indikator Penilaian 172
c. Dasar Teori 172
d. Alat dan Bahan 172
e. Prosedur Kerja 172
f. Hasil dan Pembahasan 173
g. Kesimpulan 173
h. Rubrik Penilaian 173

Acara 41 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST 174
a. Capaian Pembelajaran Mata Kuliah (CPMK) 174
b. Indikator Penilaian 174
c. Dasar Teori 174
d. Alat dan Bahan 175
e. Prosedur Kerja 175
f. Hasil dan Pembahasan 185
g. Kesimpulan 185
h. Rubrik Penilaian 185

Acara 42 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST 186
a. Capaian Pembelajaran Mata Kuliah (CPMK) 186
b. Indikator Penilaian 186
c. Dasar Teori 186
d. Alat dan Bahan 186
e. Prosedur Kerja 187
f. Hasil dan Pembahasan 188
g. Kesimpulan 188
h. Rubrik Penilaian 189

Acara 43 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST 190
a. Capaian Pembelajaran Mata Kuliah (CPMK) 190
b. Indikator Penilaian 190
c. Dasar Teori 190
d. Alat dan Bahan 190
e. Prosedur Kerja 190
f. Hasil dan Pembahasan 195
g. Kesimpulan 195
h. Rubrik Penilaian 195

Acara 44 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST 196
a. Capaian Pembelajaran Mata Kuliah (CPMK) 196
b. Indikator Penilaian 196
c. Dasar Teori 196
d. Alat dan Bahan 196
e. Prosedur Kerja 196
f. Hasil dan Pembahasan 200
g. Kesimpulan 201
h. Rubrik Penilaian 201

Acara 45 : PROJECT FINAL 202
a. Capaian Pembelajaran Mata Kuliah (CPMK) 202
b. Indikator Penilaian 202
c. Dasar Teori 202
d. Alat dan Bahan 203
e. Prosedur Kerja 203
f. Hasil dan Pembahasan 204
g. Kesimpulan 204
h. Rubrik Penilaian 204

Acara 46 : PROJECT FINAL 205
a. Capaian Pembelajaran Mata Kuliah (CPMK) 205
b. Indikator Penilaian 205
c. Dasar Teori 205
d. Alat dan Bahan 206
e. Prosedur Kerja 206
f. Hasil dan Pembahasan 206
g. Kesimpulan 206
h. Rubrik Penilaian 206

Acara 47 : PROJECT FINAL 207
a. Capaian Pembelajaran Mata Kuliah (CPMK) 207
b. Indikator Penilaian 207
c. Dasar Teori 207
d. Alat dan Bahan 208
e. Prosedur Kerja 208
f. Hasil dan Pembahasan 208
g. Kesimpulan 208
h. Rubrik Penilaian 208

Acara 48 : PROJECT FINAL 209
a. Capaian Pembelajaran Mata Kuliah (CPMK) 209
b. Indikator Penilaian 209
c. Dasar Teori 209
d. Alat dan Bahan 209
e. Prosedur Kerja 209
f. Hasil dan Pembahasan 210
g. Kesimpulan 210
h. Rubrik Penilaian 210

Acara 49 : PROJECT FINAL 211
a. Capaian Pembelajaran Mata Kuliah (CPMK) 211
b. Indikator Penilaian 211
c. Dasar Teori 211
d. Alat dan Bahan 212
e. Prosedur Kerja 212
f. Hasil dan Pembahasan 212
g. Kesimpulan 212
h. Rubrik Penilaian 212

Acara 50 : PROJECT FINAL 213
a. Capaian Pembelajaran Mata Kuliah (CPMK) 213
b. Indikator Penilaian 213
c. Dasar Teori 213
d. Alat dan Bahan 213
e. Prosedur Kerja 214
f. Hasil dan Pembahasan 214
g. Kesimpulan 214
h. Rubrik Penilaian 214

Acara 51 : PROJECT FINAL 215
a. Capaian Pembelajaran Mata Kuliah (CPMK) 215
b. Indikator Penilaian 215

c. Dasar Teori 215
d. Alat dan Bahan 216
e. Prosedur Kerja 216
f. Hasil dan Pembahasan 216
g. Kesimpulan 216
h. Rubrik Penilaian 216

Acara 52 : PROJECT FINAL 217
a. Capaian Pembelajaran Mata Kuliah (CPMK) 217
b. Indikator Penilaian 217
c. Dasar Teori 217
d. Alat dan Bahan 218
e. Prosedur Kerja 218
f. Hasil dan Pembahasan 218
g. Kesimpulan 218
h. Rubrik Penilaian 218

Acara 53 : PROJECT FINAL 219
a. Capaian Pembelajaran Mata Kuliah (CPMK) 219
b. Indikator Penilaian 219
c. Dasar Teori 219
d. Alat dan Bahan 219
e. Prosedur Kerja 220
f. Hasil dan Pembahasan 220
g. Kesimpulan 220
h. Rubrik Penilaian 220

Acara 54 : PROJECT FINAL 221
a. Capaian Pembelajaran Mata Kuliah (CPMK) 221
b. Indikator Penilaian 221
c. Dasar Teori 221
d. Alat dan Bahan 221
e. Prosedur Kerja 222
f. Hasil dan Pembahasan 222
g. Kesimpulan 222
h. Rubrik Penilaian 222

Acara 55 : PROJECT FINAL 223
a. Capaian Pembelajaran Mata Kuliah (CPMK) 223
b. Indikator Penilaian 223
c. Dasar Teori 223
d. Alat dan Bahan 223
e. Prosedur Kerja 224
f. Hasil dan Pembahasan 224
g. Kesimpulan 224
h. Rubrik Penilaian 224

Acara 56 : PROJECT FINAL 225
a. Capaian Pembelajaran Mata Kuliah (CPMK) 225
b. Indikator Penilaian 225
c. Dasar Teori 225
d. Alat dan Bahan 225
e. Prosedur Kerja 226
f. Hasil dan Pembahasan 226
g. Kesimpulan 226
h. Rubrik Penilaian 226

Acara 1 : PENGENALAN DAN WORKFLOW PADA GIT

Pokok Bahasan	:	Pengenalan Git
Acara Praktikum/Praktik	:	Minggu 1 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep git.
2. Mahasiswa mampu menerapkan git pada pengelolaan source code project Flutter.

b. Indikator Penilaian

Kemampuan mahasiswa dalam memahami konsep dasar git.

c. Dasar Teori

Pengenalan Git

Git adalah *version control system* (VCS) terdistribusi yang digunakan untuk mencatat setiap perubahan (revisi) pada source code. Dalam pengembangan aplikasi Flutter, Git dipakai untuk menyimpan riwayat perubahan file Dart (folder `lib/`), file konfigurasi `pubspec.yaml`, aset, serta folder platform (`android/`, `ios/`). Dengan Git, pengembang dapat kembali ke versi sebelumnya ketika terjadi kesalahan, bekerja secara paralel dengan anggota tim, serta mengunggah project ke layanan repositori seperti GitHub, GitLab atau Bitbucket.

Instalasi Git di Windows tidak seperti di Linux yang cukup mengetikkan perintah langsung terinstal. Kita harus mengunduh installer terlebih dahulu, kemudian melakukan langkah next → next → finish. Dalam proses tersebut terdapat pilihan yang harus diperhatikan agar perintah `git` dapat dikenali di CMD/PowerShell maupun di terminal Android Studio dan VS Code.

Terdapat tiga kondisi file dalam Git:

1) **Modified**

Modified adalah kondisi di mana revisi atau perubahan sudah dilakukan, tetapi belum ditandai dan belum disimpan di version control. Contohnya ketika kita mengubah isi file `lib/main.dart`, maka file tersebut berada pada kondisi modified.

2) **Staged**

Staged adalah kondisi di mana revisi sudah ditandai, tetapi belum disimpan di version control. Untuk mengubah kondisi file dari modified ke staged gunakan perintah `git add nama_file`. Contoh:

```bash
git add lib/main.dart
```

3) **Committed**

Committed adalah kondisi di mana revisi sudah disimpan di version control. Perintah untuk mengubah kondisi file dari staged ke committed adalah `git commit`.

Workflow Git

Alur kerja dasar Git dapat digambarkan sebagai berikut: *Working Directory* (modified) → `git add` → *Staging Area* (staged) → `git commit` → *Local Repository* (committed) → `git push` → *Remote Repository* (GitHub/GitLab).

Pada project Flutter, beberapa folder hasil build tidak perlu disimpan ke Git (misalnya `build/`, `.dart_tool/`). Daftar file yang diabaikan sudah dibuat otomatis oleh perintah `flutter create` di dalam file `.gitignore`.

d. Alat dan Bahan

1. Kertas Folio
2. Spidol

e. Prosedur Kerja

Mahasiswa memperhatikan penjelasan dosen pengampu mata kuliah Workshop Aplikasi Mobile berbasis Flutter.

1. Mahasiswa menentukan kelompok dengan 4–5 anggota.
2. Membuat resume secara individu tentang konsep dasar dari pengenalan dan workflow git pada kertas folio.
3. Diskusikan dengan kelompok hasil resume yang telah dikerjakan secara individu, dan salin ke dalam PPT.
4. Presentasikan hasil resume di depan kelas setiap kelompok secara bergantian.

f. Hasil dan Pembahasan

1. Dokumentasi tugas berupa laporan (definisi, konsep dasar pengenalan dan workflow git).
2. Dokumentasi tugas berupa file PPT.

g. Kesimpulan

Mahasiswa memahami definisi git, konsep dasar pengenalan dan workflow git.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan dari tugas	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan tugas	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan tugas	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 2 : PENGENALAN DAN WORKFLOW PADA GIT

Pokok Bahasan	:	Instalasi Git
Acara Praktikum/Praktik	:	Minggu 1 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu melakukan instalasi Git pada masing-masing PC.
2. Mahasiswa mampu melakukan konfigurasi.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam melakukan tahap-tahap instalasi git, sampai dengan mampu melakukan konfigurasi awal untuk akun git setiap individu.

c. Dasar Teori

Git merupakan perangkat wajib bagi pengembang Flutter. Selain untuk version control, Flutter SDK sendiri didistribusikan dan diperbarui menggunakan Git (perintah `flutter upgrade` dan `flutter channel` memanfaatkan Git di belakang layar). Karena itu Git harus terpasang dan dapat dipanggil dari terminal sebelum Flutter SDK diinstal pada Acara 5.

d. Alat dan Bahan

1. Kertas A4
2. Spidol
3. Jaringan Internet

e. Prosedur Kerja

Instalasi Git

1. Download Git. Silakan buka website resmi Git (git-scm.com).
2. Kemudian unduh Git sesuai dengan arsitektur komputer yang digunakan. Kalau menggunakan 64 bit, unduh yang 64 bit.
3. Setelah file terunduh, klik 2x file installer Git yang sudah diunduh. Maka akan muncul informasi lisensi Git, klik **Next >** untuk melanjutkan.
4. Selanjutnya menentukan lokasi instalasi. Biarkan saja apa adanya, kemudian klik **Next >**.
5. Selanjutnya pemilihan komponen, biarkan saja seperti default kemudian klik **Next >**.
6. Selanjutnya pemilihan direktori start menu, klik **Next >**.
7. Selanjutnya pengaturan PATH Environment. Pilih opsi yang tengah (*Git from the command line and also from 3rd-party software*) agar perintah git dapat dikenali di

Command Prompt (CMD), PowerShell, serta terminal Android Studio/VS Code. Setelah itu klik **Next >**.

8. Selanjutnya konversi line ending. Biarkan saja seperti default, kemudian klik **Next >**.
9. Selanjutnya pemilihan emulator terminal. Pilih saja yang bawah, lalu klik **Next >**.
10. Selanjutnya pemilihan opsi ekstra. Klik saja **Next >**.
11. Selanjutnya pemilihan opsi eksperimental, langsung saja klik **Install** untuk instalasi.
12. Tunggu beberapa saat, instalasi sedang dilakukan.
13. Setelah selesai, klik **Finish**.
14. Git sudah terinstal di Windows. Untuk mencobanya, buka CMD atau PowerShell, kemudian ketik perintah:

```bash
git --version
```

Konfigurasi Awal

Ada beberapa konfigurasi yang harus dipersiapkan sebelum mulai menggunakan Git, seperti name dan email.

1. Lakukan konfigurasi dengan perintah berikut ini.

```bash
git config --global user.name "developer"
git config --global user.email developer@polije.ac.id
```

2. Kemudian periksa konfigurasinya dengan perintah:

```bash
git config --list
```

Apabila nilai `user.name` dan `user.email` tampil pada daftar, berarti konfigurasi berhasil.

- Konfigurasi `core.editor` bersifat opsional, sedangkan name dan email wajib.
- Username dan email harus mengikuti akun GitHub, GitLab, Bitbucket atau yang lainnya, agar mudah dalam mengintegrasikan.

f. Hasil dan Pembahasan

1. Dokumentasi tugas berupa laporan secara individu tahap instalasi git yang dibuktikan dengan screenshot pada masing-masing PC pada setiap prosesnya.
2. Dokumentasi hasil konfigurasi berupa akun git pribadi.

g. Kesimpulan

Mahasiswa berhasil menginstal Git pada masing-masing PC serta dilanjutkan melakukan konfigurasi dengan menggunakan akun individu.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam instalasi serta konfigurasi git ditunjang dengan bukti screenshot setiap proses	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam instalasi serta konfigurasi git	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam instalasi serta konfigurasi git	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam instalasi serta konfigurasi git	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 3 : PENGENALAN DAN WORKFLOW PADA GIT

Pokok Bahasan	:	Membuat Revisi Pertama pada Git
Acara Praktikum/Pertemuan	:	Minggu 1 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu melakukan revisi pertama.
2. Mahasiswa mampu menjelaskan maksud command git revisi pertama.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam melakukan revisi pertama pada git, serta memahami setiap command git revisi pertama.

c. Dasar Teori

Sebuah folder baru dapat dijadikan repositori Git dengan perintah `git init`. Setelah itu setiap file di dalamnya dapat dipantau perubahannya. Perintah `git status` digunakan untuk melihat kondisi file (untracked, modified, staged), sedangkan `git add` dan `git commit` digunakan untuk menyimpan revisi.

d. Alat dan Bahan

1. Kertas A4
2. Spidol
3. Jaringan Internet

e. Prosedur Kerja

1. Buat folder project baru bernama `belajar-git`, lalu buka terminal di folder tersebut dan jadikan repositori Git.

```bash
mkdir belajar-git
cd belajar-git
git init
```

2. Buat tiga file HTML kosong: `index.html`, `about.html` dan `contact.html`, kemudian ketik `git status`. Ketiga file akan berstatus *untracked/modified*.
3. Ubah kondisi tiga file HTML tadi menjadi staged dengan perintah `git add`.

```bash
git add index.html
git add about.html
git add contact.html
```

atau bisa melakukannya seperti ini:

```bash
git add index.html about.html contact.html
```

atau:

```bash
git add *.html
```

atau seperti ini (semua file dan direktori):

```bash
git add .
```

4. Selanjutnya, ketik perintah `git status` lagi. Kondisi filenya sekarang akan menjadi staged.
5. Lalu, ubah kondisi file tersebut ke committed agar semua perubahan disimpan oleh Git.

```bash
git commit -m "Commit pertama"
```

6. Setelah itu, cek dengan perintah `git status` lagi.
7. Revisi pertama sudah dibuat. Selanjutnya cobalah untuk membuat revisi kedua.

> Catatan: langkah yang sama berlaku pada project Flutter. Jalankan `git init` di folder project Flutter, lalu `git add .` dan `git commit -m "Initial Flutter project"`. File di dalam `.gitignore` (misalnya `build/` dan `.dart_tool/`) otomatis tidak ikut di-commit.

f. Hasil dan Pembahasan

1. Dokumentasi tugas berupa laporan secara individu tahap revisi pertama pada git yang dibuktikan dengan screenshot pada masing-masing pekerjaannya.
2. Penjelasan pada laporan maksud dan fungsi setiap command yang digunakan.

g. Kesimpulan

Mahasiswa berhasil melakukan revisi pertama dan paham maksud command git yang digunakan pada revisi pertama.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam melakukan revisi pertama serta menjelaskan secara detail setiap command git	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam melakukan revisi pertama serta menjelaskan secara singkat setiap command git	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam melakukan revisi pertama serta menjelaskan command git	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam melakukan revisi pertama serta menjelaskan command git	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 4 : PENGENALAN DAN WORKFLOW PADA GIT

Pokok Bahasan	:	Membuat Revisi Kedua pada Git
Acara Praktikum/Pertemuan	:	Minggu 1 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu melakukan revisi kedua.
2. Mahasiswa mampu menjelaskan maksud command git revisi kedua.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam melakukan revisi kedua pada git, serta memahami setiap command git revisi kedua.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Spidol
3. Jaringan Internet

e. Prosedur Kerja

Ceritanya ada perubahan yang akan kita lakukan pada file `index.html`.

1. Modifikasi isi file `index.html`. Sebagai contoh diisikan seperti ini.

```HTML
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Belajar Git - Project 01</title>
</head>
<body>
  <p>Hello Semua, Saya sedang belajar Git</p>
</body>
</html>
```

2. Setelah itu ketik lagi perintah `git status`. Terlihat di sana, file `index.html` sudah dimodifikasi. Kondisinya sekarang berada dalam modified.
3. Lakukan commit lagi seperti revisi pertama.

```BASH
git add index.html
git commit -m "ditambahkan isi"
```

- Dengan demikian, revisi kedua sudah disimpan oleh Git.

- Argumen `-m` untuk menambahkan pesan setiap menyimpan revisi.

4. Sekarang Git sudah mencatat dua revisi yang sudah dilakukan. Lihat riwayatnya dengan perintah:

```bash
git log --oneline
```

Revisi-revisi ini dapat diibaratkan sebagai *checkpoint* pada game. Apabila nanti ada kesalahan, kita bisa kembali ke checkpoint ini.

5. (Pengayaan) Hubungkan repositori lokal ke GitHub agar dapat dipakai bersama kelompok pada project Flutter berikutnya:

```bash
git branch -M main
git remote add origin https://github.com/<username>/belajar-git.git
git push -u origin main
```

f. Hasil dan Pembahasan

1. Dokumentasi tugas berupa laporan secara individu tahap revisi kedua pada git yang dibuktikan dengan screenshot pada masing-masing pekerjaannya.
2. Penjelasan pada laporan maksud dan fungsi setiap command yang digunakan.

g. Kesimpulan

Mahasiswa berhasil melakukan revisi kedua dan paham maksud command git yang digunakan pada revisi kedua.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam melakukan revisi kedua serta menjelaskan secara detail setiap command git	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam melakukan revisi kedua serta menjelaskan secara singkat setiap command git	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam melakukan revisi kedua serta menjelaskan command git	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam melakukan revisi kedua serta menjelaskan command git	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 5 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO

Pokok Bahasan	:	Instalasi Flutter SDK dan Android Studio
Acara Praktikum/Pertemuan	:	Minggu 2 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu melakukan instalasi Flutter SDK dan Android Studio.
2. Mahasiswa mampu melakukan *setting up* toolchain Flutter (Android SDK, plugin Flutter/Dart, dan `flutter doctor`).

b. Indikator Penilaian

Keberhasilan mahasiswa melakukan instalasi Flutter SDK dan Android Studio serta *setting up* toolchain hingga perintah `flutter doctor` tidak menampilkan error untuk platform Android.

c. Dasar Teori

Flutter adalah *framework* UI open source buatan Google untuk membangun aplikasi yang dikompilasi secara native untuk Android, iOS, web dan desktop dari **satu basis kode**. Bahasa pemrograman yang digunakan adalah **Dart**. Seluruh tampilan pada Flutter disusun dari **widget**, dan Flutter menggambar sendiri tampilannya menggunakan mesin render (Impeller/Skia) sehingga tampilan aplikasi konsisten di semua platform.

Untuk membuat aplikasi Android dengan Flutter tetap diperlukan **Android Studio**, karena di dalamnya terdapat Android SDK, *build tools*, Gradle, dan emulator (AVD). Android Studio juga dapat dipakai sebagai editor kode Flutter setelah plugin Flutter dan Dart dipasang (alternatifnya adalah Visual Studio Code).

Beberapa hal yang perlu diperhatikan sebelum instalasi:

1. RAM minimal 8 GB (direkomendasikan 16 GB agar emulator berjalan lancar).
2. Penyimpanan kosong minimal 10 GB (Flutter SDK ± 2,5 GB, Android Studio dan Android SDK ± 4 GB, image emulator ± 2 GB).
3. Resolusi layar minimum 1280 x 800.
4. Git sudah terinstal (Acara 2), karena Flutter SDK membutuhkan Git.

d. Alat dan Bahan

1. Laptop minimal RAM 8 GB
2. Jaringan Internet

e. Prosedur Kerja

Cara instalasi terdiri dari tiga langkah utama, yaitu:

- Setup Android Studio
- Install Android SDK
- Install dan konfigurasi Flutter SDK

1. Setup Android Studio

1.1. **Download Android Studio.** Buka https://developer.android.com/studio lalu unduh installer Android Studio.

1.2. **Install Android Studio.** Setelah selesai download, buka file tersebut dan klik **Next** untuk melanjutkan ke proses instalasi.

1.3. **Pilih Komponen Tambahan.** Centang *Android Virtual Device* (AVD). AVD berfungsi untuk mengonfigurasi perangkat yang dijalankan dengan emulator Android. Klik **Next**.

1.4. **Tentukan Lokasi Instalasi.** Misalnya `C:\Program Files\Android\Android Studio`. Klik **Next**.

1.5. **Tentukan Nama Aplikasi pada Start Menu.** Sebaiknya gunakan nama Android Studio saja. Klik **Install**.

1.6. **Mulai Proses Instalasi.** Tunggu hingga proses selesai, lalu klik **Finish**.

2. Install Android SDK

SDK adalah seperangkat alat dan program perangkat lunak yang digunakan oleh pengembang untuk membuat aplikasi untuk platform tertentu. Pastikan perangkat terhubung ke internet karena komponen SDK akan diunduh.

2.1. Buka Android Studio. Pada *Setup Wizard* klik **Next**.

2.2. **Pilih Tipe Instalasi.** *Standard* (pengaturan default) atau *Custom* (memilih komponen sendiri). Pada praktikum ini pilih **Custom**, lalu klik **Next**.

2.3. **Pilih Tema.** Darcula (gelap) atau Light (terang). Klik **Next**.

2.4. **Pilih Komponen SDK.** Centang Android SDK, Android SDK Platform (versi terbaru), Performance (Intel HAXM/Hypervisor Driver) dan Android Virtual Device. Klik **Next**.

2.5. **Tentukan RAM untuk Emulator.** Sebaiknya minimal 2–4 GB. Klik **Next**.

2.6. **Review Pengaturan Instalasi**, lalu klik **Finish** untuk memulai proses download.

2.7. Tunggu proses download dan instalasi komponen SDK hingga selesai. Pastikan koneksi internet stabil.

2.8. Setelah masuk ke halaman *Welcome*, buka **More Actions → SDK Manager → tab SDK Tools**, lalu centang **Android SDK Command-line Tools (latest)** dan klik **Apply**. Komponen ini wajib untuk Flutter.

3. Install dan Konfigurasi Flutter SDK

3.1. Buka https://docs.flutter.dev/get-started/install, pilih sistem operasi **Windows** dan target **Android**, lalu unduh file ZIP Flutter SDK versi *stable* terbaru.

3.2. Ekstrak ZIP ke folder tanpa spasi dan tanpa hak akses khusus, misalnya `C:\src\flutter` (jangan di `C:\Program Files`).

3.3. Tambahkan `C:\src\flutter\bin` ke variabel **Path**: buka *Edit the system environment variables* → *Environment Variables* → *User variables* → *Path* → *Edit* → *New*, isikan `C:\src\flutter\bin`, lalu **OK**.

3.4. Tutup lalu buka kembali PowerShell, dan cek versi Flutter:

```bash
flutter --version
```

3.5. Setujui lisensi Android SDK:

```bash
flutter doctor --android-licenses
```

Ketik `y` untuk setiap lisensi yang ditampilkan.

3.6. Pasang plugin Flutter pada Android Studio: **More Actions** → **Plugins** → **Marketplace**, cari **Flutter**, klik **Install** (plugin Dart ikut terpasang otomatis), lalu **Restart IDE**.

3.7. Jalankan pemeriksaan toolchain:

```bash
flutter doctor
```

Hasil yang diharapkan adalah tanda centang (✓) pada **Flutter**, **Android toolchain** dan **Android Studio**. Bagian *Visual Studio* hanya diperlukan untuk membuat aplikasi desktop Windows dan dapat diabaikan.

> Pada project Android native, pengaturan build dilakukan melalui Gradle secara langsung. Pada Flutter, Gradle tetap digunakan di dalam folder `android/`, tetapi dijalankan otomatis oleh perintah `flutter run` dan `flutter build apk`. Pengaturan dependensi/library Flutter dilakukan melalui file `pubspec.yaml`.

f. Hasil dan Pembahasan

Perangkat lunak Flutter SDK dan Android Studio terinstal di laptop masing-masing mahasiswa, dibuktikan dengan hasil `flutter doctor`.

g. Kesimpulan

Mahasiswa berhasil menginstal Flutter SDK dan Android Studio serta melakukan setting up toolchain Flutter.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam instalasi Flutter SDK dan Android Studio, hasil flutter doctor tanpa error	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam instalasi Flutter SDK dan Android Studio, terdapat peringatan minor pada flutter doctor	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam instalasi Flutter SDK dan Android Studio	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam instalasi Flutter SDK dan Android Studio	1	2	3	4

No	Indikator	Skor
Jumlah Skor		

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 6 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO

Pokok Bahasan	:	Instalasi Flutter SDK dan Android Studio
Acara Praktikum/Pertemuan	:	Minggu 2 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu melakukan instalasi Flutter SDK dan Android Studio.
2. Mahasiswa mampu melakukan setting up toolchain Flutter (Android SDK, emulator, dan `flutter doctor`).

b. Indikator Penilaian

Keberhasilan mahasiswa melakukan instalasi dan setting up toolchain Flutter pada Android Studio.

c. Dasar Teori

Android Virtual Device (AVD) adalah emulator yang mensimulasikan perangkat Android di komputer. Aplikasi Flutter juga dapat dijalankan langsung di HP Android melalui kabel USB dengan mengaktifkan **Developer Options → USB Debugging**. Perintah `flutter devices` menampilkan semua perangkat (emulator maupun HP) yang siap dipakai.

d. Alat dan Bahan

1. Laptop minimal RAM 8 GB
2. Jaringan Internet
3. Kabel Data USB

e. Prosedur Kerja

1. Buat emulator melalui **Device Manager → Create Virtual Device**, pilih perangkat (misalnya Pixel 7), pilih *system image* versi terbaru, lalu **Finish**.
2. Jalankan emulator, kemudian ketik perintah berikut untuk memastikan emulator terdeteksi oleh Flutter:

```bash
flutter devices
flutter emulators
```

3. (Opsional) Hubungkan HP Android dengan kabel USB, aktifkan USB Debugging, lalu pastikan HP muncul pada `flutter devices`.
4. Mahasiswa membuat laporan dokumentasi dengan penjelasan sedetail mungkin dalam proses instalasi Flutter SDK, Android Studio, plugin Flutter/Dart, emulator, serta hasil `flutter doctor -v`.

f. Hasil dan Pembahasan

Mahasiswa berhasil membuat dokumentasi dalam bentuk laporan instalasi Flutter SDK dan Android Studio.

g. Kesimpulan

Mahasiswa berhasil membuat dokumentasi dalam bentuk laporan instalasi Flutter SDK dan Android Studio.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam membuat laporan instalasi dengan penjelasan detail disertai screenshot setiap proses	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam membuat laporan instalasi dengan penjelasan singkat	1	2	3	4
3	Ketepatan waktu akan tetapi laporan instalasi kurang lengkap	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam membuat laporan instalasi	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 7 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO

Pokok Bahasan	:	Pengenalan Flutter dan Android Studio
Acara Praktikum/Pertemuan	:	Minggu 2 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu membuat project Flutter pertama di Android Studio.
2. Mahasiswa mampu mengenal beberapa komponen yang ada di Android Studio dan struktur project Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa membuat project Flutter pertama di Android Studio, dan mengenal beberapa komponennya.

c. Dasar Teori

Antarmuka Android Studio

Tools — alat-alat yang sering digunakan dalam development seperti copy/paste, build, menjalankan aplikasi (*Run*), *Hot Reload*, hingga menjalankan emulator.

Navigasi — membantu melihat struktur kedalaman (*depth*) dan posisi file project yang sedang dibuka.

Project Explorer dan Editor — bagian utama IDE tempat kita menuliskan kode. Sebelah kiri adalah struktur project dan sebelah kanan adalah editor.

Tool Window Bar — menu di sekeliling editor yang dapat di-*expand*, misalnya *Flutter Inspector*, *Flutter Outline*, *Run*, *Terminal* dan *Dart Analysis*.

Status Bar — terletak di bagian terbawah, menampilkan status project dan pesan peringatan (*warning message*) apabila ada.

Struktur Project Flutter

Setiap project Flutter memiliki struktur folder sebagai berikut:

Folder / File	Fungsi
lib/	Berisi source code Dart. File awal aplikasi adalah lib/main.dart (fungsi main() dan runApp()).
pubspec.yaml	Konfigurasi project: nama, versi, dependensi (library/package), dan deklarasi aset (gambar, font, audio).
android/	Project Android native yang dibungkus Flutter. Di dalamnya terdapat AndroidManifest.xml (izin/permission, nama aplikasi) dan konfigurasi Gradle.
ios/, web/, windows/, linux/, macos/	Folder untuk platform lain (dibuat sesuai pilihan platform).

Folder / File	Fungsi
test/	Berisi kode pengujian (widget test dan unit test).
assets/ (dibuat sendiri)	Tempat menyimpan gambar, file JSON, audio dan aset lain yang didaftarkan di pubspec.yaml.

Perbandingan dengan project Android native: **Manifest** tetap ada di `android/app/src/main/AndroidManifest.xml`; folder **java/kotlin** digantikan oleh folder `lib/` yang berisi kode Dart; folder **res** (layout XML) digantikan oleh widget yang ditulis langsung dalam kode Dart; dan **Gradle** untuk manajemen library digantikan oleh `pubspec.yaml` (Gradle tetap dipakai secara internal saat build Android).

Widget

Pada Flutter, *everything is a widget*. Terdapat dua jenis widget utama:

- **StatelessWidget** — widget yang tampilannya tidak berubah setelah dibuat (misalnya teks judul, ikon).
- **StatefulWidget** — widget yang memiliki *state* (data) yang dapat berubah, dan tampilan diperbarui dengan memanggil `setState()`.

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

Membuat Project Flutter Pertama

1. Jalankan Android Studio. Pada halaman Welcome pilih **New Flutter Project**.
2. Pilih **Flutter** di panel kiri, pastikan **Flutter SDK path** mengarah ke folder instalasi (misalnya `C:\src\flutter`), lalu klik **Next**.
3. Isi **Project name** dengan huruf kecil dan garis bawah, misalnya `hello_polije`. Tentukan **Project location**.
4. **Organization** diisi seperti domain terbalik, misalnya `ac.id.polije`. Organization digunakan sebagai identitas (*package name/applicationId*) ketika aplikasi dipublikasikan.
5. Pilih **Platforms**: centang Android (dan platform lain bila diperlukan). Klik **Create**.
6. Cara lain melalui terminal:

```bash
flutter create --org ac.id.polije hello_polije
cd hello_polije
flutter run
```

7. Buka `lib/main.dart`, ganti seluruh isinya dengan kode berikut:

```dart
import 'package:flutter/material.dart';
```

```
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello Polije',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const CounterPage(),
    );
  }
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _counter = 0;

  void _tambah() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Project Flutter Pertama')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('Tombol sudah ditekan sebanyak:'),
            Text('$_counter',
                style: Theme.of(context).textTheme.headlineMedium),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _tambah,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

8. Pilih emulator pada *device selector* di toolbar, kemudian klik **Run** (Shift + F10).
9. Ubah teks pada kode, simpan (Ctrl + S), dan amati fitur **Hot Reload** yang langsung memperbarui tampilan tanpa menjalankan ulang aplikasi.

Antarmuka Android Studio untuk Flutter

Setelah project dibuat, perhatikan tampilan penuh IDE Android Studio yang berbasis IntelliJ IDEA: Project Explorer, Editor, Flutter Inspector, Run console dan Terminal. Tampilan dapat sedikit berbeda karena perbedaan konfigurasi dan versi Android Studio.

f. Hasil dan Pembahasan

Project Flutter pertama yang telah dibuat dan berhasil dijalankan pada emulator/HP.

g. Kesimpulan

Mahasiswa berhasil membuat project Flutter pertama pada Android Studio.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam membuat project Flutter pertama serta menjelaskan struktur project secara detail	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam membuat project Flutter pertama serta menjelaskan struktur project secara singkat	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam membuat project Flutter pertama	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam membuat project Flutter pertama	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 8 : PENGENALAN DAN INSTALASI FLUTTER SDK DAN ANDROID STUDIO

Pokok Bahasan : Pengenalan Flutter dan Android Studio
Acara Praktikum/Pertemuan : Minggu 2 / 4
Tempat : Daring / Luring Politeknik Negeri Jember
Alokasi Waktu : 170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu membuat project Flutter pertama di Android Studio.
2. Mahasiswa mampu mengenal beberapa komponen yang ada di Android Studio dan struktur project Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa membuat project Flutter pertama di Android Studio, dan mengenal beberapa komponennya.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

Mahasiswa membuat laporan dokumentasi dengan penjelasan dalam pembuatan project Flutter pertama pada Android Studio, meliputi: langkah pembuatan project, penjelasan struktur folder (`lib/`, `pubspec.yaml`, `android/`, `test/`), penjelasan kode `main.dart` (StatelessWidget, StatefulWidget, `setState`), serta screenshot hasil Run dan Hot Reload.

f. Hasil dan Pembahasan

Dokumentasi dalam bentuk laporan hasil pembuatan project Flutter pertama pada Android Studio.

g. Kesimpulan

Mahasiswa berhasil membuat dokumentasi dalam bentuk laporan pembuatan project Flutter pertama pada Android Studio.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam membuat laporan project pertama dengan penjelasan detail setiap baris code	1	2	3	4

No	Indikator	Skor			
2	Ketepatan waktu dan ketepatan dalam membuat laporan project pertama dengan penjelasan singkat	1	2	3	4
3	Ketepatan waktu akan tetapi laporan project pertama kurang lengkap	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam membuat laporan project pertama	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 9 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW

Pokok Bahasan	:	Linear Layout (Column & Row) dan Relative Layout (Stack & Align)
Acara Praktikum/Pertemuan	:	Minggu 3 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep linear layout (Column dan Row) pada Flutter.
2. Mahasiswa mampu memahami konsep relative layout (Stack, Align dan Positioned) pada Flutter.
3. Mahasiswa mampu menerapkan konsep linear layout.
4. Mahasiswa mampu menerapkan konsep relative layout.

b. Indikator Penilaian

1. Keberhasilan mahasiswa dalam menerapkan linear layout pada studi kasus.
2. Keberhasilan mahasiswa dalam menerapkan relative layout pada studi kasus.

c. Dasar Teori

Layout

Layout merupakan tata letak tampilan untuk mengatur penempatan teks, gambar, ataupun komponen lainnya sehingga tampilan aplikasi terlihat rapi dan nyaman dilihat oleh pengguna. Pada Flutter, layout tidak ditulis dalam file XML seperti Android native, melainkan disusun dari **widget layout** yang membungkus widget lain (*child*/*children*) langsung di dalam kode Dart.

Prinsip dasar layout Flutter adalah: *constraints go down, sizes go up, parent sets position* — widget induk memberikan batasan ukuran kepada anaknya, anak menentukan ukurannya sendiri dalam batasan tersebut, lalu induk menentukan posisi anak.

1) Linear Layout — Column dan Row

Linear layout menyejajarkan semua child dalam satu arah. Di Flutter digunakan widget **Column** (arah vertikal) dan **Row** (arah horizontal), setara dengan `LinearLayout` dengan atribut `android:orientation="vertical"` atau `"horizontal"`.

LinearLayout (Android)	Column / Row (Flutter)
android:orientation="vertical"	Column
android:orientation="horizontal"	Row
android:gravity	mainAxisAlignment & crossAxisAlignment
android:layout_weight="1"	Expanded(flex: 1, child: ...)
android:layout_margin / padding	Padding atau SizedBox sebagai jarak

LinearLayout (Android)	Column / Row (Flutter)
android:layout_gravity="right"	Align(alignment: Alignment.centerRight)

Semua anak Column akan ditumpuk satu per satu dari atas ke bawah, dan anak Row disusun dari kiri ke kanan. `mainAxisAlignment` mengatur posisi pada sumbu utama, sedangkan `crossAxisAlignment` mengatur posisi pada sumbu silang.

2) Relative Layout — Stack, Align dan Positioned

Relative layout menempatkan widget secara lebih bebas: sebuah widget dapat berada di atas/bawah/samping widget lain atau menempel pada sisi induknya. Pada Flutter, perilaku ini dicapai dengan:

- **Stack** — menumpuk beberapa widget seperti layer.
- **Align** — menempatkan child relatif terhadap induk (misalnya `Alignment.topRight`, setara `alignParentTop` + `alignParentRight`).
- **Positioned** — menempatkan child di dalam Stack berdasarkan jarak dari sisi `top`, `bottom`, `left`, `right`.

Jadi relative layout adalah desain tampilan dengan tata letak objek secara bebas tanpa aturan orientasi (horizontal atau vertikal) seperti yang diterapkan pada linear layout.

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Linear Layout (Column & Row)

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `linear_layout`.
3. Pastikan target platform Android tercentang (minimal Android 5.0/API 21 sesuai default Flutter).
4. Buka file `lib/main.dart`.
5. Hapus seluruh isi file bawaan.
6. Implementasikan code di bawah ini:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
      home: const LinearPage(),
    );
  }
}

class LinearPage extends StatefulWidget {
  const LinearPage({super.key});

  @override
  State<LinearPage> createState() => _LinearPageState();
}

class _LinearPageState extends State<LinearPage> {
  final _to = TextEditingController();
  final _subject = TextEditingController();
  final _message = TextEditingController();

  @override
  void dispose() {
    _to.dispose();
    _subject.dispose();
    _message.dispose();
    super.dispose();
  }

  void _kirim() {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Pesan dikirim ke ${_to.text}')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Linear Layout')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        // Column = LinearLayout vertical
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            TextField(
              controller: _to,
              decoration: const InputDecoration(hintText: 'to'),
            ),
            TextField(
              controller: _subject,
              decoration: const InputDecoration(hintText: 'subject'),
            ),
            // Expanded = layout_weight="1"
            Expanded(
              child: TextField(
                controller: _message,
                maxLines: null,
                expands: true,
                textAlignVertical: TextAlignVertical.top,
                decoration: const InputDecoration(hintText: 'message'),
              ),
            ),
            const SizedBox(height: 8),
            // Align kanan = layout_gravity="right"
            Align(
              alignment: Alignment.centerRight,
              child: SizedBox(
                width: 120,
                child: FilledButton(
                  onPressed: _kirim,
```

```
                  child: const Text('SEND'),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

7. Running hasil code di atas (Shift + F10). Aplikasi akan menampilkan tiga kolom isian (*to*, *subject*, *message*) yang tersusun vertikal dan tombol **SEND** di kanan bawah.



Gambar 1. Hasil Linear Layout (kolom to, subject, message tersusun vertikal dan tombol SEND di kanan bawah)

8. Tambahkan contoh **Row** (LinearLayout horizontal) dengan mengganti tombol SEND menjadi dua tombol berdampingan:

DART
```
Row(
  mainAxisAlignment: MainAxisAlignment.end,
  children: [
    OutlinedButton(
      onPressed: () {
        _to.clear();
        _subject.clear();
        _message.clear();
      },
      child: const Text('RESET'),
    ),
    const SizedBox(width: 8),
    FilledButton(onPressed: _kirim, child: const Text('SEND')),
  ],
),
```

Relative Layout (Stack, Align, Positioned)

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `relative_layout`.
3. Pastikan target platform Android tercentang.
4. Buka file `lib/main.dart`.
5. Hapus seluruh isi file bawaan.
6. Implementasikan code di bawah ini:

DART
```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: RelativePage(),
    ));

class RelativePage extends StatelessWidget {
```

```
  const RelativePage({super.key});

  Widget _kotak(String teks, Color warna) {
    return Container(
      width: 110,
      height: 60,
      alignment: Alignment.center,
      color: warna,
      child: Text(teks, style: const TextStyle(color: Colors.white)),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Relative Layout')),
      body: Stack(
        children: [
          // alignParentTop + alignParentLeft
          Align(alignment: Alignment.topLeft,
              child: _kotak('Kiri Atas', Colors.red)),
          // alignParentTop + alignParentRight
          Align(alignment: Alignment.topRight,
              child: _kotak('Kanan Atas', Colors.green)),
          // centerInParent
          Align(alignment: Alignment.center,
              child: _kotak('Tengah', Colors.blue)),
          // layout_below dari kotak tengah (relatif terhadap tengah)
          Align(
            alignment: const Alignment(0, 0.25),
            child: _kotak('Di Bawah Tengah', Colors.orange),
          ),
          // alignParentBottom + alignParentLeft
          Align(alignment: Alignment.bottomLeft,
              child: _kotak('Kiri Bawah', Colors.purple)),
          // posisi bebas dengan Positioned
          Positioned(
            right: 16,
            bottom: 16,
            child: _kotak('Kanan Bawah', Colors.teal),
          ),
        ],
      ),
    );
  }
}
```

7. Tambahkan interaksi: bungkus kotak **Tengah** dengan `GestureDetector` dan tampilkan SnackBar saat disentuh:

DART

```
Align(
  alignment: Alignment.center,
  child: GestureDetector(
    onTap: () => ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Kotak tengah disentuh')),
    ),
    child: _kotak('Tengah', Colors.blue),
  ),
),
```

8. Running hasil code, dan akan menampilkan output berupa kotak-kotak yang menempel pada sisi kiri atas, kanan atas, tengah, kiri bawah dan kanan bawah layar.

[Tempelkan screenshot hasil running aplikasi]



Gambar 2. Hasil Relative Layout dengan Stack, Align dan Positioned

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan linear layout.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project linear layout yang dibuktikan dengan screenshot pada proses percobaan.
3. Project aplikasi dari percobaan relative layout.
4. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project relative layout yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

1. Mahasiswa berhasil memahami konsep linear layout (Column & Row), serta berhasil menerapkan konsep linear layout.
2. Mahasiswa mampu memahami konsep relative layout (Stack, Align, Positioned), serta berhasil menerapkan konsep relative layout.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code linear dan relative layout	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code linear dan relative layout	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code linear dan relative layout	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code linear dan relative layout	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 10 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW

Pokok Bahasan	:	Constraint Layout dan Frame Layout
Acara Praktikum/Pertemuan	:	Minggu 3 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep constraint layout pada Flutter.
2. Mahasiswa mampu memahami konsep frame layout pada Flutter.
3. Mahasiswa mampu menerapkan konsep constraint layout.
4. Mahasiswa mampu menerapkan konsep frame layout.

b. Indikator Penilaian

1. Keberhasilan mahasiswa dalam menerapkan constraint layout pada studi kasus.
2. Keberhasilan mahasiswa dalam menerapkan frame layout pada studi kasus.

c. Dasar Teori

1) Constraint Layout

Pada Android native, ConstraintLayout mirip dengan RelativeLayout karena letak View bergantung pada View lain atau pada Parent Layout, tetapi lebih fleksibel karena setiap sisi View diikat oleh "tali" (*constraint*) ke *anchor point*. Flutter tidak memiliki widget bernama ConstraintLayout; seluruh layout Flutter sudah berbasis **constraints** (BoxConstraints) yang diturunkan dari induk ke anak. Perilaku ConstraintLayout dapat dibuat dengan kombinasi widget berikut:

- **Expanded / Flexible** — mengisi sisa ruang secara proporsional (setara *match_constraint* / 0dp dengan weight).
- **Spacer** — "tali" elastis yang mendorong widget ke sisi tertentu (setara *chain* dengan *bias*).
- **Align dengan Alignment(x, y)** — posisi relatif dengan nilai -1 sampai 1, setara `layout_constraintHorizontal_bias` dan `layout_constraintVertical_bias`.
- **ConstrainedBox / SizedBox / AspectRatio** — membatasi ukuran minimum/maksimum dan rasio (setara `layout_constraintDimensionRatio`).
- **LayoutBuilder** — membaca constraint dari induk sehingga tata letak dapat menyesuaikan ukuran layar (responsif).

2) Frame Layout

Frame Layout adalah layout yang biasanya digunakan untuk membuat objek yang saling bertindihan, contohnya membuat tombol di atas gambar. Pada Flutter, padanannya adalah widget **Stack**: anak pertama berada paling bawah dan anak terakhir berada paling atas.

3) Table Layout

Table Layout adalah layout yang digunakan untuk membangun antarmuka berdasarkan **baris** dan **kolom**. Pada Flutter digunakan widget **Table** yang berisi beberapa **TableRow**.

- Row/baris pada dasarnya digunakan untuk menyimpan satu jenis record.
- Kolom adalah sub bagian dari setiap baris, satu baris dapat menampung beberapa kolom dengan jenis informasi berbeda.

Beberapa widget pada Table Layout Flutter:

- `Table` : widget pembuka untuk menggunakan tabel (setara tag `TableLayout`).
- `TableRow` : widget untuk membuat baris (setara tag `TableRow`).

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Constraint Layout

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `constraint_layout`.
3. Pastikan target platform Android tercentang.
4. Buka file `lib/main.dart`.
5. Hapus seluruh isi file bawaan.
6. Implementasikan code di bawah ini (form profil yang posisinya diikat terhadap induk dan widget lain):

DART
```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: ConstraintPage(),
    ));

class ConstraintPage extends StatelessWidget {
  const ConstraintPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Constraint Layout')),
      body: LayoutBuilder(
        builder: (context, constraints) {
          // constraints.maxWidth = lebar yang diberikan induk
          final lebar = constraints.maxWidth;
          return Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              children: [
                // Avatar: horizontal bias 0.5 (tengah)
                const Align(
```

```
                  alignment: Alignment(0, 0),
                  child: CircleAvatar(radius: 40,
                      child: Icon(Icons.person, size: 40)),
                ),
                const SizedBox(height: 16),
                // Dua field sejajar, lebar mengikuti sisa ruang
                Row(
                  children: const [
                    Expanded(child: TextField(
                        decoration: InputDecoration(labelText: 'Nama Depan'))),
                    SizedBox(width: 12),
                    Expanded(child: TextField(
                        decoration: InputDecoration(labelText: 'Nama Belakang'))),
                  ],
                ),
                const SizedBox(height: 12),
                // Lebar maksimal dibatasi 60% lebar layar
                ConstrainedBox(
                  constraints: BoxConstraints(maxWidth: lebar * 0.6),
                  child: const TextField(
                      decoration: InputDecoration(labelText: 'NIM')),
                ),
                // Spacer = constraint elastis yang mendorong tombol ke bawah
                const Spacer(),
                // Tombol dengan bias horizontal 0.8 (agak ke kanan)
                Align(
                  alignment: const Alignment(0.6, 0),
                  child: FilledButton(
                      onPressed: () {}, child: const Text('SIMPAN')),
                ),
                const SizedBox(height: 12),
                // Banner dengan rasio 16:9
                AspectRatio(
                  aspectRatio: 16 / 9,
                  child: Container(
                    color: Colors.indigo.shade100,
                    alignment: Alignment.center,
                    child: Text('Lebar layar: ${lebar.toStringAsFixed(0)} px'),
                  ),
                ),
              ],
            ),
          );
        },
      ),
    );
  }
}
```

7. Running hasil code di atas. Ubah orientasi emulator (portrait/landscape) dan amati bagaimana ukuran widget menyesuaikan constraint dari induknya.

[Tempelkan screenshot hasil running aplikasi]

Gambar 3. Hasil Constraint Layout

Frame Layout (Stack)

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `frame_layout`.
3. Pastikan target platform Android tercentang.
4. Buka file `lib/main.dart`.

5. Hapus seluruh isi file bawaan.
6. Implementasikan code di bawah ini (tombol dan teks di atas gambar):

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: FramePage(),
    ));

class FramePage extends StatelessWidget {
  const FramePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Frame Layout')),
      body: Stack(
        fit: StackFit.expand,
        children: [
          // Layer 1 (paling bawah): gambar
          Image.network(
            'https://picsum.photos/id/1018/800/1200',
            fit: BoxFit.cover,
          ),
          // Layer 2: lapisan gelap transparan
          Container(color: Colors.black26),
          // Layer 3: judul di bagian atas
          const Positioned(
            top: 24,
            left: 16,
            child: Text('Politeknik Negeri Jember',
                style: TextStyle(fontSize: 22, color: Colors.white,
                    fontWeight: FontWeight.bold)),
          ),
          // Layer 4 (paling atas): tombol di atas gambar
          Center(
            child: FilledButton.icon(
              onPressed: () {},
              icon: const Icon(Icons.play_arrow),
              label: const Text('MULAI'),
            ),
          ),
        ],
      ),
    );
  }
}
```

> Gambar memakai `Image.network` sehingga emulator perlu koneksi internet. Pada build release Android, izin `android.permission.INTERNET` harus ditambahkan pada `android/app/src/main/AndroidManifest.xml` (pada mode debug izin ini sudah tersedia otomatis).

7. Running hasil code di atas. Tombol **MULAI** dan teks judul tampil bertumpuk di atas gambar.

[Tempelkan screenshot hasil running aplikasi]

Gambar 4. Hasil Frame Layout dengan Stack

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan constraint layout.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project constraint layout yang dibuktikan dengan screenshot pada proses percobaan.
3. Project aplikasi dari percobaan frame layout.
4. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project frame layout yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

1. Mahasiswa berhasil memahami konsep constraint layout, serta berhasil menerapkan konsep constraint layout.
2. Mahasiswa mampu memahami konsep frame layout, serta berhasil menerapkan konsep frame layout.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code constraint dan frame layout	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code constraint dan frame layout	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code constraint dan frame layout	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code constraint dan frame layout	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 11 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW

Pokok Bahasan	:	Table Layout dan Material Design
Acara Praktikum/Pertemuan	:	Minggu 3 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep table layout.
2. Mahasiswa mampu memahami konsep material design.
3. Mahasiswa mampu menerapkan konsep table layout.
4. Mahasiswa mampu menerapkan konsep material design.

b. Indikator Penilaian

1. Keberhasilan mahasiswa dalam menerapkan table layout pada studi kasus.
2. Keberhasilan mahasiswa dalam menerapkan material design pada studi kasus.

c. Dasar Teori

Material Design

"Material Design adalah *design system* yang dikembangkan oleh Google. Dirilis pertama kali untuk Android 5.0 Lollipop dan diperkenalkan ke publik pada acara Google I/O tanggal 25 Juni 2014." Versi terbarunya adalah **Material 3 (Material You)**.

Dengan menggunakan Material Design kita bisa membuat tampilan UI/UX aplikasi terlihat lebih nyata, bertekstur, modern, dan profesional. Berbeda dengan Android native yang harus menambahkan dependency `com.google.android.material`, pada Flutter seluruh komponen Material **sudah tersedia** di package `flutter/material.dart` dan Material 3 aktif secara default. Beberapa komponen Material Design yang dapat diintegrasikan antara lain:

- Tema Material (`ThemeData`, `ColorScheme`, `TextTheme`)
- List dan Card (`ListTile`, `Card`)
- Shadow/elevation (`elevation`, `Material`)
- Animasi (`AnimatedContainer`, transisi halaman, *ripple effect* `InkWell`)
- Komponen lain: `AppBar`, `FloatingActionButton`, `NavigationBar`, `TextField` dengan `OutlineInputBorder`, `Chip`, `SnackBar`, dan lainnya.

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Table Layout

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `table_layout`.
3. Pastikan target platform Android tercentang.
4. Buka file `lib/main.dart`.
5. Hapus seluruh isi file bawaan.
6. Implementasikan code di bawah ini:

DART
```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TablePage(),
    ));

class TablePage extends StatelessWidget {
  const TablePage({super.key});

  TableRow _baris(List<String> isi, {bool header = false}) {
    return TableRow(
      decoration: BoxDecoration(
        color: header ? Colors.indigo : null,
      ),
      children: isi
          .map((teks) => Padding(
                padding: const EdgeInsets.all(8),
                child: Text(
                  teks,
                  style: TextStyle(
                    color: header ? Colors.white : Colors.black87,
                    fontWeight: header ? FontWeight.bold : FontWeight.normal,
                  ),
                ),
              ))
          .toList(),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Table Layout')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Table(
          border: TableBorder.all(color: Colors.grey),
          columnWidths: const {
            0: FixedColumnWidth(48),
            1: FlexColumnWidth(2),
            2: FlexColumnWidth(3),
          },
          children: [
            _baris(['No', 'NIM', 'Nama'], header: true),
            _baris(['1', 'E32241001', 'Andi Pratama']),
            _baris(['2', 'E32241002', 'Budi Santoso']),
            _baris(['3', 'E32241003', 'Citra Lestari']),
          ],
        ),
      ),
    );
  }
```

```
}
```

7. Running hasil code di atas, akan tampil tabel tiga kolom (No, NIM, Nama) dengan baris header berwarna.



Gambar 5. Hasil Table Layout

Material Design

1. **Menambahkan Required Dependency.** Pada Flutter tidak perlu menambahkan dependency Material karena sudah termasuk dalam SDK. Pastikan pada `pubspec.yaml` terdapat baris berikut (default dari `flutter create`):

```yaml
flutter:
  uses-material-design: true
```

2. **Merubah Tema Aplikasi.** Tema diatur melalui properti `theme` pada `MaterialApp` (setara file `styles.xml`).
3. **Merubah Warna Aplikasi.** Warna diatur melalui `ColorScheme.fromSeed` (setara file `colors.xml`). Flutter otomatis membuat palet warna primary, secondary, surface, dan lainnya dari satu warna dasar (*seed color*).
4. Buat project Flutter baru dengan nama `material_design`, lalu ganti isi `lib/main.dart` dengan sintaks berikut:

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Material Design',
      // Tema terang (setara styles.xml + colors.xml)
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF6200EE)),
        useMaterial3: true,
        inputDecorationTheme: const InputDecorationTheme(
          border: OutlineInputBorder(),
        ),
      ),
      // Tema gelap otomatis mengikuti pengaturan HP
      darkTheme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6200EE),
          brightness: Brightness.dark,
        ),
        useMaterial3: true,
      ),
      home: const MaterialDemoPage(),
```

```
    );
  }
}

class MaterialDemoPage extends StatelessWidget {
  const MaterialDemoPage({super.key});

  @override
  Widget build(BuildContext context) {
    final warna = Theme.of(context).colorScheme;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Material Design'),
        backgroundColor: warna.primary,
        foregroundColor: warna.onPrimary,
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const TextField(
            decoration: InputDecoration(
              labelText: 'Email',
              prefixIcon: Icon(Icons.email),
            ),
          ),
          const SizedBox(height: 12),
          const TextField(
            obscureText: true,
            decoration: InputDecoration(
              labelText: 'Password',
              prefixIcon: Icon(Icons.lock),
            ),
          ),
          const SizedBox(height: 16),
          FilledButton(onPressed: () {}, child: const Text('FILLED BUTTON')),
          OutlinedButton(onPressed: () {}, child: const Text('OUTLINED BUTTON')),
          TextButton(onPressed: () {}, child: const Text('TEXT BUTTON')),
          const SizedBox(height: 16),
          Card(
            elevation: 4, // shadow
            child: ListTile(
              leading: const Icon(Icons.school),
              title: const Text('Workshop Aplikasi Mobile'),
              subtitle: const Text('Material Card dengan elevation'),
              trailing: const Icon(Icons.chevron_right),
              onTap: () {},
            ),
          ),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            children: const [
              Chip(label: Text('Flutter')),
              Chip(label: Text('Dart')),
              Chip(label: Text('Material 3')),
            ],
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {},
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

5. Maka akan menghasilkan layout seperti gambar berikut. Ubah mode HP ke *dark mode* dan amati perubahan tema secara otomatis.

[Tempelkan screenshot hasil running aplikasi]

Gambar 6. Hasil Material Design

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan table layout.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project table layout yang dibuktikan dengan screenshot pada proses percobaan.
3. Project aplikasi dari percobaan material design.
4. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project material design yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

1. Mahasiswa berhasil memahami konsep table layout, serta berhasil menerapkan konsep table layout.
2. Mahasiswa mampu memahami konsep material design, serta berhasil menerapkan konsep material design.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code table layout dan material design	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code table layout dan material design	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code table layout dan material design	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code table layout dan material design	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 12 : LAYOUT, MATERIAL DESIGN DAN SCROLL VIEW

Pokok Bahasan	:	Scroll View
Acara Praktikum/Pertemuan	:	Minggu 3 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep scroll view.
2. Mahasiswa mampu menerapkan konsep scroll view.
3. Mahasiswa mampu menyelesaikan studi kasus.

b. Indikator Penilaian

1. Keberhasilan mahasiswa dalam menerapkan scroll view pada studi kasus.
2. Keberhasilan mahasiswa dalam memecahkan studi kasus tambahan.

c. Dasar Teori

Scroll View

Scroll view adalah *container* yang memungkinkan pengguna menggulir (*scroll*) tampilan jika isinya lebih besar dari layar. Jika konten melebihi layar tanpa scroll view, Flutter akan menampilkan peringatan garis kuning-hitam **"BOTTOM OVERFLOWED BY ... PIXELS"**.

Hampir semua aplikasi menggunakan scroll view. Padanan jenis-jenis ScrollView Android pada Flutter adalah:

Android	Flutter
ScrollView (vertikal)	SingleChildScrollView (scrollDirection: Axis.vertical)
HorizontalScrollView	SingleChildScrollView (scrollDirection: Axis.horizontal)
NestedScrollView	NestedScrollView / CustomScrollView dengan Sliver (SliverAppBar, SliverList)

`SingleChildScrollView` cocok untuk konten yang jumlahnya sedikit dan tetap (misalnya form). Untuk daftar data yang panjang dan dinamis gunakan `ListView` (dibahas pada Acara 13–16).

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Scroll View Vertikal

1. Buat project Flutter baru dengan nama `scroll_view`. Buat file `lib/scroll_vertical.dart` dengan sintaks berikut:

```dart
import 'package:flutter/material.dart';

class ScrollVerticalPage extends StatelessWidget {
  const ScrollVerticalPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Scroll View Vertikal')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: List.generate(
            20,
            (i) => Container(
              height: 80,
              margin: const EdgeInsets.only(bottom: 8),
              alignment: Alignment.center,
              color: Colors.primaries[i % Colors.primaries.length],
              child: Text('Item ${i + 1}',
                  style: const TextStyle(color: Colors.white, fontSize: 18)),
            ),
          ),
        ),
      ),
    );
  }
}
```

Maka akan menghasilkan layout berupa 20 kotak berwarna yang dapat digulir ke atas dan ke bawah.

[Tempelkan screenshot hasil running aplikasi]

Gambar 7. Hasil Scroll View Vertikal

Scroll View Horizontal

2. Buat file `lib/scroll_horizontal.dart` dengan sintaks berikut:

```dart
import 'package:flutter/material.dart';

class ScrollHorizontalPage extends StatelessWidget {
  const ScrollHorizontalPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Scroll View Horizontal')),
      body: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Padding(
```

```
            padding: EdgeInsets.all(16),
            child: Text('Kategori', style: TextStyle(fontSize: 18)),
          ),
          SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: Row(
              children: List.generate(
                10,
                (i) => Container(
                  width: 120,
                  height: 120,
                  margin: const EdgeInsets.only(right: 8),
                  alignment: Alignment.center,
                  decoration: BoxDecoration(
                    color: Colors.accents[i % Colors.accents.length],
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Text('Menu ${i + 1}'),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

3. Ubah `lib/main.dart` agar memiliki dua tombol untuk membuka kedua halaman:

DART

```
import 'package:flutter/material.dart';
import 'scroll_vertical.dart';
import 'scroll_horizontal.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: HomePage(),
    ));

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Scroll View')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            FilledButton(
              onPressed: () => Navigator.push(context,
                  MaterialPageRoute(builder: (_) => const ScrollVerticalPage())),
              child: const Text('SCROLL VERTIKAL'),
            ),
            const SizedBox(height: 12),
            FilledButton(
              onPressed: () => Navigator.push(context,
                  MaterialPageRoute(builder: (_) => const ScrollHorizontalPage())),
              child: const Text('SCROLL HORIZONTAL'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Maka akan menghasilkan layout yang dapat digulir secara horizontal.

[Tempelkan screenshot hasil running aplikasi]

Gambar 8. Hasil Scroll View Horizontal

Tugas

Membuat aplikasi baru dengan aturan tampilan minimal seperti tabel di bawah ini (lebih dari contoh diperbolehkan). Halaman utama berisi menu yang membuka setiap halaman percobaan:

No	Halaman	Widget Flutter yang digunakan
1	Linear Layout	Column / Row
2	Relative Layout	Stack + Align / Positioned
3	Constraint Layout	Expanded, Spacer, Align(bias), ConstrainedBox, LayoutBuilder
4	Linear Layout (horizontal)	Row
5	Table Layout	Table + TableRow
6	Material Design	ThemeData, Card, FilledButton, FAB
7	Scroll View	SingleChildScrollView vertikal
8	Scroll View Horizontal	SingleChildScrollView horizontal

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan scroll view.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project scroll view yang dibuktikan dengan screenshot pada proses percobaan.
3. Project aplikasi dari tugas layout, material design dan scroll view.
4. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project tugas layout, material design dan scroll view yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

1. Mahasiswa berhasil memahami konsep scroll view, serta berhasil menerapkan konsep scroll view.
2. Mahasiswa mampu menyelesaikan studi kasus dan mampu menjelaskan secara tepat.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi scroll view dan tugas studi kasus serta menjelaskan secara detail setiap baris code	1	2	3	4

No	Indikator	Skor			
2	Ketepatan waktu dan ketepatan dalam implementasi scroll view dan tugas studi kasus serta menjelaskan secara singkat	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi scroll view dan tugas studi kasus	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi scroll view dan tugas studi kasus	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 13 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW)

Pokok Bahasan	:	Selection Widget
Acara Praktikum/Pertemuan	:	Minggu 4 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep dasar selection widget.
2. Mahasiswa mampu membedakan jenis-jenis selection widget.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam memahami konsep dasar selection widget.

c. Dasar Teori

Selection Widget

Selection widget adalah widget yang menyediakan daftar pilihan bagi pengguna. Pada Android native, selection widget dibangun dengan *framework* **Adapter** (ArrayAdapter, BaseAdapter) yang menghubungkan data (array atau database) ke tampilan. Pada Flutter tidak ada kelas Adapter: data cukup disimpan pada `List` Dart, lalu widget daftar membangun tampilan setiap item melalui fungsi **builder** (`itemBuilder`). Pola ini lebih sederhana tetapi memiliki peran yang sama dengan adapter.

Beberapa selection widget pada Flutter antara lain:

1. **ListView** — menampilkan daftar item vertikal yang dapat di-scroll apabila data melebihi besarnya layar (setara `ListView` Android). Item biasanya menggunakan `ListTile`.
2. **DropdownButton / DropdownMenu** — cara cepat untuk memilih satu nilai dari satu set (setara `Spinner`). Dalam keadaan default menampilkan nilai yang sedang dipilih; ketika disentuh menampilkan menu tarik-turun berisi semua nilai lain.
3. **Autocomplete** — kolom teks yang dapat diedit dan menunjukkan saran penyelesaian secara otomatis saat pengguna mengetik (setara `AutoCompleteTextView`). Daftar saran ditampilkan di menu drop-down; pengguna dapat memilih item untuk menggantikan isi kolom teks.

d. Alat dan Bahan

1. Kertas A4
2. Spidol

e. Prosedur Kerja

1. Mahasiswa membuat rancangan desain untuk contoh tampilan ListView, DropdownButton (spinner), dan Autocomplete pada kertas A4.

2. Buat dalam PPT/Figma dalam bentuk prototype.
3. Presentasikan di depan kelas.

f. Hasil dan Pembahasan

1. Dokumen tugas rancangan desain contoh tampilan ListView, DropdownButton, dan Autocomplete.
2. Dokumen prototype dalam bentuk PPT/Figma.

g. Kesimpulan

Mahasiswa memahami konsep dasar ListView, DropdownButton (spinner), dan Autocomplete dengan mampu memahami perbedaan dari ketiganya.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam rancangan desain selection widget ditunjang penjelasan detail perbedaannya	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam rancangan desain selection widget dengan penjelasan singkat	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam rancangan desain selection widget	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam rancangan desain selection widget	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 14 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW)

Pokok Bahasan	:	Implementasi List View
Acara Praktikum/Pertemuan	:	Minggu 4 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu mengimplementasikan list view.
2. Mahasiswa mampu memahami konsep mengimplementasikan list view.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam mengimplementasikan list view.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

List View

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `list_view_app`.
3. Pastikan target platform Android tercentang.
4. Buat file `lib/data_negara.dart` untuk menyimpan data array (setara `strings.xml` dengan `string-array`):

```dart
// Data array nama negara (setara <string-array name="countries_array">)
const List<String> countriesArray = [
  'Indonesia', 'Malaysia', 'Singapura', 'Thailand', 'Filipina',
  'Vietnam', 'Jepang', 'Korea Selatan', 'Tiongkok', 'India',
  'Arab Saudi', 'Mesir', 'Inggris', 'Prancis', 'Jerman',
  'Belanda', 'Italia', 'Spanyol', 'Amerika Serikat', 'Brasil',
];
```

Di sini dicontohkan menampilkan beberapa nama negara di dunia.

5. Ubah isi `lib/main.dart` menjadi seperti berikut:

```dart
import 'package:flutter/material.dart';
import 'data_negara.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainPage(),
```

```
    ));

class MainPage extends StatelessWidget {
  const MainPage({super.key});

  // Setara method onItemClick() pada AdapterView.OnItemClickListener
  void _onItemClick(BuildContext context, int position) {
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(SnackBar(
        content: Text('Anda memilih: ${countriesArray[position]}'),
      ));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('List View Negara')),
      body: ListView.separated(
        itemCount: countriesArray.length,
        separatorBuilder: (_, __) => const Divider(height: 1),
        // itemBuilder setara getView() pada adapter
        itemBuilder: (context, position) {
          return ListTile(
            leading: CircleAvatar(child: Text('${position + 1}')),
            title: Text(countriesArray[position]),
            onTap: () => _onItemClick(context, position),
          );
        },
      ),
    );
  }
}
```

Penjelasan program:

- `ListView.separated(itemCount: ..., itemBuilder: ...)` berarti mengambil nilai dari list `countriesArray` lalu menampilkannya satu per satu dengan model tampilan `ListTile` (setara `ArrayAdapter` + `simple_list_item_1`). Item hanya dibangun ketika terlihat di layar sehingga hemat memori.
- Properti `onTap` pada `ListTile` berarti ketika data pada list ditekan akan menampilkan pesan berupa **SnackBar** (setara Toast). Fungsi ini setara `listView.setOnItemClickListener(this)`.
- Pada Flutter tidak perlu meng-*implements* interface listener; cukup mengirimkan fungsi (*callback*) ke properti `onTap`.

6. (Opsional) Untuk menampilkan pesan bergaya Toast, tambahkan package `fluttertoast` dengan perintah `flutter pub add fluttertoast`, lalu panggil `Fluttertoast.showToast(msg: countriesArray[position]);`.
7. Running hasil code di atas dan lakukan uji coba, kemudian perhatikan apa yang terjadi.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan list view.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project list view yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan list view pada project individu serta memahami konsep mengimplementasikan list view.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code list view	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code list view	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code list view	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code list view	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 15 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW)

Pokok Bahasan	:	Card View dan Recycler View (Card & ListView.builder)
Acara Praktikum/Pertemuan	:	Minggu 4 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu mengimplementasikan card view dan recycler view (Card dan ListView.builder).
2. Mahasiswa mampu memahami konsep mengimplementasikan card view dan recycler view.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam mengimplementasikan card view dan recycler view.

c. Dasar Teori

Card View dan Recycler View

Terkadang dalam sebuah aplikasi kita ingin menampilkan set data yang berjumlah besar (ratusan atau bahkan jutaan). Di Android native, RecyclerView hadir sebagai pengganti ListView karena lebih cepat dan efisien: hanya membuat tampilan untuk item yang terlihat, lalu mendaur ulang (*recycle*) tampilan tersebut ketika di-scroll. Contoh penggunaannya ada pada aplikasi Gmail.

Pada Flutter, perilaku RecyclerView didapatkan dengan konstruktor `ListView.builder` (atau `GridView.builder`). Widget item hanya dibangun secara *lazy* ketika akan tampil di layar, dan elemen yang sudah keluar layar dibuang sehingga konsumsi memori tetap kecil walaupun datanya sangat banyak.

Card merupakan widget dari Material Design untuk membungkus sepaket data (beberapa widget) yang ditandai dengan bayangan (*elevation*) dan sudut membulat (*corner radius*). Card di Flutter setara dengan `CardView` di Android.

Android (RecyclerView)	Flutter
Dependency recyclerview & cardview	Tidak perlu (bawaan Flutter)
Model class (Mahasiswa.java)	Class model Dart (mahasiswa.dart)
Layout item list_item.xml	Widget item (MahasiswaCard)
Adapter + ViewHolder	itemBuilder pada ListView.builder
recyclerView.setAdapter()	ListView.builder(itemCount, itemBuilder)
LayoutManager	scrollDirection / GridView.builder

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Card dan ListView.builder

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `recycler_view_app`.
3. Pastikan target platform Android tercentang.
4. Tidak perlu menambahkan dependency; `Card` dan `ListView` sudah tersedia di `package:flutter/material.dart`.
5. **Membuat model class sesuai dengan data.** Buat file `lib/mahasiswa.dart`:

```dart
class Mahasiswa {
  final String nama;
  final String npm;
  final String nohp;

  const Mahasiswa({
    required this.nama,
    required this.npm,
    required this.nohp,
  });
}
```

6. **Membuat layout item (list_item).** Ini untuk mendeskripsikan apa saja elemen dalam satu baris list. Di sinilah kita letakkan Card-nya. Buat file `lib/mahasiswa_card.dart`, dalam contoh ini kita menampilkan nama, npm dan nohp:

```dart
import 'package:flutter/material.dart';
import 'mahasiswa.dart';

class MahasiswaCard extends StatelessWidget {
  final Mahasiswa mahasiswa;
  final VoidCallback? onTap;

  const MahasiswaCard({super.key, required this.mahasiswa, this.onTap});

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 3,
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      child: InkWell(
        borderRadius: BorderRadius.circular(12),
        onTap: onTap,
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(mahasiswa.nama,
                  style: const TextStyle(
```

```
                    fontSize: 18, fontWeight: FontWeight.bold)),
              const SizedBox(height: 4),
              Text('NPM  : ${mahasiswa.npm}'),
              Text('No HP: ${mahasiswa.nohp}'),
            ],
          ),
        ),
      ),
    );
  }
}
```

7. **Menghubungkan data ke ListView.builder (setara set Adapter ke RecyclerView).** Di sini ditambahkan *dummy data* ke dalam list dan dimuat ke `ListView.builder`. Ubah isi `lib/main.dart`:

DART

```
import 'package:flutter/material.dart';
import 'mahasiswa.dart';
import 'mahasiswa_card.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainPage(),
    ));

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  final List<Mahasiswa> _mahasiswaList = [];

  @override
  void initState() {
    super.initState();
    _addData();
  }

  // Dummy data, setara addData() pada MainActivity.java
  void _addData() {
    _mahasiswaList.addAll(const [
      Mahasiswa(nama: 'Dimas Maulana', npm: '1414370309', nohp: '123456789'),
      Mahasiswa(nama: 'Fadly Yonk', npm: '1214234560', nohp: '987654321'),
      Mahasiswa(nama: 'Ariyandi Nugraha', npm: '1214230345', nohp: '987648765'),
      Mahasiswa(nama: 'Aham Siswana', npm: '1214378098', nohp: '098758124'),
    ]);
    // Data tambahan agar terlihat efek scroll pada data banyak
    for (var i = 1; i <= 100; i++) {
      _mahasiswaList.add(Mahasiswa(
        nama: 'Mahasiswa $i',
        npm: 'E3224${i.toString().padLeft(4, '0')}',
        nohp: '08123${i.toString().padLeft(6, '0')}',
      ));
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Card & ListView.builder')),
      body: ListView.builder(
        itemCount: _mahasiswaList.length,
        itemBuilder: (context, index) {
```

```
          final mhs = _mahasiswaList[index];
          return MahasiswaCard(
            mahasiswa: mhs,
            onTap: () => ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('${mhs.nama} dipilih')),
            ),
          );
        },
      ),
    );
  }
}
```

8. Apabila tidak ada error jalankan dan lihat hasilnya seperti apa, bandingkan dengan penggunaan list view pada Acara 14 (coba ganti `ListView.builder` menjadi `ListView(children: ...)` lalu bandingkan performanya ketika jumlah data dinaikkan menjadi 10.000).

[Tempelkan screenshot hasil running aplikasi]

Gambar 9. Hasil Card dan ListView.builder

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code card view dan recycler view (Card dan ListView.builder).

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan Card dan ListView.builder pada project individu serta memahami konsep implementasinya.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code card view dan recycler view	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code card view dan recycler view	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code card view dan recycler view	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code card view dan recycler view	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 16 : ACTIVITY (LISTVIEW BUILDER / RECYCLER VIEW)

Pokok Bahasan	:	Card View dan Recycler View (Card & ListView.builder)
Acara Praktikum/Pertemuan	:	Minggu 4 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep dasar card view dan recycler view (Card dan ListView.builder).
2. Mahasiswa mampu memahami setiap baris code dari implementasi card view dan recycler view.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam memahami konsep dasar card view dan recycler view.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

1. Mahasiswa melakukan analisa code untuk setiap baris code dari implementasi Card dan ListView.builder.
2. Menjelaskan fungsi dan arti dari setiap baris code.
3. Analisa hasil running dari implementasi project Card dan ListView.builder, termasuk perbandingan dengan ListView biasa.
4. Screenshot output setiap proses dari project Card dan ListView.builder.
5. Tuliskan dalam bentuk laporan.

f. Hasil dan Pembahasan

Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project Card dan ListView.builder yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa memahami konsep dasar card view dan recycler view (Card dan ListView.builder) dengan mampu memahami setiap baris code dari implementasinya.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan secara detail setiap baris code card view dan recycler view	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan secara singkat setiap baris code card view dan recycler view	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan code card view dan recycler view	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan code card view dan recycler view	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 17 : FRAGMENT (WIDGET MODULAR)

Materi Pembelajaran	:	Konsep Dasar Fragment pada Flutter
Acara Praktikum/Praktik	:	Minggu 5 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep fragment dan padanannya pada Flutter (widget modular).
2. Mahasiswa mampu menerapkan dan mengimplementasikan fragment (widget modular) pada aplikasi Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep fragment, implementasi widget modular pada Flutter, serta analisa source code.

c. Dasar Teori

Fragment

Pada Android native, **Fragment** merupakan komponen tampilan yang fleksibel dan dapat disisipkan ke dalam sebuah Activity. Fragment memiliki layout dan daur hidup (*lifecycle*) sendiri yang bergantung pada Activity tempat ia ditanamkan. Dengan Fragment, satu layar dapat dibagi menjadi beberapa bagian yang dikontrol secara terpisah dan dapat digunakan kembali (*reusable*) di beberapa Activity. Analogi yang mendekati fragment pada platform lain adalah komponen *iframe* pada aplikasi web.

Fragment pada Flutter

Flutter tidak memiliki kelas Fragment maupun Activity terpisah. Seluruh tampilan adalah **widget**, sehingga konsep fragment diterapkan secara alami dengan cara:

- Memecah tampilan menjadi **widget kecil yang berdiri sendiri** (StatelessWidget/StatefulWidget) yang dapat dipakai ulang di banyak halaman.
- Menampilkan widget yang berbeda pada **area tertentu** dari satu halaman (misalnya bagian `body` dari Scaffold) lalu menggantinya saat tombol ditekan — setara `FragmentTransaction.replace()`.
- Membagi layar menjadi beberapa panel (*multi-pane UI*) dengan `Row` + `Expanded` pada layar lebar (tablet), setara beberapa fragment dalam satu activity.
- Menggunakan **Navigator bersarang** (*nested navigator*) bila sebuah area membutuhkan *back-stack* sendiri, setara `addToBackStack()`.

Seperti fragment, widget tidak perlu didaftarkan pada `AndroidManifest.xml`. Sebuah kelas Dart menjadi widget ketika kelas tersebut meng-*extends* `StatelessWidget` atau `StatefulWidget`.

Beberapa poin penting mengenai widget modular di Flutter:

1. Widget dapat ditambah, diganti atau dihapus saat aplikasi berjalan cukup dengan mengubah *state* lalu memanggil `setState()`.
2. Widget memiliki tampilan (method `build()`) dan perilakunya sendiri, termasuk lifecycle untuk StatefulWidget.
3. Widget dapat digunakan di berbagai halaman.
4. Beberapa widget dapat digabungkan dalam satu halaman untuk membangun UI multi panel.

Kegunaan pendekatan ini:

1. Layar dapat dibagi menjadi beberapa bagian dan setiap bagian dikontrol secara terpisah.
2. Kode lebih rapi karena satu file berisi satu komponen tampilan.
3. Komponen memiliki event, tampilan, dan siklus hidupnya sendiri.

Kelas-kelas Utama

Android (Fragment)	Flutter
FragmentActivity	Scaffold / halaman (Widget) yang menjadi wadah
Fragment	StatelessWidget / StatefulWidget
onCreateView() + inflate layout XML	method build(BuildContext context)
FragmentManager	State milik halaman induk / Navigator
FragmentTransaction.replace()	setState(() => halamanAktif = ...)
addToBackStack()	Navigator.push() / Navigator bersarang

Contoh membuat "fragment" sebagai widget:

```dart
class FirstFragment extends StatelessWidget {
  const FirstFragment({super.key});

  // build() setara onCreateView(): mengembalikan tampilan fragment
  @override
  Widget build(BuildContext context) {
    return const Center(child: Text('Ini Fragment Pertama'));
  }
}
```

Lifecycle Widget (Setara Fragment Life Cycle)

StatefulWidget memiliki callback lifecycle pada objek `State`-nya:

Method	Kapan dipanggil	Padanan pada Fragment
createState()	Saat widget pertama kali dibuat	onAttach()
initState()	Sekali, ketika State dimasukkan ke pohon widget	onCreate()
didChangeDependencies()	Setelah initState dan ketika InheritedWidget yang dipakai berubah	onActivityCreated()

Method	Kapan dipanggil	Padanan pada Fragment
build()	Setiap kali tampilan perlu digambar ulang	onCreateView() / onStart()
didUpdateWidget()	Ketika widget induk membangun ulang dengan konfigurasi baru	-
deactivate()	Ketika State dikeluarkan dari pohon widget	onPause() / onStop()
dispose()	Ketika State dihapus permanen	onDestroyView() / onDestroy()

Selain itu, status aplikasi secara keseluruhan dapat dipantau melalui **AppLifecycleListener** / `AppLifecycleState`:

- **resumed** — aplikasi terlihat dan menerima input (setara state *Resume*).
- **inactive** — aplikasi terlihat sebagian/tidak menerima input, misalnya saat ada dialog sistem atau panggilan masuk (setara state *Paused*).
- **hidden** dan **paused** — aplikasi tidak terlihat di layar dan berjalan di latar belakang (setara state *Stopped*).
- **detached** — engine Flutter masih ada tetapi tidak terhubung ke tampilan apa pun (aplikasi akan dihancurkan).

Perubahan state aplikasi akan memengaruhi seluruh widget di dalamnya, sama seperti perubahan state Activity memengaruhi lifecycle fragment.

d. Alat dan Bahan

1. Kertas A4
2. Spidol

e. Prosedur Kerja

1. Mahasiswa mencari video referensi tentang implementasi fragment pada Android dan widget modular/pergantian konten pada Flutter.
2. Analisa kegunaan dan manfaat fragment serta padanannya di Flutter dari video yang sudah dilihat.
3. Resume pada kertas A4.

f. Hasil dan Pembahasan

1. Dokumen berupa video referensi yang telah diunduh dan dianalisa.
2. Dokumen tugas resume analisa video dan contoh pada aplikasi yang digunakan.

g. Kesimpulan

Mahasiswa memahami konsep dasar fragment dan penerapannya sebagai widget modular di Flutter dengan contoh pada aplikasi yang sering digunakan sehari-hari.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan konsep fragment/widget modular ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan konsep fragment/widget modular	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan konsep fragment/widget modular	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan konsep fragment/widget modular	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 18 : FRAGMENT (WIDGET MODULAR)

Pokok Bahasan	:	Implementasi Fragment pada Flutter
Acara Praktikum/Praktik	:	Minggu 5 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep fragment.
2. Mahasiswa mampu menerapkan dan mengimplementasikan fragment (widget modular) pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep fragment, implementasi widget modular pada Flutter, serta analisa source code.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

- Percobaan pada acara 18 praktikum minggu 5 pertemuan 2 terdapat contoh dua fragmen dan memuatnya dengan klik tombol.
- Menampilkan dua tombol dan satu area kontainer (setara FrameLayout) pada halaman utama, serta memberikan aksi `onPressed` pada kedua tombol.
- Pada klik tombol pertama area kontainer diganti dengan fragmen pertama dan pada klik tombol kedua diganti dengan fragmen kedua.
- Di kedua fragmen menampilkan Text dan tombol; ketika tombol diklik akan menampilkan nama fragmen dengan bantuan SnackBar (setara Toast).

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `fragment_example`.
3. Pastikan target platform Android tercentang.
4. Buat folder `lib/fragments/`.
5. **Menentukan warna** yang digunakan (setara `colors.xml`). Buat file `lib/colors.dart`:

```dart
import 'package:flutter/material.dart';
```

```
class AppColors {
  static const Color button1 = Color(0xFF3F51B5);
  static const Color button2 = Color(0xFFE91E63);
  static const Color fragment1 = Color(0xFFC5CAE9);
  static const Color fragment2 = Color(0xFFF8BBD0);
}
```

6. **Membuat 2 fragmen.** Buat file `lib/fragments/first_fragment.dart`:

DART
```
import 'package:flutter/material.dart';
import '../colors.dart';

class FirstFragment extends StatelessWidget {
  const FirstFragment({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: AppColors.fragment1,
      alignment: Alignment.center,
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          const Text('Ini Fragment Pertama',
              style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
          const SizedBox(height: 16),
          FilledButton(
            onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Fragmen Pertama')),
              );
            },
            child: const Text('Fragmen Pertama'),
          ),
        ],
      ),
    );
  }
}
```

- Pertama membangun tampilan pada method `build()` (setara *inflate* layout) yang berisi Text dan tombol.
- Setelah itu memberikan aksi `onPressed` pada tombol sehingga setiap kali pengguna klik tombol, pesan "Fragmen Pertama" ditampilkan di layar menggunakan SnackBar.

Buat file `lib/fragments/second_fragment.dart`:

DART
```
import 'package:flutter/material.dart';
import '../colors.dart';

class SecondFragment extends StatelessWidget {
  const SecondFragment({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: AppColors.fragment2,
      alignment: Alignment.center,
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          const Text('Ini Fragment Kedua',
              style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold)),
```

```
          const SizedBox(height: 16),
          FilledButton(
            onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Fragmen Kedua')),
              );
            },
            child: const Text('Fragmen Kedua'),
          ),
        ],
      ),
    );
  }
}
```

- Pertama membangun tampilan dan mendapatkan tombol.
- Setelah itu memberikan aksi `onPressed` pada tombol, sehingga setiap kali pengguna klik tombol pesan "Fragmen Kedua" ditampilkan di layar dengan SnackBar.

7. **Halaman utama (setara MainActivity).** Ubah isi `lib/main.dart`:

DART

```
import 'package:flutter/material.dart';
import 'colors.dart';
import 'fragments/first_fragment.dart';
import 'fragments/second_fragment.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainPage(),
    ));

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  // Fragment yang sedang tampil di area kontainer (setara FrameLayout)
  Widget _fragmentAktif = const FirstFragment();

  // Setara: FragmentManager -> beginTransaction() -> replace() -> commit()
  void _loadFragment(Widget fragment) {
    setState(() {
      _fragmentAktif = fragment;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Fragment Example')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(8),
            child: Row(
              children: [
                Expanded(
                  child: FilledButton(
                    style: FilledButton.styleFrom(
                        backgroundColor: AppColors.button1),
                    onPressed: () => _loadFragment(const FirstFragment()),
                    child: const Text('Fragmen Pertama'),
                  ),
                ),
```

```
                const SizedBox(width: 8),
                Expanded(
                  child: FilledButton(
                    style: FilledButton.styleFrom(
                        backgroundColor: AppColors.button2),
                    onPressed: () => _loadFragment(const SecondFragment()),
                    child: const Text('Fragmen Kedua'),
                  ),
                ),
              ],
            ),
          ),
          // Area kontainer (setara FrameLayout id="frameLayout")
          Expanded(
            child: AnimatedSwitcher(
              duration: const Duration(milliseconds: 300),
              child: KeyedSubtree(
                key: ValueKey(_fragmentAktif.runtimeType),
                child: _fragmentAktif,
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

- Pada langkah ini halaman utama menyiapkan dua tombol dan memberikan aksi `onPressed` pada kedua tombol.
- Pada klik tombol pertama area kontainer diganti dengan fragmen pertama dan pada klik tombol kedua diganti dengan fragmen kedua.
- Untuk mengganti fragmen: (1) simpan widget yang aktif pada variabel state; (2) ubah variabel tersebut di dalam `setState()`; (3) Flutter otomatis membangun ulang area kontainer dengan fragmen yang baru. `AnimatedSwitcher` menambahkan animasi transisi saat pergantian.

8. **AndroidManifest.xml.** Buka `android/app/src/main/AndroidManifest.xml`. Tidak ada yang perlu diubah karena aplikasi Flutter hanya memiliki satu Activity (`MainActivity`) dan semua halaman adalah widget. Kedua fragmen juga tidak perlu didefinisikan dalam manifest.

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code fragment (widget modular).

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan fragment (widget modular) pada project individu serta memahami konsep implementasinya.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code fragment	1	2	3	4

No	Indikator	Skor			
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code fragment	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code fragment	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code fragment	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 19 : FRAGMENT (WIDGET MODULAR)

Pokok Bahasan	:	Implementasi Fragment pada Flutter
Acara Praktikum/Praktik	:	Minggu 5 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep fragment.
2. Mahasiswa mampu menerapkan dan mengimplementasikan fragment (widget modular) pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep fragment, implementasi widget modular pada Flutter, serta analisa source code.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

1. Mahasiswa melanjutkan implementasi code pada acara 18 praktikum minggu 5 pertemuan 2 sampai dengan selesai.
2. Setelah selesai menuliskan code, running aplikasi dan lakukan percobaan.
3. Aplikasi memiliki dua tombol dengan nama **Fragmen Pertama** dan **Fragmen Kedua**.
4. Klik tombol pertama menampilkan Fragmen Pertama.
5. Klik tombol kedua menampilkan Fragmen Kedua.
6. Kedua tombol tersebut sebenarnya menggantikan isi area kontainer (setara FrameLayout).
7. **Pengayaan lifecycle.** Ubah `FirstFragment` menjadi StatefulWidget dan tambahkan `print()` pada setiap method lifecycle, lalu amati urutan log di tab **Run** ketika berpindah fragmen:

DART
```
class FirstFragment extends StatefulWidget {
  const FirstFragment({super.key});

  @override
  State<FirstFragment> createState() => _FirstFragmentState();
}
```

```
class _FirstFragmentState extends State<FirstFragment> {
  @override
  void initState() {
    super.initState();
    debugPrint('FirstFragment: initState');
  }

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    debugPrint('FirstFragment: didChangeDependencies');
  }

  @override
  void dispose() {
    debugPrint('FirstFragment: dispose');
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    debugPrint('FirstFragment: build');
    return const Center(child: Text('Ini Fragment Pertama'));
  }
}
```

8. **Pengayaan multi-pane.** Pada layar lebar (tablet/landscape), tampilkan kedua fragmen berdampingan:

```
DART
LayoutBuilder(
  builder: (context, c) => c.maxWidth > 600
      ? const Row(children: [
          Expanded(child: FirstFragment()),
          Expanded(child: SecondFragment()),
        ])
      : _fragmentAktif,
)
```

9. Hasil running akan menampilkan seperti di bawah:

[Tempelkan screenshot hasil running aplikasi]

Gambar 1. Hasil running implementasi fragment (widget modular)

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code fragment (widget modular).

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan fragment (widget modular) pada project individu serta memahami konsep implementasinya.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code fragment	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code fragment	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code fragment	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code fragment	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 20 : FRAGMENT (WIDGET MODULAR)

Pokok Bahasan	:	Analisa Hasil Implementasi Fragment
Acara Praktikum/Praktik	:	Minggu 5 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep fragment.
2. Mahasiswa mampu menerapkan dan mengimplementasikan fragment (widget modular) pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep fragment, implementasi widget modular pada Flutter, serta analisa source code.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

1. Mahasiswa melakukan analisa code untuk setiap baris code dari implementasi fragment (widget modular).
2. Menjelaskan fungsi dan arti dari setiap baris code.
3. Analisa hasil running dari implementasi project fragment, termasuk urutan log lifecycle.
4. Screenshot output setiap proses dari project fragment.
5. Tuliskan dalam bentuk laporan.

f. Hasil dan Pembahasan

Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project fragment yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa memahami konsep dasar fragment dengan mampu memahami setiap baris code dari implementasi fragment (widget modular) pada Flutter.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan secara detail setiap baris code fragment	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan secara singkat setiap baris code fragment	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan code fragment	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan code fragment	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 21 : INTENT (NAVIGATOR DAN URL LAUNCHER)

Pokok Bahasan	:	Konsep Intent pada Flutter
Acara Praktikum/Praktik	:	Minggu 6 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep intent dan padanannya pada Flutter.
2. Mahasiswa mampu menerapkan dan mengimplementasikan intent (Navigator dan url_launcher) pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep intent, implementasi intent pada Flutter, serta analisa source code.

c. Dasar Teori

Intent

Pada Android native, **Intent** adalah kelas yang berfungsi untuk perpindahan halaman (Activity) dan juga mekanisme untuk melakukan sebuah *action* serta komunikasi antar komponen aplikasi (activity, service, dan broadcast receiver). Terdapat tiga penggunaan umum intent:

- Memindahkan satu activity ke activity lain dengan atau tanpa membawa data.
- Menjalankan *background service*, misalnya sinkronisasi ke server dan proses berulang (*periodic/scheduler task*).
- Mengirimkan objek *broadcast* ke aplikasi yang membutuhkan (misalnya event selesai *booting*).

Intent dibagi menjadi 2, yaitu:

1. **Explicit Intent** — mengaktifkan komponen dalam satu aplikasi yang sama dengan menyebutkan nama kelas tujuan, misalnya berpindah Activity.
2. **Implicit Intent** — memanggil fungsi yang sudah disediakan aplikasi lain/sistem Android, seperti membuka browser, *dial number*, kamera, atau berbagi konten, tanpa menyebutkan nama kelasnya.

Intent pada Flutter

Aplikasi Flutter berjalan di dalam **satu Activity** Android. Perpindahan halaman tidak memakai Intent, melainkan **Navigator**, sedangkan aksi ke aplikasi lain dilakukan melalui **package plugin** yang di belakang layar memanggil Intent Android.

Konsep Android	Padanan Flutter
Explicit Intent: new Intent(this, SecondActivity.class) + startActivity()	Navigator.push(context, MaterialPageRoute(builder: (_) => SecondPage()))

Konsep Android	Padanan Flutter
putExtra() / getExtras()	Parameter konstruktor widget tujuan, atau arguments pada named route
startActivityForResult() / setResult()	final hasil = await Navigator.push(...); Navigator.pop(context, data)
finish()	Navigator.pop(context)
Implicit Intent ACTION_VIEW (browser)	url_launcher: launchUrl(Uri.parse('https://...'))
ACTION_DIAL, ACTION_SENDTO (telepon, SMS, email)	url_launcher dengan skema tel:, sms:, mailto:
ACTION_SEND (berbagi konten)	package share_plus: Share.share('teks')
Background service / broadcast	package workmanager, flutter_local_notifications, atau platform channel

Implementasi implicit intent memudahkan pengembang agar tetap fokus pada proses bisnis inti aplikasi; misalnya untuk mengambil foto cukup memanggil aplikasi kamera bawaan (package `image_picker`) daripada membuat fungsi kamera sendiri.

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

1. Mahasiswa mencari video referensi tentang implementasi intent pada Android serta Navigator dan url_launcher pada Flutter.
2. Analisa kegunaan dan manfaat intent dari video yang sudah dilihat.
3. Resume pada kertas A4.

f. Hasil dan Pembahasan

1. Dokumen berupa video referensi yang telah diunduh dan dianalisa.
2. Dokumen tugas resume analisa video dan contoh pada aplikasi yang digunakan.

g. Kesimpulan

Mahasiswa memahami konsep dasar intent dan padanannya di Flutter dengan menerapkan contoh pada aplikasi yang sering digunakan sehari-hari.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan konsep intent ditunjang dengan bukti referensi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan konsep intent	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan konsep intent	1	2	3	4

No	Indikator	Skor			
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan konsep intent	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 22 : INTENT (NAVIGATOR DAN URL LAUNCHER)

Pokok Bahasan	:	Implementasi Intent Implicit
Acara Praktikum/Praktik	:	Minggu 6 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep intent.
2. Mahasiswa mampu menerapkan dan mengimplementasikan intent pada Flutter.
3. Mahasiswa mampu membedakan penggunaan intent implicit dan intent explicit.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep intent, implementasi intent pada Flutter, perbedaan intent implicit dan intent explicit serta analisa source code.

c. Dasar Teori

Package **url_launcher** adalah plugin resmi tim Flutter untuk membuka URL dengan aplikasi lain. Pada Android, plugin ini membuat Intent `ACTION_VIEW` sehingga sistem memilih aplikasi yang mampu menangani URL tersebut (browser untuk `http/https`, aplikasi telepon untuk `tel:`, aplikasi email untuk `mailto:`). Mulai Android 11 (API 30), aplikasi wajib mendeklarasikan jenis intent yang ingin diperiksa di tag `<queries>` pada AndroidManifest.

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Intent Implicit

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `imp_intent_app`.
3. Pastikan target platform Android tercentang.
4. Tambahkan package url_launcher melalui terminal:

```bash
flutter pub add url_launcher
```

5. Buat satu komponen kolom teks (TextField) dan tombol seperti pada source code `lib/main.dart` di bawah ini:

```dart
import 'package:flutter/material.dart';
import 'package:url_launcher/url_launcher.dart';
```

```
void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainPage(),
    ));

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  final _editText = TextEditingController(text: 'https://');

  @override
  void dispose() {
    _editText.dispose();
    super.dispose();
  }

  Future<void> _visit() async {
    final url = Uri.parse(_editText.text.trim());
    // Setara: new Intent(Intent.ACTION_VIEW, Uri.parse(url)); startActivity(intent);
    final berhasil = await launchUrl(url, mode: LaunchMode.externalApplication);
    if (!berhasil && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Tidak dapat membuka $url')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Intent Implicit')),
      body: Padding(
        padding: const EdgeInsets.fromLTRB(24, 60, 24, 24),
        child: Column(
          children: [
            TextField(
              controller: _editText,
              keyboardType: TextInputType.url,
              decoration: const InputDecoration(
                labelText: 'Alamat URL',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _visit, child: const Text('VISIT')),
          ],
        ),
      ),
    );
  }
}
```

6. Terakhir pada file `android/app/src/main/AndroidManifest.xml` tambahkan tag `<queries>` di dalam tag `<manifest>` (di luar `<application>`) agar aplikasi dapat menemukan browser pada Android 11 ke atas:

XML

```
<queries>
    <intent>
        <action android:name="android.intent.action.VIEW" />
        <data android:scheme="https" />
```

```
    </intent>
    <intent>
        <action android:name="android.intent.action.VIEW" />
        <data android:scheme="http" />
    </intent>
</queries>
```

7. (Pengayaan) Agar aplikasi Flutter ini juga muncul sebagai pilihan saat aplikasi lain membuka link `http`, tambahkan `intent-filter` berikut di dalam tag `<activity android:name=".MainActivity">`:

```xml
<intent-filter tools:ignore="AppLinkUrlError">
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:scheme="http" />
</intent-filter>
```

Jika menggunakan atribut `tools:ignore`, pastikan tag `<manifest>` memiliki namespace `xmlns:tools="http://schemas.android.com/tools"`.

8. Hentikan aplikasi lalu jalankan ulang (bukan hot reload, karena ada perubahan plugin dan manifest). Perhatikan apa yang terjadi apabila kolom teks diisi alamat URL kemudian tombol **VISIT** ditekan.
9. (Pengayaan) Coba ganti isi kolom dengan `tel:081234567890` dan `mailto:developer@polije.ac.id`, lalu amati aplikasi apa yang terbuka.

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code intent implicit.

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan intent implicit pada project individu serta memahami konsep implementasi intent implicit.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code intent implicit	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code intent implicit	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code intent implicit	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code intent implicit	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 23 : INTENT (NAVIGATOR DAN URL LAUNCHER)

Pokok Bahasan	:	Implementasi Intent Explicit
Acara Praktikum/Praktik	:	Minggu 6 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep intent.
2. Mahasiswa mampu menerapkan dan mengimplementasikan intent pada Flutter.
3. Mahasiswa mampu membedakan penggunaan intent implicit dan intent explicit.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep intent, implementasi intent pada Flutter, perbedaan intent implicit dan intent explicit serta analisa source code.

c. Dasar Teori

Intent explicit pada Flutter diwujudkan dengan **Navigator**. Navigator mengelola tumpukan halaman (*stack of routes*): `Navigator.push()` menambahkan halaman baru di atas tumpukan, dan `Navigator.pop()` menghapus halaman teratas sehingga kembali ke halaman sebelumnya. Data dikirim ke halaman tujuan melalui **parameter konstruktor** (setara `putExtra`) dan dapat dikembalikan ke halaman asal melalui `Navigator.pop(context, data)` (setara `setResult`).

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

Intent Explicit

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `exp_intent_app`.
3. Pastikan target platform Android tercentang.
4. Buat file `lib/second_page.dart` (setara `SecondActivity` + `activity_second.xml`):

DART
```
import 'package:flutter/material.dart';

class SecondPage extends StatelessWidget {
  // Setara: Bundle extras = getIntent().getExtras(); extras.getString(KEY_NAME)
  final String nama;

  const SecondPage({super.key, required this.nama});
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Second Page')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('Hello, $nama !',
                style: const TextStyle(
                    fontSize: 20, fontWeight: FontWeight.bold)),
            const SizedBox(height: 4),
            const Text('This is second page'),
            const SizedBox(height: 24),
            OutlinedButton(
              // Mengembalikan data ke halaman sebelumnya (setara setResult)
              onPressed: () => Navigator.pop(context, 'Terima kasih, $nama'),
              child: const Text('KEMBALI'),
            ),
          ],
        ),
      ),
    );
  }
}
```

5. Ketikkan source code pada `lib/main.dart` (setara `MainActivity` + `activity_main.xml`):

DART

```
import 'package:flutter/material.dart';
import 'second_page.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainPage(),
    ));

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  final _name = TextEditingController();

  @override
  void dispose() {
    _name.dispose();
    super.dispose();
  }

  Future<void> _send() async {
    final messenger = ScaffoldMessenger.of(context);
    try {
      final nama = _name.text.trim();
      if (nama.isNotEmpty) {
        // Setara: Intent i = new Intent(MainActivity.this, SecondActivity.class);
        //         i.putExtra(KEY_NAME, nama); startActivity(i);
        final balasan = await Navigator.push<String>(
          context,
          MaterialPageRoute(builder: (_) => SecondPage(nama: nama)),
        );
        if (balasan != null) {
          messenger.showSnackBar(SnackBar(content: Text(balasan)));
        }
```

```
      } else {
        messenger.showSnackBar(
          const SnackBar(content: Text('YOU NEED TO FILL YOUR NAME')),
        );
      }
    } catch (e) {
      debugPrint(e.toString());
      messenger.showSnackBar(
        const SnackBar(content: Text('ERROR, TRY AGAIN !')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Intent Explicit')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const Text('Who are you?'),
            const SizedBox(height: 24),
            Row(
              children: [
                const Text('I am '),
                const SizedBox(width: 8),
                Expanded(
                  child: TextField(
                    controller: _name,
                    textCapitalization: TextCapitalization.words,
                    decoration: const InputDecoration(hintText: 'Nama'),
                  ),
                ),
              ],
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _send, child: const Text('SEND')),
          ],
        ),
      ),
    );
  }
}
```

Perhatikan perbedaan dengan kode Java pada modul Android: perbandingan `nama != ""` di Java membandingkan referensi objek (bukan isi teks) sehingga sebaiknya memakai `isEmpty()`. Di Dart, validasi dilakukan dengan `nama.isNotEmpty`. Selain itu Toast di Java harus diakhiri `.show()`; di Flutter pesan ditampilkan dengan `showSnackBar()`.

6. (Pengayaan) **Named route** — cara lain melakukan navigasi dengan nama rute (mirip deklarasi activity di manifest):

DART
```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const MainPage(),
  },
  onGenerateRoute: (settings) {
    if (settings.name == '/second') {
      final nama = settings.arguments as String;
      return MaterialPageRoute(builder: (_) => SecondPage(nama: nama));
    }
    return null;
  },
);
```

```
// Pemanggilan:
Navigator.pushNamed(context, '/second', arguments: 'Polije');
```

7. Build dan run project tersebut, lalu perhatikan hasilnya.

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code intent explicit.

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan intent explicit (Navigator) pada project individu serta memahami konsep implementasi intent explicit.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code intent explicit	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code intent explicit	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code intent explicit	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code intent explicit	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 24 : INTENT (NAVIGATOR DAN URL LAUNCHER)

Pokok Bahasan	:	Analisa Code Intent Implicit dan Intent Explicit
Acara Praktikum/Praktik	:	Minggu 6 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep intent.
2. Mahasiswa mampu menerapkan dan mengimplementasikan intent pada Flutter.
3. Mahasiswa mampu membedakan penggunaan intent implicit dan intent explicit.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep intent, implementasi intent pada Flutter, perbedaan intent implicit dan intent explicit serta analisa source code.

c. Dasar Teori

-

d. Alat dan Bahan

1. Kertas A4
2. Jaringan Internet

e. Prosedur Kerja

1. Mahasiswa melakukan analisa code untuk setiap baris code dari implementasi intent implicit (url_launcher) dan intent explicit (Navigator).
2. Menjelaskan fungsi dan arti dari setiap baris code.
3. Analisa hasil running dari implementasi project intent implicit dan intent explicit.
4. Screenshot output setiap proses dari project intent implicit dan intent explicit.
5. Tuliskan dalam bentuk laporan.

f. Hasil dan Pembahasan

Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project intent implicit dan intent explicit yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa memahami konsep dasar intent implicit dan intent explicit dengan mampu memahami setiap baris code dari implementasi intent implicit dan intent explicit.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan secara detail setiap baris code intent implicit dan explicit	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan secara singkat setiap baris code intent implicit dan explicit	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan code intent implicit dan explicit	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan code intent implicit dan explicit	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 25 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE

Pokok Bahasan	:	Konsep Manajemen File pada Flutter
Acara Praktikum/Praktik	:	Minggu 7 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep manajemen file.
2. Mahasiswa mampu menerapkan dan mengimplementasikan manajemen file pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep manajemen file, implementasi manajemen file pada Flutter, serta analisa source code.

c. Dasar Teori

Manajemen File

Read dan write file diperlukan ketika aplikasi ingin menyimpan data berupa file teks, membaca data dari penyimpanan perangkat, atau menyimpan file unduhan. Data disimpan dalam file yang ditentukan oleh pengguna/aplikasi dan dapat diakses kembali saat aplikasi dibuka.

Pada Flutter, operasi file dilakukan dengan kelas `File` dan `Directory` dari library **dart:io** (method `writeAsString()`, `readAsString()`, `exists()`, `delete()`, `list()`), sedangkan lokasi folder yang benar pada setiap platform diperoleh dari package **path_provider**:

Android (Java)	Flutter (path_provider)	Keterangan
getFilesDir()	getApplicationDocumentsDirectory()	Penyimpanan internal privat milik aplikasi. Terhapus saat aplikasi di-uninstall. Tidak perlu izin.
getCacheDir()	getTemporaryDirectory()	Folder cache sementara, dapat dihapus sistem kapan saja.
getExternalFilesDir(String type)	getExternalStorageDirectory() (khusus Android)	Penyimpanan eksternal khusus aplikasi; file terhapus saat aplikasi di-uninstall. Cara yang disarankan untuk data pribadi aplikasi.

Android (Java)	Flutter (path_provider)	Keterangan
getExternalStoragePublicDirectory()	getDownloadsDirectory() / MediaStore melalui plugin	Penyimpanan publik yang tidak terhapus saat aplikasi di-uninstall (contoh: foto kamera). Mulai Android 10 dibatasi oleh Scoped Storage.
getExternalStorageDirectory() (cara lama)	Tidak disarankan	Akses langsung ke root kartu SD; sudah usang (*deprecated*) pada Android modern.

Karena operasi file membutuhkan waktu, seluruh method di atas bersifat **asynchronous** dan mengembalikan `Future`, sehingga dipanggil menggunakan kata kunci `async`/`await`.

d. Alat dan Bahan

1. Media Penyimpanan
2. Kertas A4
3. Jaringan Internet

e. Prosedur Kerja

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `management_file`.
3. Pastikan target platform Android tercentang.
4. Tambahkan package yang dibutuhkan:

```bash
flutter pub add path_provider
```

5. Buat file `lib/main.dart` (halaman tulis file, setara MainActivity) dan `lib/baca_file_page.dart` (halaman baca file, setara Main2Activity). Isi kedua file dikerjakan pada Acara 26.

f. Hasil dan Pembahasan

Folder project aplikasi untuk implementasi code manajemen file.

g. Kesimpulan

Mahasiswa berhasil memahami konsep manajemen file serta membuat folder project aplikasi untuk implementasi code manajemen file.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan konsep manajemen file serta menyiapkan project secara lengkap	1	2	3	4

No	Indikator	Skor			
2	Ketepatan waktu dan ketepatan dalam menjelaskan konsep manajemen file secara singkat	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan konsep manajemen file	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan konsep manajemen file	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 26 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE

Pokok Bahasan	:	Implementasi Manajemen File pada Flutter
Acara Praktikum/Praktik	:	Minggu 7 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep manajemen file.
2. Mahasiswa mampu menerapkan dan mengimplementasikan manajemen file pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep manajemen file, implementasi manajemen file pada Flutter, serta analisa source code.

c. Dasar Teori

-

d. Alat dan Bahan

1. Media Penyimpanan
2. Kertas A4
3. Jaringan Internet

e. Prosedur Kerja

Manajemen File

1. Buat file `lib/file_helper.dart` yang berisi fungsi-fungsi pengelolaan file:

DART
```
import 'dart:io';
import 'package:path_provider/path_provider.dart';

enum LokasiPenyimpanan { internal, eksternalAplikasi }

class FileHelper {
  // Menentukan folder sesuai pilihan lokasi
  static Future<Directory> _folder(LokasiPenyimpanan lokasi) async {
    if (lokasi == LokasiPenyimpanan.eksternalAplikasi && Platform.isAndroid) {
      final dir = await getExternalStorageDirectory(); // = getExternalFilesDir(null)
      if (dir != null) return dir;
    }
    return getApplicationDocumentsDirectory(); // = getFilesDir()
  }

  static Future<File> tulis(String namaFile, String isi,
      LokasiPenyimpanan lokasi) async {
    final dir = await _folder(lokasi);
    final file = File('${dir.path}/$namaFile');
    return file.writeAsString(isi);
  }
```

```
  static Future<String> baca(String namaFile, LokasiPenyimpanan lokasi) async {
    final dir = await _folder(lokasi);
    final file = File('${dir.path}/$namaFile');
    if (!await file.exists()) {
      throw FileSystemException('File tidak ditemukan', file.path);
    }
    return file.readAsString();
  }

  static Future<List<String>> daftarFile(LokasiPenyimpanan lokasi) async {
    final dir = await _folder(lokasi);
    return dir
        .listSync()
        .whereType<File>()
        .where((f) => f.path.endsWith('.txt'))
        .map((f) => f.uri.pathSegments.last)
        .toList();
  }

  static Future<void> hapus(String namaFile, LokasiPenyimpanan lokasi) async {
    final dir = await _folder(lokasi);
    final file = File('${dir.path}/$namaFile');
    if (await file.exists()) await file.delete();
  }
}
```

2. Ubah source `lib/main.dart` (halaman menulis file, setara `MainActivity.java` + `activity_main.xml`):

DART

```
import 'package:flutter/material.dart';
import 'file_helper.dart';
import 'baca_file_page.dart';

void main() => runApp(MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Manajemen File',
      theme: ThemeData(colorSchemeSeed: Colors.teal),
      home: const MainPage(),
    ));

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  final _namaFile = TextEditingController(text: 'catatan.txt');
  final _isiFile = TextEditingController();
  LokasiPenyimpanan _lokasi = LokasiPenyimpanan.internal;

  @override
  void dispose() {
    _namaFile.dispose();
    _isiFile.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    var nama = _namaFile.text.trim();
    if (nama.isEmpty || _isiFile.text.isEmpty) {
      _pesan('Nama file dan isi tidak boleh kosong');
      return;
    }
    if (!nama.endsWith('.txt')) nama = '$nama.txt';
    try {
```

```
      final file = await FileHelper.tulis(nama, _isiFile.text, _lokasi);
      _pesan('Tersimpan di ${file.path}');
      _isiFile.clear();
    } catch (e) {
      _pesan('Gagal menyimpan: $e');
    }
  }

  void _pesan(String teks) {
    ScaffoldMessenger.of(context)
        .showSnackBar(SnackBar(content: Text(teks)));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tulis File')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          SegmentedButton<LokasiPenyimpanan>(
            segments: const [
              ButtonSegment(
                  value: LokasiPenyimpanan.internal,
                  label: Text('Internal'),
                  icon: Icon(Icons.phone_android)),
              ButtonSegment(
                  value: LokasiPenyimpanan.eksternalAplikasi,
                  label: Text('Eksternal'),
                  icon: Icon(Icons.sd_card)),
            ],
            selected: {_lokasi},
            onSelectionChanged: (s) => setState(() => _lokasi = s.first),
          ),
          const SizedBox(height: 16),
          TextField(
            controller: _namaFile,
            decoration: const InputDecoration(
                labelText: 'Nama File', border: OutlineInputBorder()),
          ),
          const SizedBox(height: 12),
          TextField(
            controller: _isiFile,
            maxLines: 8,
            decoration: const InputDecoration(
                labelText: 'Isi File',
                alignLabelWithHint: true,
                border: OutlineInputBorder()),
          ),
          const SizedBox(height: 16),
          FilledButton.icon(
            onPressed: _simpan,
            icon: const Icon(Icons.save),
            label: const Text('SIMPAN FILE'),
          ),
          const SizedBox(height: 8),
          OutlinedButton.icon(
            onPressed: () => Navigator.push(
              context,
              MaterialPageRoute(builder: (_) => BacaFilePage(lokasi: _lokasi)),
            ),
            icon: const Icon(Icons.folder_open),
            label: const Text('LIHAT DAFTAR FILE'),
          ),
        ],
      ),
    );
  }
}
```

3. Tambahkan source pada `lib/baca_file_page.dart` (halaman membaca file, setara `Main2Activity.java` + `activity_main2.xml`):

DART

```
import 'package:flutter/material.dart';
import 'file_helper.dart';

class BacaFilePage extends StatefulWidget {
  final LokasiPenyimpanan lokasi;
  const BacaFilePage({super.key, required this.lokasi});

  @override
  State<BacaFilePage> createState() => _BacaFilePageState();
}

class _BacaFilePageState extends State<BacaFilePage> {
  late Future<List<String>> _daftar;

  @override
  void initState() {
    super.initState();
    _daftar = FileHelper.daftarFile(widget.lokasi);
  }

  void _muatUlang() {
    setState(() {
      _daftar = FileHelper.daftarFile(widget.lokasi);
    });
  }

  Future<void> _bukaFile(String nama) async {
    final isi = await FileHelper.baca(nama, widget.lokasi);
    if (!mounted) return;
    showDialog(
      context: context,
      builder: (_) => AlertDialog(
        title: Text(nama),
        content: SingleChildScrollView(child: Text(isi)),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('TUTUP'),
          ),
        ],
      ),
    );
  }

  Future<void> _hapusFile(String nama) async {
    await FileHelper.hapus(nama, widget.lokasi);
    _muatUlang();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar File')),
      body: FutureBuilder<List<String>>(
        future: _daftar,
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          final files = snapshot.data ?? [];
          if (files.isEmpty) {
            return const Center(child: Text('Belum ada file'));
          }
          return ListView.builder(
```

```
            itemCount: files.length,
            itemBuilder: (_, i) => ListTile(
              leading: const Icon(Icons.description),
              title: Text(files[i]),
              onTap: () => _bukaFile(files[i]),
              trailing: IconButton(
                icon: const Icon(Icons.delete),
                onPressed: () => _hapusFile(files[i]),
              ),
            ),
          );
        },
      ),
    );
  }
}
```

4. **Konstanta ukuran dan teks.** Pada Android, ukuran disimpan di `dimens.xml` dan teks di `strings.xml`. Pada Flutter cukup dibuat konstanta Dart, misalnya pada `lib/konstanta.dart`:

DART
```
class Dimens {
  static const double padding = 16;
  static const double jarak = 12;
}

class Strings {
  static const String appName = 'Manajemen File';
  static const String simpan = 'SIMPAN FILE';
}
```

5. **Warna** diatur melalui `ThemeData(colorSchemeSeed: Colors.teal)` pada `main.dart` (setara `colors.xml`).
6. **AndroidManifest.xml.** Penyimpanan internal dan eksternal khusus aplikasi (*app-specific*) **tidak memerlukan izin** apa pun mulai Android 4.4. Izin `READ_EXTERNAL_STORAGE`/`WRITE_EXTERNAL_STORAGE` hanya diperlukan bila mengakses folder publik pada Android 9 ke bawah, misalnya:

XML
```
<uses-permission
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="28" />
```

7. Build dan run project tersebut, lalu perhatikan hasilnya. Simpan beberapa file, buka halaman daftar file, baca isi file dan hapus file.
8. Gunakan **Device Explorer** di Android Studio (*View → Tool Windows → Device Explorer*) untuk melihat lokasi file: internal di `/data/data/<package>/app_flutter/` dan eksternal di `/storage/emulated/0/Android/data/<package>/files/`.

[Tempelkan screenshot hasil running aplikasi]

Gambar 1. Hasil Layout Manajemen File

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code manajemen file.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project manajemen file yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan manajemen file pada project individu serta memahami konsep implementasi manajemen file.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code manajemen file	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code manajemen file	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code manajemen file	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code manajemen file	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 27 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE

Pokok Bahasan	:	Konsep SQLite
Acara Praktikum/Praktik	:	Minggu 7 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep SQLite.
2. Mahasiswa mampu menerapkan dan mengimplementasikan SQLite pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep SQLite, implementasi SQLite pada Flutter, serta analisa source code.

c. Dasar Teori

SQLite

SQLite merupakan database yang bersifat ACID-compliant dan memiliki ukuran pustaka kode yang relatif kecil, ditulis dalam bahasa C. SQLite merupakan proyek *public domain* yang dikerjakan oleh D. Richard Hipp. SQLite adalah database open source yang sudah lama ada, stabil, dan sangat terkenal pada perangkat kecil, termasuk Android dan iOS. Aplikasi dapat memanfaatkannya untuk menyimpan data relasional secara aman dan efisien.

1. Teknik Penyimpanan pada Flutter

Flutter memiliki beberapa teknik penyimpanan data. Teknik yang umum digunakan adalah sebagai berikut:

- **Shared Preferences** (package `shared_preferences`), yaitu menyimpan beberapa nilai sederhana dalam bentuk pasangan *key–value*.
- **Files** (`dart:io` + `path_provider`), yaitu menyimpan data dalam bentuk file, berupa menulis ke file atau membaca dari file.
- **SQLite Database** (package `sqflite`), yaitu menyimpan data dalam bentuk database relasional.
- **Database lain / ORM** seperti `drift`, `hive`, `isar`, atau penyimpanan *cloud* (Firebase). Pada bagian ini akan fokus membahas SQLite dengan `sqflite`.

Tidak ada database yang otomatis disediakan; jika menggunakan SQLite kita harus membuat database sendiri, mendefinisikan tabel, index, serta datanya. Pada Android native digunakan kelas `SQLiteOpenHelper` yang menyediakan tiga bagian utama. Pada `sqflite`, ketiga bagian tersebut terdapat pada fungsi `openDatabase()`:

SQLiteOpenHelper (Android)	openDatabase() (sqflite)	Fungsi
Constructor (nama & versi database)	path dan parameter version	Menyediakan representasi versi dan skema database yang digunakan.
onCreate()	callback onCreate: (db, version)	Membuat definisi tabel dan inisialisasi data saat database pertama kali dibuat.
onUpgrade()	callback onUpgrade: (db, oldVersion, newVersion)	Konversi database dari versi lama ke versi baru.

Operasi data menggunakan method `db.insert()`, `db.query()`, `db.update()`, `db.delete()` atau `db.rawQuery()`/`db.rawInsert()` untuk perintah SQL langsung. Semua method bersifat asynchronous (`Future`).

2. Relasi antara SQLite dengan Flutter

SQLite tertanam di sistem operasi Android dan iOS, dan package `sqflite` menjadi jembatan (*plugin*) antara kode Dart dengan engine SQLite native. Alasan SQLite cocok untuk pengembangan aplikasi mobile:

- Database dengan konfigurasi nol; tidak ada pengaturan server untuk developer.
- Tidak memiliki server (*serverless*).
- Tidak ada proses database yang berjalan terpisah; satu set library menyediakan fungsionalitas database.
- *Single-file database*; seluruh database tersimpan dalam satu file sehingga mudah dikelola.
- Open source, sehingga memudahkan developer dalam pengembangan aplikasi.

d. Alat dan Bahan

1. Media Penyimpanan
2. Kertas A4
3. Jaringan Internet

e. Prosedur Kerja

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `implementasi_sqlite`.
3. Pastikan target platform Android tercentang.
4. Tambahkan package sqflite dan path:

```bash
flutter pub add sqflite path
```

5. Buat struktur file berikut di dalam folder `lib/` (isi kode dikerjakan pada Acara 28):

```
lib/
├── main.dart              (menu utama, setara MainActivity)
├── data_helper.dart       (pembuatan database, setara DataHelper.java)
├── biodata.dart           (model data)
```

```
├── buat_biodata.dart      (setara BuatBiodata.java)
├── lihat_biodata.dart     (setara LihatBiodata.java)
└── update_biodata.dart    (setara UpdateBiodata.java)
```

f. Hasil dan Pembahasan

Folder project aplikasi dari percobaan implementasi code SQLite.

g. Kesimpulan

Mahasiswa berhasil memahami konsep SQLite serta membuat folder project aplikasi untuk implementasi code SQLite.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan konsep SQLite serta menyiapkan project secara lengkap	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan konsep SQLite secara singkat	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan konsep SQLite	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan konsep SQLite	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 28 : MANAJEMEN FILE PADA FLUTTER DAN SQLITE

Pokok Bahasan	:	Implementasi SQLite
Acara Praktikum/Praktik	:	Minggu 7 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep SQLite.
2. Mahasiswa mampu menerapkan dan mengimplementasikan SQLite pada Flutter.

b. Indikator

Keberhasilan mahasiswa dalam memahami konsep SQLite, implementasi SQLite pada Flutter, serta analisa source code.

c. Dasar Teori

-

d. Alat dan Bahan

1. Media Penyimpanan
2. Kertas A4
3. Jaringan Internet

e. Prosedur Kerja

1. **Buat file** `lib/biodata.dart` sebagai model data:

DART
```
class Biodata {
  final int no;
  final String nama;
  final String tgl;
  final String jk;
  final String alamat;

  const Biodata({
    required this.no,
    required this.nama,
    required this.tgl,
    required this.jk,
    required this.alamat,
  });

  // Mengubah Map hasil query menjadi objek Biodata
  factory Biodata.fromMap(Map<String, Object?> map) => Biodata(
        no: map['no'] as int,
        nama: map['nama'] as String? ?? '',
        tgl: map['tgl'] as String? ?? '',
        jk: map['jk'] as String? ?? '',
        alamat: map['alamat'] as String? ?? '',
      );

  // Mengubah objek menjadi Map untuk disimpan ke tabel
  Map<String, Object?> toMap() =>
```

```
      {'no': no, 'nama': nama, 'tgl': tgl, 'jk': jk, 'alamat': alamat};
}
```

2. **Buat file `lib/data_helper.dart`.** File ini adalah proses pembuatan database SQLite dengan nama `biodatadiri.db`:

DART

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import 'biodata.dart';

class DataHelper {
  static const String _databaseName = 'biodatadiri.db';
  static const int _databaseVersion = 1;
  static Database? _db;

  // Setara constructor SQLiteOpenHelper + getWritableDatabase()
  static Future<Database> get database async {
    _db ??= await openDatabase(
      join(await getDatabasesPath(), _databaseName),
      version: _databaseVersion,
      onCreate: _onCreate,
      onUpgrade: _onUpgrade,
    );
    return _db!;
  }

  static Future<void> _onCreate(Database db, int version) async {
    await db.execute('''
      CREATE TABLE biodata(
        no INTEGER PRIMARY KEY,
        nama TEXT,
        tgl TEXT,
        jk TEXT,
        alamat TEXT
      )''');
    await db.insert('biodata', {
      'no': 1,
      'nama': 'Darsiwan',
      'tgl': '1996-07-12',
      'jk': 'Laki-laki',
      'alamat': 'Jember',
    });
  }

  static Future<void> _onUpgrade(Database db, int oldV, int newV) async {
    await db.execute('DROP TABLE IF EXISTS biodata');
    await _onCreate(db, newV);
  }

  // READ semua data
  static Future<List<Biodata>> semua() async {
    final db = await database;
    final rows = await db.query('biodata', orderBy: 'no');
    return rows.map(Biodata.fromMap).toList();
  }

  // READ satu data berdasarkan nama
  static Future<Biodata?> cariNama(String nama) async {
    final db = await database;
    final rows = await db.query('biodata',
        where: 'nama = ?', whereArgs: [nama], limit: 1);
    return rows.isEmpty ? null : Biodata.fromMap(rows.first);
  }

  // CREATE
  static Future<int> tambah(Biodata b) async {
    final db = await database;
```

```
    return db.insert('biodata', b.toMap(),
        conflictAlgorithm: ConflictAlgorithm.abort);
  }

  // UPDATE
  static Future<int> ubah(Biodata b) async {
    final db = await database;
    return db.update('biodata', b.toMap(),
        where: 'no = ?', whereArgs: [b.no]);
  }

  // DELETE
  static Future<int> hapus(String nama) async {
    final db = await database;
    return db.delete('biodata', where: 'nama = ?', whereArgs: [nama]);
  }
}
```

> Gunakan parameter `whereArgs` (tanda `?`) dan jangan menyambung string SQL dengan input pengguna seperti `"... WHERE nama = '" + nama + "'"`, karena rawan **SQL Injection** dan error ketika nama mengandung tanda petik.

3. **Tuliskan source di bawah pada `lib/main.dart`** (menu utama berisi daftar nama, setara `MainActivity.java` + `activity_main.xml`):

DART

```
import 'package:flutter/material.dart';
import 'biodata.dart';
import 'data_helper.dart';
import 'buat_biodata.dart';
import 'lihat_biodata.dart';
import 'update_biodata.dart';

void main() => runApp(MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Biodata SQLite',
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const MainPage(),
    ));

class MainPage extends StatefulWidget {
  const MainPage({super.key});

  @override
  State<MainPage> createState() => _MainPageState();
}

class _MainPageState extends State<MainPage> {
  List<Biodata> _daftar = [];

  @override
  void initState() {
    super.initState();
    refreshList();
  }

  Future<void> refreshList() async {
    final data = await DataHelper.semua();
    if (!mounted) return;
    setState(() => _daftar = data);
  }

  Future<void> _buka(Widget halaman) async {
    await Navigator.push(
        context, MaterialPageRoute(builder: (_) => halaman));
    refreshList(); // setara onResume()
```

```
  }

  void _pilihan(Biodata b) {
    showDialog(
      context: context,
      builder: (ctx) => SimpleDialog(
        title: const Text('Pilihan'),
        children: [
          SimpleDialogOption(
            child: const Text('Lihat Biodata'),
            onPressed: () {
              Navigator.pop(ctx);
              _buka(LihatBiodata(nama: b.nama));
            },
          ),
          SimpleDialogOption(
            child: const Text('Update Biodata'),
            onPressed: () {
              Navigator.pop(ctx);
              _buka(UpdateBiodata(nama: b.nama));
            },
          ),
          SimpleDialogOption(
            child: const Text('Hapus Biodata'),
            onPressed: () async {
              Navigator.pop(ctx);
              await DataHelper.hapus(b.nama);
              refreshList();
            },
          ),
        ],
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Biodata Diri')),
      body: _daftar.isEmpty
          ? const Center(child: Text('Data masih kosong'))
          : ListView.builder(
              itemCount: _daftar.length,
              itemBuilder: (_, i) => ListTile(
                leading: CircleAvatar(child: Text('${_daftar[i].no}')),
                title: Text(_daftar[i].nama),
                subtitle: Text(_daftar[i].alamat),
                onTap: () => _pilihan(_daftar[i]),
              ),
            ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _buka(const BuatBiodata()),
        icon: const Icon(Icons.add),
        label: const Text('Buat Biodata'),
      ),
    );
  }
}
```

4. **Buat 3 halaman baru** dengan nama file berikut: `buat_biodata.dart`, `lihat_biodata.dart`, dan `update_biodata.dart`. Pada Flutter, layout dan logika ditulis pada satu file (tidak ada pemisahan XML dan Java).
5. Tuliskan baris kode di bawah dengan nama masing-masing halaman:

lib/buat_biodata.dart

DART
```
import 'package:flutter/material.dart';
import 'biodata.dart';
```

```
import 'data_helper.dart';

class BuatBiodata extends StatefulWidget {
  const BuatBiodata({super.key});

  @override
  State<BuatBiodata> createState() => _BuatBiodataState();
}

class _BuatBiodataState extends State<BuatBiodata> {
  final _formKey = GlobalKey<FormState>();
  final _no = TextEditingController();
  final _nama = TextEditingController();
  final _tgl = TextEditingController();
  final _alamat = TextEditingController();
  String _jk = 'Laki-laki';

  @override
  void dispose() {
    for (final c in [_no, _nama, _tgl, _alamat]) {
      c.dispose();
    }
    super.dispose();
  }

  Future<void> _pilihTanggal() async {
    final t = await showDatePicker(
      context: context,
      initialDate: DateTime(2005),
      firstDate: DateTime(1950),
      lastDate: DateTime.now(),
    );
    if (t != null) {
      _tgl.text = t.toIso8601String().substring(0, 10);
    }
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final messenger = ScaffoldMessenger.of(context);
    final navigator = Navigator.of(context);
    try {
      await DataHelper.tambah(Biodata(
        no: int.parse(_no.text),
        nama: _nama.text.trim(),
        tgl: _tgl.text,
        jk: _jk,
        alamat: _alamat.text.trim(),
      ));
      messenger.showSnackBar(const SnackBar(content: Text('Berhasil')));
      navigator.pop();
    } catch (e) {
      messenger.showSnackBar(
          const SnackBar(content: Text('Gagal: nomor sudah digunakan')));
    }
  }

  String? _wajib(String? v) =>
      (v == null || v.trim().isEmpty) ? 'Wajib diisi' : null;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Buat Biodata')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
```

```
            TextFormField(
              controller: _no,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(labelText: 'Nomor'),
              validator: (v) =>
                  int.tryParse(v ?? '') == null ? 'Isi dengan angka' : null,
            ),
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(labelText: 'Nama'),
              validator: _wajib,
            ),
            TextFormField(
              controller: _tgl,
              readOnly: true,
              onTap: _pilihTanggal,
              decoration: const InputDecoration(
                  labelText: 'Tanggal Lahir (yyyy-mm-dd)',
                  suffixIcon: Icon(Icons.calendar_today)),
              validator: _wajib,
            ),
            DropdownButtonFormField<String>(
              value: _jk,
              decoration: const InputDecoration(labelText: 'Jenis Kelamin'),
              items: const [
                DropdownMenuItem(value: 'Laki-laki', child: Text('Laki-laki')),
                DropdownMenuItem(value: 'Perempuan', child: Text('Perempuan')),
              ],
              onChanged: (v) => setState(() => _jk = v!),
            ),
            TextFormField(
              controller: _alamat,
              maxLines: 2,
              decoration: const InputDecoration(labelText: 'Alamat'),
              validator: _wajib,
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _simpan, child: const Text('SIMPAN')),
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('BACK'),
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/lihat_biodata.dart

DART

```
import 'package:flutter/material.dart';
import 'biodata.dart';
import 'data_helper.dart';


class LihatBiodata extends StatelessWidget {
  final String nama;
  const LihatBiodata({super.key, required this.nama});


  Widget _baris(String label, String nilai) => Padding(
        padding: const EdgeInsets.symmetric(vertical: 6),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            SizedBox(width: 120, child: Text(label)),
            const Text(': '),
            Expanded(child: Text(nilai,
                style: const TextStyle(fontWeight: FontWeight.bold))),
          ],
        ),
      );
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Lihat Biodata')),
      body: FutureBuilder<Biodata?>(
        future: DataHelper.cariNama(nama),
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          final b = snapshot.data;
          if (b == null) {
            return const Center(child: Text('Data tidak ditemukan'));
          }
          return Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              children: [
                _baris('Nomor', '${b.no}'),
                _baris('Nama', b.nama),
                _baris('Tanggal Lahir', b.tgl),
                _baris('Jenis Kelamin', b.jk),
                _baris('Alamat', b.alamat),
                const SizedBox(height: 24),
                FilledButton(
                  onPressed: () => Navigator.pop(context),
                  child: const Text('BACK'),
                ),
              ],
            ),
          );
        },
      ),
    );
  }
}
```

lib/update_biodata.dart

DART

```
import 'package:flutter/material.dart';
import 'biodata.dart';
import 'data_helper.dart';

class UpdateBiodata extends StatefulWidget {
  final String nama;
  const UpdateBiodata({super.key, required this.nama});

  @override
  State<UpdateBiodata> createState() => _UpdateBiodataState();
}

class _UpdateBiodataState extends State<UpdateBiodata> {
  final _nama = TextEditingController();
  final _tgl = TextEditingController();
  final _alamat = TextEditingController();
  String _jk = 'Laki-laki';
  int? _no;

  @override
  void initState() {
    super.initState();
    _muatData();
  }

  Future<void> _muatData() async {
    final b = await DataHelper.cariNama(widget.nama);
    if (b == null || !mounted) return;
    setState(() {
```

```
      _no = b.no;
      _nama.text = b.nama;
      _tgl.text = b.tgl;
      _jk = b.jk.isEmpty ? 'Laki-laki' : b.jk;
      _alamat.text = b.alamat;
    });
  }

  @override
  void dispose() {
    _nama.dispose();
    _tgl.dispose();
    _alamat.dispose();
    super.dispose();
  }

  Future<void> _update() async {
    if (_no == null) return;
    final messenger = ScaffoldMessenger.of(context);
    final navigator = Navigator.of(context);
    await DataHelper.ubah(Biodata(
      no: _no!,
      nama: _nama.text.trim(),
      tgl: _tgl.text.trim(),
      jk: _jk,
      alamat: _alamat.text.trim(),
    ));
    messenger.showSnackBar(const SnackBar(content: Text('Berhasil')));
    navigator.pop();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Update Biodata')),
      body: _no == null
          ? const Center(child: CircularProgressIndicator())
          : ListView(
              padding: const EdgeInsets.all(16),
              children: [
                Text('Nomor: $_no'),
                TextField(
                    controller: _nama,
                    decoration: const InputDecoration(labelText: 'Nama')),
                TextField(
                    controller: _tgl,
                    decoration: const InputDecoration(
                        labelText: 'Tanggal Lahir (yyyy-mm-dd)')),
                DropdownButtonFormField<String>(
                  value: _jk,
                  decoration:
                      const InputDecoration(labelText: 'Jenis Kelamin'),
                  items: const [
                    DropdownMenuItem(
                        value: 'Laki-laki', child: Text('Laki-laki')),
                    DropdownMenuItem(
                        value: 'Perempuan', child: Text('Perempuan')),
                  ],
                  onChanged: (v) => setState(() => _jk = v!),
                ),
                TextField(
                    controller: _alamat,
                    decoration: const InputDecoration(labelText: 'Alamat')),
                const SizedBox(height: 24),
                FilledButton(onPressed: _update, child: const Text('UPDATE')),
                TextButton(
                  onPressed: () => Navigator.pop(context),
                  child: const Text('BACK'),
                ),
              ],
            ),
```

```
    );
  }
}
```

> Pada versi Flutter terbaru, parameter `value` pada `DropdownButtonFormField` dapat ditandai *deprecated* dan diganti `initialValue`. Jika muncul peringatan tersebut, cukup ganti `value:` menjadi `initialValue:`.

6. **Tambahan: AndroidManifest.xml.** Tidak ada izin yang perlu ditambahkan karena database disimpan di penyimpanan internal aplikasi, dan tidak ada activity yang perlu didaftarkan karena seluruh halaman adalah widget.
7. **Warna dan style** diatur melalui `ThemeData` pada `main.dart` (setara `color.xml` dan `styles.xml`).
8. Build dan run project tersebut, lalu perhatikan hasilnya. Hasil run project akan menghasilkan layout seperti di bawah:

[Tempelkan screenshot hasil running aplikasi]

Gambar 2. Hasil Menu Utama (daftar biodata)

[Tempelkan screenshot hasil running aplikasi]

Gambar 3. Hasil dialog Pilihan (Lihat, Update, Hapus)

[Tempelkan screenshot hasil running aplikasi]

Gambar 4. Hasil Buat Biodata

[Tempelkan screenshot hasil running aplikasi]

Gambar 5. Hasil Lihat Biodata

[Tempelkan screenshot hasil running aplikasi]

Gambar 6. Hasil Update Biodata

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code SQLite.
2. Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project SQLite yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa berhasil mengimplementasikan SQLite pada project individu serta memahami konsep implementasi SQLite.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code SQLite	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code SQLite	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code SQLite	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code SQLite	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 29 : SHARED PREFERENCES

Pokok Bahasan	:	Konsep Shared Preferences
Acara Praktikum/Praktik	:	Minggu 9 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep shared preferences.
2. Mahasiswa mampu membuat desain sederhana untuk form register dan form login.

b. Indikator Penilaian

Keberhasilan dalam memahami konsep shared preferences, serta membuat desain sederhana form register dan form login.

c. Dasar Teori

Shared Preferences

Terdapat beberapa opsi penyimpanan (*storage*) pada aplikasi mobile. Salah satunya adalah **Shared Preferences**. Karakter penyimpanan ini adalah *key–value storage*, sehingga hanya bisa menyimpan data dengan tipe sederhana. Pada Flutter, package **shared_preferences** mendukung tipe `String`, `int`, `double`, `bool`, dan `List<String>`.

Shared Preferences menyimpan data melalui pasangan nama/nilai. Kita menentukan nama (*key*) untuk data yang ingin disimpan, kemudian key beserta nilainya akan disimpan secara otomatis. Di Android, package ini memakai `SharedPreferences` native (file XML), di iOS memakai `NSUserDefaults`, dan di web memakai `localStorage`. Oleh karena itu, penyimpanan ini tergolong sangat minimalis dan cocok untuk data yang sedikit, misalnya status login, nama pengguna, tema gelap/terang, atau pengaturan bahasa. Data yang bersifat rahasia (password/token) sebaiknya disimpan dengan `flutter_secure_storage`.

Contoh penggunaan dasar:

```dart
final prefs = await SharedPreferences.getInstance();
await prefs.setString('username', 'polije');   // simpan
final user = prefs.getString('username');      // baca (null jika belum ada)
await prefs.remove('username');                // hapus
```

d. Alat dan Bahan

1. Kertas A4
2. Spidol

e. Prosedur Kerja

1. Praktikum ini akan menggunakan Shared Preferences untuk pembuatan aplikasi login dan register sederhana dengan Flutter.

2. Aplikasi ini terdapat fitur login, register, dan logout.
3. Dengan memanfaatkan Shared Preferences, meskipun aplikasi sudah ditutup, status login tetap tersimpan sehingga aplikasi tidak *logout* kecuali pengguna memutuskan untuk logout.
4. Dengan demikian pengguna tidak perlu melakukan login berulang kali setiap membuka aplikasinya.
5. Mahasiswa membuat desain sederhana untuk form register dan form login pada kertas A4, lengkap dengan nama widget Flutter yang akan digunakan (TextField/TextFormField, FilledButton, TextButton, dan sebagainya).

f. Hasil dan Pembahasan

Dokumen tugas desain sederhana form register dan form login, dilengkapi dengan penjelasan fungsi setiap item yang terdapat pada form tersebut.

g. Kesimpulan

Mahasiswa berhasil memahami konsep shared preferences, serta membuat desain sederhana form register dan form login.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Pemahaman konsep dasar shared preferences dengan penjelasan detail, dan desain sederhana form register dan form login	1	2	3	4
2	Pemahaman konsep dasar shared preferences dengan penjelasan singkat dan desain sederhana form register dan form login	1	2	3	4
3	Pemahaman konsep dasar shared preferences dengan penjelasan yang kurang, dan desain sederhana form register dan form login	1	2	3	4
4	Tidak paham konsep dasar shared preferences dengan penjelasan singkat, dan desain sederhana form register dan form login	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 30 : SHARED PREFERENCES

Pokok Bahasan	:	Konsep Shared Preferences
Acara Praktikum/Praktik	:	Minggu 9 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep shared preferences.
2. Mahasiswa mampu membuat code pada file preferences.

b. Indikator Penilaian

Keberhasilan dalam memahami konsep shared preferences, serta implementasi code pada file preferences.

c. Dasar Teori

Shared Preferences

Shared Preferences adalah penyimpanan *key–value* untuk data sederhana (`String`, `int`, `double`, `bool`, `List<String>`). Pada Flutter, objek preferences diperoleh secara asynchronous dengan `SharedPreferences.getInstance()`, sehingga setiap getter dan setter dibuat sebagai fungsi `async` yang mengembalikan `Future`. Berbeda dengan Java, method pada Flutter tidak memerlukan parameter `Context`.

d. Alat dan Bahan

1. Kertas A4
2. Kabel Data USB
3. Jaringan Internet

e. Prosedur Kerja

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `preference_app`.
3. Pastikan target platform Android tercentang.
4. Tambahkan package shared_preferences:

```bash
flutter pub add shared_preferences
```

5. Dibutuhkan file `.dart` baru sebagai tempat Shared Preferences yang akan digunakan nantinya.
6. Klik kanan folder `lib` → **New** → **Dart File**, ketikkan `preferences` lalu tekan **Enter** sehingga terbentuk file `lib/preferences.dart`.
7. Berikut ini adalah source code dari kelas `Preferences`:

DART

```dart
import 'package:shared_preferences/shared_preferences.dart';

class Preferences {
  // Deklarasi key yang digunakan saat menambah atau mengedit data
  static const String keyUserTeregister = 'user';
  static const String keyPassTeregister = 'pass';
  static const String keyUsernameSedangLogin = 'Username_logged_in';
  static const String keyStatusSedangLogin = 'Status_logged_in';

  // "Mutator" SharedPreferences: setiap akses wajib melalui fungsi ini
  static Future<SharedPreferences> _getSharedPreference() {
    return SharedPreferences.getInstance();
  }

  // ---------- Setter & Getter user terdaftar ----------
  static Future<void> setRegisteredUser(String username) async {
    final sp = await _getSharedPreference();
    await sp.setString(keyUserTeregister, username);
  }

  static Future<String> getRegisteredUser() async {
    final sp = await _getSharedPreference();
    return sp.getString(keyUserTeregister) ?? '';
  }

  // ---------- Setter & Getter password terdaftar ----------
  static Future<void> setRegisteredPass(String password) async {
    final sp = await _getSharedPreference();
    await sp.setString(keyPassTeregister, password);
  }

  static Future<String> getRegisteredPass() async {
    final sp = await _getSharedPreference();
    return sp.getString(keyPassTeregister) ?? '';
  }

  // ---------- Setter & Getter user yang sedang login ----------
  static Future<void> setLoggedInUser(String username) async {
    final sp = await _getSharedPreference();
    await sp.setString(keyUsernameSedangLogin, username);
  }

  static Future<String> getLoggedInUser() async {
    final sp = await _getSharedPreference();
    return sp.getString(keyUsernameSedangLogin) ?? '';
  }

  // ---------- Setter & Getter status login ----------
  static Future<void> setLoggedInStatus(bool status) async {
    final sp = await _getSharedPreference();
    await sp.setBool(keyStatusSedangLogin, status);
  }

  static Future<bool> getLoggedInStatus() async {
    final sp = await _getSharedPreference();
    return sp.getBool(keyStatusSedangLogin) ?? false;
  }

  // ---------- Menghapus data login (logout) ----------
  static Future<void> clearLoggedInUser() async {
    final sp = await _getSharedPreference();
    await sp.remove(keyStatusSedangLogin);
    await sp.remove(keyUsernameSedangLogin);
  }
}
```

8. Dalam kelas `Preferences` ditambahkan fungsi `_getSharedPreference()` yang berguna untuk mendapatkan objek `SharedPreferences` di halaman yang berbeda-beda; setiap kali menggunakan SharedPreferences harus melalui fungsi ini.
9. Kemudian deklarasikan key yang akan digunakan saat ingin menambah atau mengedit data (`keyUserTeregister`, `keyPassTeregister`, dan seterusnya).
10. Tambahkan getter dan setter untuk semua key yang sudah dideklarasikan, dan buat semua getter dan setter bersifat `static` dan `async`. Khusus setter, tambahkan parameter `String` atau `bool` (untuk setter status).

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code pada file preferences.

g. Kesimpulan

Mahasiswa berhasil memahami konsep shared preferences, serta implementasi code pada file preferences.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Pemahaman konsep dasar implementasi code pada file preferences dengan penjelasan detail	1	2	3	4
2	Pemahaman konsep dasar implementasi code pada file preferences dengan penjelasan singkat	1	2	3	4
3	Pemahaman konsep dasar implementasi code pada file preferences dengan penjelasan yang kurang	1	2	3	4
4	Tidak paham konsep dasar implementasi code pada file preferences dengan penjelasan singkat	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 31 : SHARED PREFERENCES

Pokok Bahasan	:	Shared Preferences pada Register dan Login
Acara Praktikum/Praktik	:	Minggu 9 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami shared preferences pada register dan login.
2. Mahasiswa mampu mengimplementasikan code shared preferences untuk register dan login pada Flutter.

b. Indikator Penilaian

Keberhasilan dalam memahami shared preferences pada register dan login, serta implementasi code shared preferences untuk register dan login pada Flutter.

c. Dasar Teori

Shared Preferences

Shared Preferences adalah penyimpanan *key–value* untuk data sederhana yang tetap tersimpan walaupun aplikasi ditutup. Pada aplikasi login, status login disimpan sebagai `bool`; ketika aplikasi dibuka, status tersebut dibaca terlebih dahulu untuk menentukan halaman awal (halaman login atau halaman utama).

d. Alat dan Bahan

- Kertas A4
- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

1. **Buat halaman Login dan Register pada project `preference_app`.** Klik kanan folder `lib` → **New** → **Dart File**, lalu buat file `login_page.dart` dan `register_page.dart` (setara LoginActivity dan RegisterActivity), serta `main_page.dart` untuk halaman utama.
2. **Tambahkan source code berikut ke dalam `lib/main.dart`.** Pada bagian ini status login dibaca sebelum aplikasi tampil, sehingga halaman awal ditentukan otomatis (setara pengaturan *launcher activity* di AndroidManifest).

DART
```
import 'package:flutter/material.dart';
import 'preferences.dart';
import 'login_page.dart';
import 'main_page.dart';


Future<void> main() async {
  // Wajib dipanggil sebelum memakai plugin sebelum runApp()
```

```
  WidgetsFlutterBinding.ensureInitialized();
  final sudahLogin = await Preferences.getLoggedInStatus();
  runApp(MyApp(sudahLogin: sudahLogin));
}

class MyApp extends StatelessWidget {
  final bool sudahLogin;
  const MyApp({super.key, required this.sudahLogin});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Preferences',
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: sudahLogin ? const MainPage() : const LoginPage(),
    );
  }
}
```

3. **Tambahkan source code berikut ke dalam `lib/login_page.dart`.**

DART

```
import 'package:flutter/material.dart';
import 'preferences.dart';
import 'register_page.dart';
import 'main_page.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _user = TextEditingController();
  final _pass = TextEditingController();
  String? _errorUser;
  String? _errorPass;

  @override
  void dispose() {
    _user.dispose();
    _pass.dispose();
    super.dispose();
  }

  // Validasi username & password terhadap data di Shared Preferences
  Future<void> _razia() async {
    final user = _user.text.trim();
    final pass = _pass.text;
    final registeredUser = await Preferences.getRegisteredUser();
    final registeredPass = await Preferences.getRegisteredPass();

    String? eUser;
    String? ePass;
    if (user.isEmpty) {
      eUser = 'This field is required';
    } else if (user != registeredUser) {
      eUser = 'This Username is not found';
    }
    if (pass.isEmpty) {
      ePass = 'This field is required';
    } else if (pass != registeredPass) {
      ePass = 'This password is incorrect';
    }
    setState(() {
      _errorUser = eUser;
```

```
      _errorPass = ePass;
    });
    if (eUser == null && ePass == null) {
      await _masuk(user);
    }
  }

  // Menyimpan status login lalu pindah ke halaman utama
  Future<void> _masuk(String user) async {
    await Preferences.setLoggedInUser(user);
    await Preferences.setLoggedInStatus(true);
    if (!mounted) return;
    Navigator.pushReplacement(
        context, MaterialPageRoute(builder: (_) => const MainPage()));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            const SizedBox(height: 48),
            const Icon(Icons.lock_person, size: 80),
            const SizedBox(height: 16),
            const Text('LOGIN', textAlign: TextAlign.center,
                style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
            const SizedBox(height: 24),
            TextField(
              controller: _user,
              decoration: InputDecoration(
                labelText: 'Username',
                errorText: _errorUser,
                border: const OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            TextField(
              controller: _pass,
              obscureText: true,
              decoration: InputDecoration(
                labelText: 'Password',
                errorText: _errorPass,
                border: const OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _razia, child: const Text('LOGIN')),
            TextButton(
              onPressed: () => Navigator.push(context,
                  MaterialPageRoute(builder: (_) => const RegisterPage())),
              child: const Text('Belum punya akun? Register'),
            ),
          ],
        ),
      ),
    );
  }
}
```

4. **Tambahkan source code berikut ke dalam `lib/register_page.dart`.**

DART

```
import 'package:flutter/material.dart';
import 'preferences.dart';
import 'main_page.dart';

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});
```

```
  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final _formKey = GlobalKey<FormState>();
  final _user = TextEditingController();
  final _pass = TextEditingController();
  final _repass = TextEditingController();
  String _registeredUser = '';

  @override
  void initState() {
    super.initState();
    Preferences.getRegisteredUser().then((u) => _registeredUser = u);
  }

  @override
  void dispose() {
    _user.dispose();
    _pass.dispose();
    _repass.dispose();
    super.dispose();
  }

  Future<void> _daftar() async {
    if (!_formKey.currentState!.validate()) return;
    // Simpan data register ke Shared Preferences
    await Preferences.setRegisteredUser(_user.text.trim());
    await Preferences.setRegisteredPass(_pass.text);
    // Langsung login setelah register
    await Preferences.setLoggedInUser(_user.text.trim());
    await Preferences.setLoggedInStatus(true);
    if (!mounted) return;
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const MainPage()),
      (route) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Register')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            TextFormField(
              controller: _user,
              decoration: const InputDecoration(
                  labelText: 'Username', border: OutlineInputBorder()),
              validator: (v) {
                if (v == null || v.trim().isEmpty) return 'This field is required';
                if (v.trim() == _registeredUser) {
                  return 'This username already exists';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _pass,
              obscureText: true,
              decoration: const InputDecoration(
                  labelText: 'Password', border: OutlineInputBorder()),
              validator: (v) =>
```

```
                (v == null || v.isEmpty) ? 'This field is required' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _repass,
              obscureText: true,
              decoration: const InputDecoration(
                  labelText: 'Re-type Password', border: OutlineInputBorder()),
              validator: (v) =>
                  v != _pass.text ? 'This password is incorrect' : null,
            ),
            const SizedBox(height: 24),
            FilledButton(onPressed: _daftar, child: const Text('REGISTER')),
          ],
        ),
      ),
    );
  }
}
```

5. **Tambahkan source code berikut ke dalam `lib/main_page.dart`.**

DART

```
import 'package:flutter/material.dart';
import 'preferences.dart';
import 'login_page.dart';

class MainPage extends StatelessWidget {
  const MainPage({super.key});

  Future<void> _logout(BuildContext context) async {
    await Preferences.clearLoggedInUser();
    if (!context.mounted) return;
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const LoginPage()),
      (route) => false,
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Halaman Utama')),
      body: Center(
        child: FutureBuilder<String>(
          future: Preferences.getLoggedInUser(),
          builder: (context, snapshot) => Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const Icon(Icons.verified_user, size: 72),
              const SizedBox(height: 12),
              Text('Selamat datang, ${snapshot.data ?? ''}',
                  style: const TextStyle(fontSize: 20)),
              const SizedBox(height: 24),
              FilledButton.icon(
                onPressed: () => _logout(context),
                icon: const Icon(Icons.logout),
                label: const Text('LOGOUT'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

6. **Perhatikan code pada tombol Login.** Tanpa kelas `Preferences`, kode menyimpan status login harus ditulis lengkap setiap kali:

```dart
final sp = await SharedPreferences.getInstance();
await sp.setString('Username_logged_in', user);
await sp.setBool('Status_logged_in', true);
```

Akan tetapi apabila sudah membuat kelas `Preferences`, cukup menambahkan code berikut:

```dart
await Preferences.setLoggedInUser(user);
await Preferences.setLoggedInStatus(true);
```

7. **AndroidManifest.xml.** Pada Flutter tidak perlu mendeklarasikan activity baru. Halaman pertama ditentukan oleh properti `home` pada `MaterialApp` berdasarkan status login (langkah 2), sehingga `LoginPage` menjadi halaman utama ketika pengguna belum login.
8. Build dan run project tersebut, lalu perhatikan hasilnya. Lakukan register, tutup aplikasi sepenuhnya (*swipe* dari recent apps), buka kembali dan amati bahwa aplikasi langsung membuka halaman utama tanpa login ulang. Kemudian tekan **LOGOUT** dan amati kembali.

f. Hasil dan Pembahasan

Project aplikasi dari percobaan implementasi code shared preferences pada register dan login.

g. Kesimpulan

Mahasiswa berhasil memahami konsep shared preferences pada register dan login, serta implementasi code shared preferences pada register dan login.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Pemahaman konsep dasar implementasi code shared preferences pada register dan login dengan penjelasan detail	1	2	3	4
2	Pemahaman konsep dasar implementasi code shared preferences pada register dan login dengan penjelasan singkat	1	2	3	4
3	Pemahaman konsep dasar implementasi code shared preferences pada register dan login dengan penjelasan yang kurang	1	2	3	4
4	Tidak paham konsep dasar implementasi code shared preferences register dan login dengan penjelasan singkat	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 32 : SHARED PREFERENCES

Pokok Bahasan	:	Analisa Shared Preferences pada Register dan Login
Acara Praktikum/Praktik	:	Minggu 9 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami shared preferences pada register dan login.
2. Mahasiswa mampu mengimplementasikan code shared preferences untuk register dan login pada Flutter.

b. Indikator Penilaian

Keberhasilan dalam memahami shared preferences pada register dan login, serta implementasi code shared preferences untuk register dan login pada Flutter.

c. Dasar Teori

Shared Preferences

Shared Preferences adalah penyimpanan *key–value* yang hanya dapat menyimpan data bertipe sederhana (`String`, `int`, `double`, `bool`, `List<String>`) dan cocok untuk data yang sedikit. Data disimpan secara otomatis oleh platform (file XML pada Android) dan tetap ada sampai aplikasi di-uninstall atau datanya dihapus.

d. Alat dan Bahan

- Kertas A4
- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

1. Mahasiswa melakukan analisa code untuk setiap baris code dari implementasi shared preferences register dan login.
2. Menjelaskan fungsi dan arti dari setiap baris code shared preferences register dan login.
3. Analisa hasil running dari implementasi project shared preferences register dan login.
4. Screenshot output setiap proses dari project shared preferences register dan login.
5. Tuliskan dalam bentuk laporan.

f. Hasil dan Pembahasan

Dokumentasi tugas berupa laporan secara individu penjelasan setiap code pada project shared preferences register dan login yang dibuktikan dengan screenshot pada proses percobaan.

g. Kesimpulan

Mahasiswa memahami konsep dasar shared preferences register dan login dengan mampu memahami setiap baris code dari implementasi shared preferences register dan login.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menjelaskan secara detail setiap baris code pada implementasi shared preferences register dan login	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menjelaskan setiap baris code pada implementasi shared preferences register dan login	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menjelaskan code shared preferences register dan login	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam menjelaskan code shared preferences register dan login	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 33 : GPS DAN AKSES SENSOR

Pokok Bahasan	:	Menampilkan Maps
Acara Praktikum/Praktik	:	Minggu 10 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep location based service.
2. Mahasiswa mampu menerapkan location based service pada Flutter.
3. Mahasiswa mampu menampilkan maps.
4. Mahasiswa mampu mendapatkan maps API key, menampilkan zoom control, dan menampilkan navigasi ke lokasi khusus.

b. Indikator Penilaian

1. Keberhasilan mahasiswa menampilkan maps.
2. Keberhasilan mahasiswa mendapatkan maps API key.
3. Keberhasilan mahasiswa menampilkan zoom control.
4. Keberhasilan mahasiswa menampilkan navigasi ke lokasi khusus.

c. Dasar Teori

Location Based Service

Location Based Service (LBS) atau Layanan Berbasis Lokasi adalah suatu sistem yang menyediakan layanan yang mempunyai lokasi geografis sehingga dapat dilihat pada peta. Salah satu sumber data geografis yang mudah didapatkan adalah koordinat lokasi yang diambil dari smartphone.

Smartphone mempunyai dua macam jenis lokasi, **coarse-location** dan **fine-location**. Lokasi *coarse*/kasar adalah lokasi pengguna yang ditentukan lewat BTS/jaringan terdekat; presisinya tidak akurat, namun hemat baterai. Sedangkan *fine-location*/lokasi halus adalah lokasi pengguna yang didapatkan melalui GPS; presisinya lebih tinggi, namun boros baterai.

Pada Flutter, peta Google ditampilkan dengan package resmi **google_maps_flutter** (widget `GoogleMap`), dan lokasi perangkat diperoleh dengan package **geolocator** (Acara 34).

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

Location Based Service (LBS): implementasi LBS pada smartphone.

1. Buka Android Studio.
2. Buat project Flutter baru dengan nama `lbs` dan organization `com.example` (sehingga *package name* menjadi `com.example.lbs`).
3. Tambahkan package Google Maps:

```
BASH
flutter pub add google_maps_flutter
```

4. Buka `android/app/build.gradle` (atau `build.gradle.kts`) dan pastikan `minSdk` minimal 21 (nilai `flutter.minSdkVersion` pada versi Flutter terbaru sudah memenuhi).

Menampilkan Maps

Google Maps adalah salah satu aplikasi yang dibundel dengan platform Android. Selain menggunakan aplikasi Maps, kita juga dapat menanamkan peta ke dalam aplikasi Flutter. Bagian ini menjelaskan cara menggunakan Google Maps di aplikasi Flutter.

A. Mendapatkan Maps API Key

Terdapat 3 informasi yang dibutuhkan untuk membuat API key Android:

1. *Package name*, yaitu nama package aplikasi (lihat `applicationId` pada `android/app/build.gradle`, misalnya `com.example.lbs`).
2. *SHA-1 certificate fingerprint*, yaitu kode *fingerprint* sertifikat debug. Dapatkan dengan perintah berikut dari folder project:

```
BASH
cd android
./gradlew signingReport
```

Pada Windows gunakan `gradlew signingReport`. Salin nilai **SHA1** pada bagian *Variant: debug*.

3. API key itu sendiri, yang dibuat melalui Google Cloud Console dengan langkah-langkah berikut:

- Buka URL https://console.cloud.google.com/
- Login menggunakan akun Google pribadi.
- Buat proyek dengan nama sesuai keinginan, lalu pilih proyek yang baru dibuat.
- Buka *Navigation Menu* → *APIs & Services* → *Library*.
- Enable library berikut: (a) Maps SDK for Android; (b) Directions API; (c) Geocoding API; (d) Geolocation API; (e) Places API; (f) Maps JavaScript API (bila akan dijalankan di web).
- Buka *Navigation Menu* → *APIs & Services* → *Credentials*.
- Klik **Create Credentials** → **API key** untuk membuat API key.
- Untuk meningkatkan keamanan tambahkan *restriction*: pilih *Android apps*, masukkan *package name* dan SHA-1.

5. Salin API key dan isikan pada file `android/app/src/main/AndroidManifest.xml` di dalam tag `<application>`:

```xml
<application ...>
    <meta-data
        android:name="com.google.android.geo.API_KEY"
        android:value="ISI_DENGAN_API_KEY_ANDA" />
    ...
</application>
```

6. Tambahkan izin lokasi di atas tag `<application>` (diperlukan pada Acara 34 untuk menampilkan lokasi pengguna):

```xml
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

7. Ubah isi `lib/main.dart` untuk menampilkan peta:

```dart
import 'package:flutter/material.dart';
import 'maps_page.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MapsPage(),
    ));
```

Buat file `lib/maps_page.dart`:

```dart
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

class MapsPage extends StatefulWidget {
  const MapsPage({super.key});

  @override
  State<MapsPage> createState() => _MapsPageState();
}

class _MapsPageState extends State<MapsPage> {
  GoogleMapController? mMap;

  // Posisi awal kamera (default template Android: Sydney)
  static const CameraPosition _awal = CameraPosition(
    target: LatLng(-34, 151),
    zoom: 3,
  );

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Maps')),
      body: GoogleMap(
        initialCameraPosition: _awal,
        onMapCreated: (controller) => mMap = controller,
      ),
    );
  }
}
```

8. Hentikan aplikasi lalu jalankan ulang (*full restart*, bukan hot reload). Jika berhasil, peta akan tampil seperti gambar berikut.



Gambar 1. Menampilkan peta pada aplikasi

9. Jika terdapat kesalahan (peta abu-abu/kosong), cek tab **Run/Logcat** untuk melihat *warning* atau *error*, biasanya karena API key salah, SHA-1 belum didaftarkan, atau *Maps SDK for Android* belum di-enable.
10. Untuk menguji aplikasi pada emulator, pastikan menggunakan emulator dengan *system image* yang menyertakan **Google Play Services** (Google APIs/Google Play).

B. Menampilkan Zoom Control

Pada perangkat nyata peta dapat diperbesar dengan mencubit (*pinch*), tetapi pada emulator hal ini sulit dilakukan. Bagian ini memungkinkan pengguna memperbesar atau memperkecil peta menggunakan kontrol zoom bawaan.

1. Buka kembali `lib/maps_page.dart`.
2. Tambahkan properti berikut pada widget `GoogleMap` (setara atribut `map:uiZoomControls="true"`):

```dart
GoogleMap(
  initialCameraPosition: _awal,
  zoomControlsEnabled: true,   // tombol + dan -
  zoomGesturesEnabled: true,   // pinch to zoom
  onMapCreated: (controller) => mMap = controller,
)
```

3. Jalankan aplikasi pada emulator.
4. Amati kontrol zoom bawaan yang muncul di bagian kanan bawah peta. Klik ikon minus (−) untuk memperkecil peta, dan ikon plus (+) untuk memperbesar peta.



Gambar 2. Menampilkan zoom control

C. Navigasi ke Lokasi Khusus

Peta dapat diatur untuk menampilkan lokasi tertentu secara programatik dengan method `moveCamera()` atau `animateCamera()` milik `GoogleMapController`.

1. Ubah `lib/maps_page.dart` sehingga peta berpindah ke Boston dan diberi marker:

```dart
class _MapsPageState extends State<MapsPage> {
  GoogleMapController? mMap;
  static const LatLng boston = LatLng(42.3601, -71.0589);

  static const CameraPosition _awal = CameraPosition(
```

```
    target: LatLng(-34, 151),
    zoom: 3,
  );

  final Set<Marker> _markers = {
    const Marker(
      markerId: MarkerId('boston'),
      position: boston,
      infoWindow: InfoWindow(title: 'Boston, Mass'),
    ),
  };

  // Setara onMapReady(GoogleMap googleMap)
  void _onMapCreated(GoogleMapController controller) {
    mMap = controller;
    controller.moveCamera(CameraUpdate.newLatLngZoom(boston, 12));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Maps')),
      body: GoogleMap(
        initialCameraPosition: _awal,
        zoomControlsEnabled: true,
        markers: _markers,
        onMapCreated: _onMapCreated,
      ),
    );
  }
}
```

2. Jalankan aplikasi. Ketika peta dimuat, amati bahwa sekarang kamera berpindah ke Boston, Massachusetts. Pada kode di atas, objek `LatLng` dibuat dengan koordinat 42,3601 LU dan 71,0589 BB. Perhatikan bahwa untuk mewakili W (barat) atau S (selatan) digunakan nilai koordinat negatif.

[Tempelkan screenshot hasil running aplikasi]

Gambar 3. Menampilkan lokasi Boston

3. (Latihan) Tambahkan marker kedua untuk Politeknik Negeri Jember (`LatLng(-8.1595, 113.7230)`) dan tombol FloatingActionButton yang memanggil `mMap?.animateCamera(CameraUpdate.newLatLngZoom(polije, 16))`.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code untuk menampilkan maps dengan mendapatkan maps API key, menampilkan zoom control, dan menampilkan navigasi ke lokasi khusus.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa mampu menampilkan maps dengan mendapatkan maps API key, menampilkan zoom control, dan menampilkan navigasi ke lokasi khusus.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code untuk menampilkan maps	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code untuk menampilkan maps	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code untuk menampilkan maps	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code untuk menampilkan maps	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 34 : GPS DAN AKSES SENSOR

Pokok Bahasan	:	Mendapatkan dan Monitoring Lokasi
Acara Praktikum/Praktik	:	Minggu 10 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep location based service.
2. Mahasiswa mampu menerapkan location based service pada Flutter.
3. Mahasiswa mampu mendapatkan lokasi fisik pengguna.
4. Mahasiswa mampu memonitoring lokasi tertentu.

b. Indikator Penilaian

1. Keberhasilan mahasiswa mendapatkan lokasi dengan menentukan lokasi fisik pengguna.
2. Keberhasilan mahasiswa mampu memonitoring lokasi untuk memantau lokasi tertentu.

c. Dasar Teori

Location Based Service

Saat ini perangkat seluler umumnya dilengkapi dengan penerima GPS. Karena banyaknya satelit yang mengorbit bumi, kita dapat menggunakan penerima GPS untuk menemukan lokasi dengan mudah. Namun, GPS membutuhkan langit yang cerah untuk bekerja dan karenanya tidak selalu berfungsi di dalam ruangan atau tempat yang tidak dapat ditembus sinyal satelit (seperti terowongan).

Cara efektif lain untuk menemukan posisi adalah melalui **triangulasi menara seluler**. Ketika ponsel dihidupkan, ponsel terus-menerus berhubungan dengan BTS di sekitarnya. Dengan mengetahui identitas menara sel, lokasi fisik dapat diterjemahkan melalui basis data yang berisi identitas menara sel dan lokasi geografisnya. Keuntungannya adalah dapat bekerja di dalam ruangan tanpa informasi satelit, tetapi tidak setepat GPS karena akurasinya bergantung pada cakupan sinyal yang tumpang tindih. Triangulasi menara seluler bekerja paling baik di daerah padat penduduk.

Metode ketiga adalah **triangulasi Wi-Fi**. Perangkat terhubung ke jaringan Wi-Fi dan memeriksa penyedia layanan terhadap basis data untuk menentukan lokasi yang dilayani oleh penyedia.

Pada Android native, lokasi diperoleh dengan kelas `LocationManager`. Pada Flutter digunakan package **geolocator** yang menyediakan:

LocationManager (Android)	geolocator (Flutter)
checkSelfPermission / requestPermissions	Geolocator.checkPermission() / requestPermission()

LocationManager (Android)	geolocator (Flutter)
isProviderEnabled()	Geolocator.isLocationServiceEnabled()
getLastKnownLocation()	Geolocator.getLastKnownPosition() / getCurrentPosition()
requestLocationUpdates(provider, minTime, minDistance, listener)	Geolocator.getPositionStream(locationSettings: ...).listen(...)
onLocationChanged(Location)	callback pada listen((Position p) { ... })
onProviderEnabled / onProviderDisabled	Geolocator.getServiceStatusStream()
removeUpdates(listener)	StreamSubscription.cancel()
Location.distanceTo() / addProximityAlert()	Geolocator.distanceBetween() (dipantau manual pada stream)

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

Mendapatkan Lokasi

1. Menggunakan project `lbs` yang dibuat sebelumnya, tambahkan package geolocator dan url_launcher:

```bash
flutter pub add geolocator url_launcher
```

2. Pastikan izin lokasi sudah ditambahkan pada `android/app/src/main/AndroidManifest.xml` (Acara 33 langkah 6). Tambahkan juga tag `<queries>` untuk membuka browser seperti pada Acara 22.
3. Ubah `lib/maps_page.dart` menjadi seperti berikut:

```dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:url_launcher/url_launcher.dart';

class MapsPage extends StatefulWidget {
  const MapsPage({super.key});

  @override
  State<MapsPage> createState() => _MapsPageState();
}

class _MapsPageState extends State<MapsPage> {
  GoogleMapController? mMap;
  StreamSubscription<Position>? _locationListener;
  StreamSubscription<ServiceStatus>? _providerListener;
  late final AppLifecycleListener _lifecycle;
  bool permissionGranted = false;

  // Lokasi yang dimonitor (proximity alert)
  static const LatLng _target = LatLng(-8.1595, 113.7230);
```

```
static const double _radius = 5; // meter
bool _sudahDiTarget = false;

static const CameraPosition _awal =
    CameraPosition(target: LatLng(-8.17, 113.70), zoom: 12);

@override
void initState() {
  super.initState();
  // Setara onPause()/onResume() Activity
  _lifecycle = AppLifecycleListener(
    onPause: _hentikanUpdate,
    onResume: _mulaiUpdate,
  );
  _mulaiUpdate();
}

// ----- Izin lokasi (setara checkSelfPermission & requestPermissions) -----
Future<bool> _cekIzin() async {
  if (!await Geolocator.isLocationServiceEnabled()) {
    _toast('GPS tidak aktif');
    return false;
  }
  var izin = await Geolocator.checkPermission();
  if (izin == LocationPermission.denied) {
    izin = await Geolocator.requestPermission();
  }
  if (izin == LocationPermission.denied ||
      izin == LocationPermission.deniedForever) {
    _toast('Izin lokasi ditolak');
    return false;
  }
  return true;
}

// ----- Setara requestLocationUpdates(GPS_PROVIDER, 0, 0, listener) -----
Future<void> _mulaiUpdate() async {
  permissionGranted = await _cekIzin();
  if (!permissionGranted || _locationListener != null) return;

  const settings = LocationSettings(
    accuracy: LocationAccuracy.high, // GPS (fine location)
    distanceFilter: 0,               // minDistance = 0 meter
  );
  _locationListener = Geolocator.getPositionStream(locationSettings: settings)
      .listen(_onLocationChanged);

  // Setara onProviderEnabled() / onProviderDisabled()
  _providerListener ??= Geolocator.getServiceStatusStream().listen((s) {
    _toast(s == ServiceStatus.enabled
        ? 'Provider lokasi diaktifkan'
        : 'Provider lokasi dinonaktifkan');
  });
}

// ----- Setara removeUpdates(locationListener) -----
void _hentikanUpdate() {
  _locationListener?.cancel();
  _locationListener = null;
}

// ----- Setara onLocationChanged(Location loc) -----
void _onLocationChanged(Position loc) {
  _toast('Location changed: Lat: ${loc.latitude} Lng: ${loc.longitude}');
  final p = LatLng(loc.latitude, loc.longitude);
  mMap?.animateCamera(CameraUpdate.newLatLngZoom(p, 16));
  _cekProximity(loc);
}
```

```
  // ----- Monitoring lokasi (setara addProximityAlert) -----
  Future<void> _cekProximity(Position loc) async {
    final jarak = Geolocator.distanceBetween(
        loc.latitude, loc.longitude, _target.latitude, _target.longitude);
    if (jarak <= _radius && !_sudahDiTarget) {
      _sudahDiTarget = true;
      _toast('Anda berada di lokasi target!');
      await launchUrl(Uri.parse('https://www.polije.ac.id'),
          mode: LaunchMode.externalApplication);
    } else if (jarak > _radius) {
      _sudahDiTarget = false;
    }
  }

  void _toast(String pesan) {
    if (!mounted) return;
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(SnackBar(content: Text(pesan)));
  }

  @override
  void dispose() {
    _hentikanUpdate();
    _providerListener?.cancel();
    _lifecycle.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('LBS - Lokasi Saya')),
      body: GoogleMap(
        initialCameraPosition: _awal,
        zoomControlsEnabled: true,
        myLocationEnabled: true,        // titik biru lokasi pengguna
        myLocationButtonEnabled: true,
        circles: {
          Circle(
            circleId: const CircleId('target'),
            center: _target,
            radius: _radius,
            fillColor: Colors.red.withValues(alpha: 0.3),
            strokeWidth: 1,
          ),
        },
        markers: {
          const Marker(
            markerId: MarkerId('target'),
            position: _target,
            infoWindow: InfoWindow(title: 'Lokasi yang dimonitor'),
          ),
        },
        onMapCreated: (c) => mMap = c,
      ),
    );
  }
}
```

> `Colors.red.withValues(alpha: 0.3)` digunakan pada Flutter 3.27 ke atas. Pada versi lebih lama gunakan `Colors.red.withOpacity(0.3)`.

4. Jalankan aplikasi pada emulator. Izinkan akses lokasi ketika muncul dialog permission.
5. Untuk mensimulasikan data GPS yang diterima oleh emulator, gunakan menu **Extended Controls (⋯) → Location** di sisi kanan emulator, masukkan koordinat lalu klik **Set Location**.

6. Perhatikan bahwa peta pada emulator sekarang berpindah ke lokasi lain dan muncul pesan lintang–bujur, ini membuktikan bahwa aplikasi telah menerima data GPS.



Gambar 4. Mendapatkan lokasi tertentu

Penjelasan kode:

- Fungsi `_cekIzin()` memeriksa apakah layanan lokasi aktif dan meminta izin lokasi kepada pengguna (setara `ActivityCompat.checkSelfPermission` dan `requestPermissions`).
- `Geolocator.getPositionStream()` mendaftarkan permintaan pembaruan lokasi secara berkala (setara `requestLocationUpdates()`). Parameternya diatur melalui `LocationSettings`:
- `accuracy` : tingkat akurasi/penyedia lokasi. `LocationAccuracy.high` menggunakan GPS; `LocationAccuracy.low` menggunakan jaringan (BTS/Wi-Fi).
- `distanceFilter` : jarak minimum (meter) sebelum notifikasi berikutnya. 0 berarti ingin terus diberi tahu setiap perubahan lokasi.
- `timeLimit` (opsional) : batas waktu menunggu pembaruan lokasi.
- Fungsi `_onLocationChanged()` dipanggil setiap kali lokasi berubah; menampilkan lintang dan bujur dengan SnackBar, lalu memindahkan kamera peta ke lokasi tersebut.
- `getServiceStatusStream()` memantau apakah penyedia lokasi diaktifkan atau dinonaktifkan oleh pengguna (setara `onProviderEnabled()`/`onProviderDisabled()`).
- Pada `AppLifecycleListener.onPause` dan `dispose()`, pembaruan lokasi dihentikan dengan `cancel()` sehingga aplikasi tidak lagi mendengarkan perubahan lokasi dan menghemat baterai (setara `removeUpdates()`).

Monitoring Lokasi

Salah satu fitur `LocationManager` di Android adalah kemampuan memantau lokasi tertentu dengan `addProximityAlert()`. Pada Flutter, fitur ini diwujudkan dengan menghitung jarak antara posisi pengguna dan lokasi target pada setiap pembaruan lokasi menggunakan `Geolocator.distanceBetween()`.

1. Perhatikan fungsi `_cekProximity()` pada kode di atas. Jika pengguna berada dalam radius **lima meter** dari lokasi target, aplikasi akan meluncurkan browser web ke alamat tertentu.
2. Data yang dibutuhkan untuk monitoring lokasi:

- Lintang target
- Bujur target

- Radius (dalam meter)
- Aksi yang dijalankan saat masuk radius (pada contoh: membuka browser dengan url_launcher, setara *PendingIntent*)

3. Uji dengan memasukkan koordinat target (-8.1595, 113.7230) pada Extended Controls emulator.

> Pemantauan pada contoh di atas hanya berjalan ketika aplikasi terbuka. Untuk *geofencing* di latar belakang (aplikasi tertutup), diperlukan package khusus geofencing atau platform channel ke `GeofencingClient` Android.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code untuk mendapatkan lokasi fisik pengguna dan memonitoring lokasi tertentu.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa mampu mendapatkan lokasi fisik pengguna dan memonitoring lokasi untuk memantau lokasi tertentu.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code untuk mendapatkan lokasi	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code untuk mendapatkan lokasi	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code untuk mendapatkan lokasi	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code untuk mendapatkan lokasi	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 35 : GPS DAN AKSES SENSOR

Pokok Bahasan	:	Pengaturan Android Manifest, Asset dan Halaman
Acara Praktikum/Praktik	:	Minggu 10 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami sensor hardware pada smartphone.
2. Mahasiswa mampu mengakses dan menggunakan perangkat sensor yang tertanam di dalam smartphone Android melalui Flutter.
3. Mahasiswa mampu melakukan pengaturan Android Manifest, menambahkan asset dan halaman yang perlu dibuat.

b. Indikator Penilaian

1. Keberhasilan mahasiswa mengakses dan menggunakan perangkat sensor yang tertanam di dalam smartphone Android.
2. Keberhasilan mahasiswa melakukan pengaturan Android Manifest, menambahkan asset dan halaman yang perlu dibuat.

c. Dasar Teori

Akses Sensor

Akses sensor mengulas bagaimana cara mengakses sensor yang tertanam di dalam smartphone. Sensor merupakan alat yang terpasang pada perangkat mobile untuk mengukur, menganalisis, dan memantau kondisi lingkungan, serta dapat memberikan aksi dari perubahan kondisi tersebut.

Sensor dikelompokkan menjadi 3 jenis yaitu **Sensor Gerak** (*Motion Sensor*), **Sensor Lingkungan** (*Environment Sensor*), serta **Sensor Posisi** (*Position Sensor*).

- Sensor gerak memantau perubahan kondisi sepanjang 3 sumbu X, Y, dan Z (3 variabel). Contoh: accelerometer, gyroscope, dan rotation sensor.
- Sensor lingkungan mengukur parameter lingkungan sekitar ponsel, seperti suhu, tekanan udara, pencahayaan, dan kelembapan. Contoh: barometer (tekanan udara), photometer/light (pencahayaan), dan thermometer (suhu). Nilai datanya hanya pada array ke-1 (1 variabel).
- Sensor posisi mengukur posisi perangkat atau seberapa dekat ponsel dengan objek sekitar. Contoh: proximity (cm), orientation (derajat) dan magnetic field (μT/mikroTesla).

Sensor pada Flutter

Flutter mengakses sensor melalui dua cara:

1. **Package plugin** — misalnya `sensors_plus` (accelerometer, gyroscope, magnetometer, barometer).
2. **Platform Channel** — jembatan antara kode Dart dan kode native Kotlin/Java. `MethodChannel` dipakai untuk memanggil fungsi native sekali jalan (misalnya mengambil daftar sensor), sedangkan `EventChannel` dipakai untuk menerima aliran data terus-menerus (misalnya perubahan nilai sensor cahaya). Cara ini membuat kita tetap dapat memanfaatkan `SensorManager` Android untuk sensor yang belum disediakan oleh plugin.

Pada praktikum ini digunakan Platform Channel agar seluruh sensor Android (termasuk Light dan Proximity) dapat diakses.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

Bagian ini mempraktikkan implementasi sensor pada smartphone, terdiri dari pengaturan Android Manifest, menambahkan asset, dan halaman yang perlu dibuat.

Android Manifest

- Agar sensor dapat bekerja secara maksimal, layar perlu tetap menyala selama pemantauan. Pada Flutter digunakan package `wakelock_plus` (setara izin `WAKE_LOCK`), sehingga layar dan CPU tetap terjaga dan smartphone dapat maksimal memantau perubahan kondisi lingkungan.
- Pada Android native diperlukan *provider* agar sistem dapat membaca file mp3. Pada Flutter cukup mendaftarkan file mp3 sebagai **asset** pada `pubspec.yaml`.

1. Pada project `lbs`, tambahkan package yang dibutuhkan:

```bash
flutter pub add wakelock_plus audioplayers fl_chart
```

2. Tambahkan izin berikut pada `android/app/src/main/AndroidManifest.xml` (di atas tag `<application>`):

```xml
<uses-permission android:name="android.permission.WAKE_LOCK" />
```

Menambahkan Asset

- Asset yang digunakan yaitu file mp3 berisi suara dalam Bahasa Indonesia.
- File ini diperoleh dengan mengakses tautan https://soundoftext.com/ kemudian menuliskan teks **cahaya gelap** dengan pilihan suara *Indonesian*.

1. Unduh file tersebut.
2. Lanjutkan dengan mengunduh file suara **cahaya gelap**, **objek menjauh**, dan **objek mendekat**.

3. Simpan semua file tersebut dengan nama sesuai suara, misalnya `cahaya_gelap.mp3`, `objek_menjauh.mp3`, dan `objek_mendekat.mp3`.
4. Buat folder `assets/audio/` pada root project (sejajar dengan folder `lib/`).
5. Masukkan semua file mp3 ke dalam folder tersebut dengan cara *copy–paste*, lalu daftarkan pada `pubspec.yaml` (perhatikan indentasi 2 spasi):

```yaml
flutter:
  uses-material-design: true
  assets:
    - assets/audio/
```

6. Jalankan `flutter pub get`. Struktur folder menjadi seperti berikut:

```
lbs/
├── assets/
│   └── audio/
│       ├── cahaya_gelap.mp3
│       ├── objek_mendekat.mp3
│       └── objek_menjauh.mp3
├── lib/
└── pubspec.yaml
```

Halaman Utama Sensor (setara Main Activity)

Halaman utama digunakan untuk menampilkan 3 tombol yang mewakili aksi menampilkan semua sensor di dalam smartphone, Sensor Cahaya (*Light Sensor*), dan Sensor Kedekatan (*Proximity Sensor*).

1. Buat file `lib/sensor_menu_page.dart`. Tiga tombol berupa kartu bergambar ikon (setara ImageView yang diberi aksi OnClick):

```dart
import 'package:flutter/material.dart';
import 'all_sensor_page.dart';
import 'light_sensor_page.dart';
import 'proximity_sensor_page.dart';

class SensorMenuPage extends StatelessWidget {
  const SensorMenuPage({super.key});

  Widget _tombol(BuildContext context, IconData ikon, String label,
      Widget tujuan) {
    return Card(
      elevation: 3,
      child: InkWell(
        // Setara Intent ke activity tujuan
        onTap: () => Navigator.push(
            context, MaterialPageRoute(builder: (_) => tujuan)),
        child: Padding(
          padding: const EdgeInsets.all(20),
          child: Row(
            children: [
              Icon(ikon, size: 48),
              const SizedBox(width: 16),
              Text(label, style: const TextStyle(fontSize: 18)),
            ],
          ),
        ),
      ),
    );
  }
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Akses Sensor')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          _tombol(context, Icons.sensors, 'Semua Sensor',
              const AllSensorPage()),
          _tombol(context, Icons.light_mode, 'Sensor Cahaya',
              const LightSensorPage()),
          _tombol(context, Icons.social_distance, 'Sensor Proximity',
              const ProximitySensorPage()),
        ],
      ),
    );
  }
}
```

2. Buat tiga file halaman baru (isinya dilengkapi pada Acara 36):

- `lib/all_sensor_page.dart` → kelas `AllSensorPage` (setara AllSensorActivity)
- `lib/light_sensor_page.dart` → kelas `LightSensorPage` (setara LightSensorActivity)
- `lib/proximity_sensor_page.dart` → kelas `ProximitySensorPage` (setara ProximitySensorActivity)

Sebagai kerangka awal, isi masing-masing file dengan StatefulWidget kosong, contohnya:

DART
```
import 'package:flutter/material.dart';

class AllSensorPage extends StatefulWidget {
  const AllSensorPage({super.key});

  @override
  State<AllSensorPage> createState() => _AllSensorPageState();
}

class _AllSensorPageState extends State<AllSensorPage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Semua Sensor')),
    );
  }
}
```

3. Ubah `lib/main.dart` agar halaman awal aplikasi adalah menu sensor (halaman Maps tetap dapat dibuka dari menu bila diinginkan):

DART
```
import 'package:flutter/material.dart';
import 'sensor_menu_page.dart';

void main() => runApp(const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: SensorMenuPage(),
    ));
```

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code untuk mengakses dan menggunakan perangkat sensor yang tertanam di dalam smartphone.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa mampu mengakses dan menggunakan perangkat sensor yang tertanam di dalam smartphone dengan melakukan pengaturan Android Manifest, menambahkan asset dan halaman yang perlu dibuat.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code untuk pengaturan android manifest, menambahkan asset dan halaman yang dibuat	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code untuk pengaturan android manifest, menambahkan asset dan halaman yang dibuat	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code untuk pengaturan android manifest, menambahkan asset dan halaman yang dibuat	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code untuk pengaturan android manifest, menambahkan asset dan halaman yang dibuat	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 36 : GPS DAN AKSES SENSOR

Pokok Bahasan	:	Menangkap Perubahan Nilai dari Suatu Sensor
Acara Praktikum/Praktik	:	Minggu 10 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami sensor hardware pada smartphone.
2. Mahasiswa mampu mengakses dan menggunakan perangkat sensor yang tertanam di dalam smartphone Android melalui Flutter.
3. Mahasiswa mampu menampilkan semua sensor.
4. Mahasiswa mampu menangkap perubahan nilai dengan implementasi sensor light dan sensor proximity.

b. Indikator Penilaian

1. Keberhasilan mahasiswa menampilkan semua sensor.
2. Keberhasilan mahasiswa menangkap perubahan nilai dengan implementasi sensor light dan sensor proximity.

c. Dasar Teori

Menampilkan Semua Sensor

Halaman ini digunakan untuk menampilkan semua sensor yang tersedia di dalam smartphone. UI berupa ListView yang memiliki data berupa Nama Sensor, Tipe, Power (daya yang digunakan dalam mA), serta Nilai Range Maksimal yang dapat ditangkap oleh sensor. Data diambil dari `SensorManager.getSensorList()` Android melalui **MethodChannel**.

Menangkap Perubahan Nilai

Pada Android native, perubahan nilai ditangkap melalui parameter `SensorEvent` pada fungsi `onSensorChanged()` ketika mengimplementasikan `SensorEventListener`. Pada Flutter, listener tersebut ditulis di sisi Kotlin, lalu setiap nilai baru dikirim ke Dart melalui **EventChannel** sebagai `Stream`. Dart cukup melakukan `listen()` pada stream tersebut.

- **Light Sensor** — mendeteksi tingkat pencahayaan lingkungan sekitar (satuan lux). Sensor ini hanya menggunakan 1 variabel array (`values[0]`). Agar perbedaan kondisi pencahayaan mudah dilihat, nilai sensor ditampilkan dalam bentuk grafik (package `fl_chart`).
- **Proximity Sensor** — mendeteksi kedekatan dengan objek terdekat (satuan cm). Sensor ini digunakan, misalnya, untuk mematikan layar ketika menerima panggilan agar menghemat baterai.

Kedua sensor ini hanya menggunakan 1 variabel array. Namun, ketika menggunakan sensor gerak, sistem harus menggunakan 3 variabel array (X, Y, Z).

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

Buka kembali project `lbs` yang sudah dibuat pada acara sebelumnya, dan tambahkan beberapa kode.

Kode Native: Platform Channel (Kotlin)

1. Buka file `android/app/src/main/kotlin/com/example/lbs/MainActivity.kt` dan ganti isinya dengan kode berikut. Kode ini mendaftarkan satu MethodChannel untuk daftar sensor dan dua EventChannel untuk sensor cahaya dan proximity:

KOTLIN
```
package com.example.lbs

import android.content.Context
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.BinaryMessenger
import io.flutter.plugin.common.EventChannel
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        val sensorManager =
            getSystemService(Context.SENSOR_SERVICE) as SensorManager
        val messenger = flutterEngine.dartExecutor.binaryMessenger

        // MethodChannel: mengambil daftar semua sensor
        val channel = MethodChannel(messenger, "polije/sensor")
        channel.setMethodCallHandler { call, result ->
            if (call.method == "getSensorList") {
                val daftar = sensorManager.getSensorList(Sensor.TYPE_ALL).map {
                    mapOf(
                        "nama" to it.name,
                        "tipe" to it.stringType,
                        "power" to it.power.toDouble(),
                        "maxRange" to it.maximumRange.toDouble()
                    )
                }
                result.success(daftar)
            } else {
                result.notImplemented()
            }
        }

        // EventChannel: aliran nilai sensor cahaya dan proximity
        daftarkanSensor(messenger, sensorManager, "polije/light",
            Sensor.TYPE_LIGHT)
        daftarkanSensor(messenger, sensorManager, "polije/proximity",
            Sensor.TYPE_PROXIMITY)
    }

    private fun daftarkanSensor(
        messenger: BinaryMessenger,
```

```
        sensorManager: SensorManager,
        nama: String,
        tipe: Int
    ) {
        val channel = EventChannel(messenger, nama)
        channel.setStreamHandler(object : EventChannel.StreamHandler {
            private var listener: SensorEventListener? = null

            override fun onListen(arguments: Any?, events: EventChannel.EventSink) {
                val sensor = sensorManager.getDefaultSensor(tipe)
                if (sensor == null) {
                    events.error("NO_SENSOR", "Sensor tidak tersedia", null)
                    return
                }
                listener = object : SensorEventListener {
                    override fun onSensorChanged(event: SensorEvent) {
                        // kirim [nilai, range maksimal] ke Dart
                        events.success(
                            listOf(event.values[0].toDouble(),
                                   event.sensor.maximumRange.toDouble())
                        )
                    }
                    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
                }
                sensorManager.registerListener(
                    listener, sensor, SensorManager.SENSOR_DELAY_NORMAL)
            }

            // Dipanggil saat stream di-cancel dari Dart (setara unregisterListener)
            override fun onCancel(arguments: Any?) {
                listener?.let { sensorManager.unregisterListener(it) }
                listener = null
            }
        })
    }
}
```

> Jika project dibuat dengan organization lain, sesuaikan baris `package com.example.lbs` dan lokasi foldernya dengan `applicationId` project.

2. Buat file `lib/sensor_channel.dart` sebagai penghubung di sisi Dart:

DART

```
import 'package:flutter/services.dart';

class SensorChannel {
  static const _method = MethodChannel('polije/sensor');
  static const _light = EventChannel('polije/light');
  static const _proximity = EventChannel('polije/proximity');

  static Future<List<Map<String, dynamic>>> getSensorList() async {
    final hasil = await _method.invokeListMethod<Map>('getSensorList') ?? [];
    return hasil.map((e) => Map<String, dynamic>.from(e)).toList();
  }

  // Stream berisi [nilai, maxRange]
  static Stream<List<double>> _stream(EventChannel c) => c
      .receiveBroadcastStream()
      .map((e) => (e as List).map((v) => (v as num).toDouble()).toList());

  static Stream<List<double>> get light => _stream(_light);
  static Stream<List<double>> get proximity => _stream(_proximity);
}
```

Menampilkan Semua Sensor

1. Masing-masing item pada daftar dibuat sebagai widget terpisah karena akan diulang terus-menerus (setara `item_sensor.xml`).
2. Lengkapi `lib/all_sensor_page.dart` agar bisa menampilkan daftar sensor yang tersedia:

DART

```
import 'package:flutter/material.dart';
import 'sensor_channel.dart';

class AllSensorPage extends StatefulWidget {
  const AllSensorPage({super.key});

  @override
  State<AllSensorPage> createState() => _AllSensorPageState();
}

class _AllSensorPageState extends State<AllSensorPage> {
  late final Future<List<Map<String, dynamic>>> _sensors =
      SensorChannel.getSensorList();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Semua Sensor')),
      body: FutureBuilder<List<Map<String, dynamic>>>(
        future: _sensors,
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          }
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final data = snapshot.data!;
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (_, i) => ItemSensor(sensor: data[i]),
          );
        },
      ),
    );
  }
}

// Setara item_sensor.xml
class ItemSensor extends StatelessWidget {
  final Map<String, dynamic> sensor;
  const ItemSensor({super.key, required this.sensor});

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
      child: Padding(
        padding: const EdgeInsets.all(12),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('${sensor['nama']}',
                style: const TextStyle(fontWeight: FontWeight.bold)),
            Text('Tipe      : ${sensor['tipe']}'),
            Text('Power     : ${sensor['power']} mA'),
            Text('Max Range : ${sensor['maxRange']}'),
          ],
        ),
```

```
      ),
    );
  }
}
```

Menangkap Perubahan Nilai

Penerapan sensor Light dan Proximity dibahas pada subbab di bawah ini.

a. Light Sensor

1. Package `fl_chart` sudah ditambahkan pada Acara 35 (setara menambahkan library grafik pada `build.gradle`).
2. Lengkapi `lib/light_sensor_page.dart`. Perhatikan fungsi untuk menginisiasi grafik, implementasi listener (setara `onSensorChanged()`), dan penghentian listener pada `dispose()`:

DART

```
import 'dart:async';
import 'package:audioplayers/audioplayers.dart';
import 'package:fl_chart/fl_chart.dart';
import 'package:flutter/material.dart';
import 'package:wakelock_plus/wakelock_plus.dart';
import 'sensor_channel.dart';

class LightSensorPage extends StatefulWidget {
  const LightSensorPage({super.key});

  @override
  State<LightSensorPage> createState() => _LightSensorPageState();
}

class _LightSensorPageState extends State<LightSensorPage> {
  StreamSubscription<List<double>>? _sub;
  final _player = AudioPlayer();
  final List<FlSpot> _titik = [];
  double _x = 0;
  double _lux = 0;
  bool _gelap = false;
  String? _error;

  @override
  void initState() {
    super.initState();
    WakelockPlus.enable(); // layar tetap menyala (WAKE_LOCK)
    // Setara registerListener(...) + onSensorChanged(SensorEvent)
    _sub = SensorChannel.light.listen(
      _onSensorChanged,
      onError: (e) => setState(() => _error = '$e'),
    );
  }

  void _onSensorChanged(List<double> data) {
    final nilai = data[0]; // values[0] = lux
    setState(() {
      _lux = nilai;
      _titik.add(FlSpot(_x++, nilai));
      if (_titik.length > 50) _titik.removeAt(0); // simpan 50 data terakhir
    });
    final gelapSekarang = nilai < 10;
    if (gelapSekarang && !_gelap) {
      _player.play(AssetSource('audio/cahaya_gelap.mp3'));
    }
    _gelap = gelapSekarang;
  }
```

```
  // Fungsi menginisiasi grafik dan garisnya
  LineChartData _grafik() => LineChartData(
        minY: 0,
        titlesData: const FlTitlesData(
          topTitles: AxisTitles(sideTitles: SideTitles(showTitles: false)),
          rightTitles: AxisTitles(sideTitles: SideTitles(showTitles: false)),
        ),
        lineBarsData: [
          LineChartBarData(
            spots: _titik.isEmpty ? [const FlSpot(0, 0)] : List.of(_titik),
            isCurved: true,
            barWidth: 3,
            dotData: const FlDotData(show: false),
            belowBarData: BarAreaData(show: true),
          ),
        ],
      );

  @override
  void dispose() {
    _sub?.cancel(); // setara unregisterListener -> hemat baterai
    _player.dispose();
    WakelockPlus.disable();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Sensor Cahaya')),
      body: _error != null
          ? Center(child: Text(_error!))
          : Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                children: [
                  // setara string label_brightness
                  Text('Kecerahan: ${_lux.toStringAsFixed(1)} lux',
                      style: const TextStyle(fontSize: 22)),
                  Text(_gelap ? 'Kondisi: GELAP' : 'Kondisi: TERANG'),
                  const SizedBox(height: 24),
                  Expanded(child: LineChart(_grafik())),
                ],
              ),
            ),
    );
  }
}
```

3. Build dan run project tersebut, lalu perhatikan hasilnya. Pada emulator, ubah nilai cahaya melalui **Extended Controls → Virtual sensors → Additional sensors → Light**. Pada HP, tutup sensor cahaya (di dekat kamera depan) dengan jari.

[Tempelkan screenshot hasil running aplikasi]

Gambar 5. Hasil Light Sensor dalam bentuk grafik

b. Proximity Sensor

1. Buat UI dengan satu Text untuk menampilkan perubahan nilai secara terus-menerus (setara label pada `strings.xml`).

2. Lakukan `listen()` pada stream proximity (setara mengimplementasikan `SensorEventListener` dengan `onSensorChanged()` dan `onAccuracyChanged()`).
3. Sensor service dipanggil di sisi Kotlin menggunakan `SensorManager` dengan tipe `TYPE_PROXIMITY`.
4. Ketika halaman ditutup (tombol back), sensor dimatikan dengan `cancel()` pada `dispose()` yang memicu `unregisterListener` agar menghemat daya baterai (setara `onBackPressed()`).
5. Selengkapnya, tulislah kode berikut ini pada `lib/proximity_sensor_page.dart`:

DART
```
import 'dart:async';
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/material.dart';
import 'sensor_channel.dart';

class ProximitySensorPage extends StatefulWidget {
  const ProximitySensorPage({super.key});

  @override
  State<ProximitySensorPage> createState() => _ProximitySensorPageState();
}

class _ProximitySensorPageState extends State<ProximitySensorPage> {
  StreamSubscription<List<double>>? _sub;
  final _player = AudioPlayer();
  double _jarak = 0;
  bool? _dekat;
  String? _error;

  @override
  void initState() {
    super.initState();
    _sub = SensorChannel.proximity.listen(
      _onSensorChanged,
      onError: (e) => setState(() => _error = '$e'),
    );
  }

  void _onSensorChanged(List<double> data) {
    final nilai = data[0];      // jarak (cm)
    final maxRange = data[1];   // jarak maksimal sensor
    final dekat = nilai < maxRange;
    if (dekat != _dekat) {
      _player.play(AssetSource(
          dekat ? 'audio/objek_mendekat.mp3' : 'audio/objek_menjauh.mp3'));
    }
    setState(() {
      _jarak = nilai;
      _dekat = dekat;
    });
  }

  @override
  void dispose() {
    _sub?.cancel(); // unregisterListener
    _player.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final dekat = _dekat ?? false;
    return Scaffold(
```

```
      appBar: AppBar(title: const Text('Sensor Proximity')),
      body: Center(
        child: _error != null
            ? Text(_error!)
            : Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Icon(dekat ? Icons.back_hand : Icons.phone_android,
                      size: 96, color: dekat ? Colors.red : Colors.green),
                  const SizedBox(height: 16),
                  Text('Jarak: ${_jarak.toStringAsFixed(1)} cm',
                      style: const TextStyle(fontSize: 24)),
                  Text(dekat ? 'Objek mendekat' : 'Objek menjauh',
                      style: const TextStyle(fontSize: 18)),
                ],
              ),
      ),
    );
  }
}
```

6. Build dan run project tersebut. Pada HP, dekatkan tangan ke bagian atas layar; pada emulator, ubah nilai **Proximity** di *Virtual sensors*.

[Tempelkan screenshot hasil running aplikasi]

Gambar 6. Hasil Proximity Sensor

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code untuk menampilkan semua sensor dan menangkap perubahan nilai dengan implementasi sensor light dan sensor proximity.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa mampu menampilkan semua sensor dan menangkap perubahan nilai dengan implementasi sensor light dan sensor proximity.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code untuk menampilkan semua sensor dan menangkap perubahan nilai	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code untuk menampilkan semua sensor dan menangkap perubahan nilai	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code untuk menampilkan semua sensor	1	2	3	4

No	Indikator	Skor			
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code untuk menampilkan semua sensor	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 37 : PENGENALAN JSON

Pokok Bahasan	:	Pengenalan JSON
Acara Praktikum/Praktik	:	Minggu 11 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep JSON.
2. Mahasiswa mampu menerapkan JSON.

b. Indikator Penilaian

Keberhasilan mahasiswa memahami konsep JSON dan menerapkan JSON pada Flutter.

c. Dasar Teori

Teori JSON

JSON adalah singkatan dari *JavaScript Object Notation*. JSON adalah format minimal yang mudah dibaca untuk menyusun data, dan merupakan format pertukaran data yang independen antara server dan aplikasi. JSON merupakan alternatif terbaik untuk XML karena mewakili data dalam format teks sehingga dapat dengan mudah diurai (*parse*).

File JSON memiliki beberapa komponen yaitu **Array** (`[ ]`), **Object** (`{ }`), **Key** dan **Value**. Contoh:

```JSON
{
  "nama": "Politeknik Negeri Jember",
  "kota": "Jember",
  "prodi": [
    { "kode": "TKK", "nama": "Teknik Komputer" },
    { "kode": "TIF", "nama": "Teknik Informatika" }
  ]
}
```

Pada Android native disediakan kelas `JSONObject`, `JSONArray`, `JSONStringer` dan `JSONTokener` untuk memanipulasi data JSON. Pada Flutter digunakan library bawaan **dart:convert**:

- `jsonDecode(String)` — mengubah teks JSON menjadi objek Dart: JSON object menjadi `Map<String, dynamic>`, JSON array menjadi `List<dynamic>`.
- `jsonEncode(Object)` — mengubah objek Dart (Map/List) menjadi teks JSON.

Agar lebih aman dan mudah digunakan, hasil `jsonDecode` biasanya diubah menjadi **kelas model** melalui *factory constructor* `fromJson()` dan method `toJson()`.

```DART
import 'dart:convert';

class Prodi {
  final String kode;
```

```
  final String nama;
  Prodi({required this.kode, required this.nama});

  factory Prodi.fromJson(Map<String, dynamic> json) =>
      Prodi(kode: json['kode'], nama: json['nama']);

  Map<String, dynamic> toJson() => {'kode': kode, 'nama': nama};
}

void contoh(String teksJson) {
  final Map<String, dynamic> data = jsonDecode(teksJson);
  final List<Prodi> prodi = (data['prodi'] as List)
      .map((e) => Prodi.fromJson(e))
      .toList();
  print(prodi.first.nama); // Teknik Komputer
}
```

JSON digunakan pada aplikasi mobile ketika aplikasi membutuhkan data yang berasal dari suatu server untuk ditampilkan. Untuk mengambil data dari server digunakan **API** (*Application Programming Interface*), yaitu sekumpulan perintah, fungsi, protokol, dan objek yang dapat digunakan oleh programmer agar aplikasi dapat berinteraksi dengan sistem eksternal (server). Pada Flutter, permintaan HTTP ke API dilakukan dengan package **http** atau **dio**.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

1. Buka Android Studio.
2. Buat project Flutter baru dan berikan nama `json_moviedb`.
3. Menambahkan dependency melalui terminal, kemudian otomatis tersinkron (`pub get`):

```bash
flutter pub add http
```

4. Menambahkan izin internet pada `android/app/src/main/AndroidManifest.xml` (di atas tag `<application>`), agar aplikasi dapat mengakses API saat di-*build* release:

```xml
<uses-permission android:name="android.permission.INTERNET" />
```

5. Membuat konstanta teks dan warna. Buat file `lib/konstanta.dart` (setara `strings.xml` dan `colors.xml`):

```dart
import 'package:flutter/material.dart';

class AppStrings {
  static const appName = 'JSON MovieDB';
  static const jsonParse = 'JSON Parse';
  static const jsonApi = 'JSON API';
  static const movieDb = 'MovieDB';
}

class AppColors {
  static const primary = Color(0xFF01B4E4);   // warna khas TMDB
```

```
  static const secondary = Color(0xFF0D253F);
}
```

6. Mengubah file `lib/main.dart` sebagai menu utama yang berisi tiga tombol (setara `activity_main.xml` dan `MainActivity.java`). Halaman tujuan akan dibuat pada Acara 38 dan 39; sementara itu baris `import` dan tombolnya boleh diberi komentar terlebih dahulu.

DART
```
import 'package:flutter/material.dart';
import 'konstanta.dart';
import 'json_parse_page.dart';
import 'json_api_page.dart';
import 'moviedb_page.dart';

void main() => runApp(MaterialApp(
      debugShowCheckedModeBanner: false,
      title: AppStrings.appName,
      theme: ThemeData(colorSchemeSeed: AppColors.primary),
      home: const MainPage(),
    ));

class MainPage extends StatelessWidget {
  const MainPage({super.key});

  Widget _menu(BuildContext context, String label, IconData ikon,
      Widget tujuan) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 12),
      child: FilledButton.icon(
        style: FilledButton.styleFrom(
            minimumSize: const Size.fromHeight(56)),
        onPressed: () => Navigator.push(
            context, MaterialPageRoute(builder: (_) => tujuan)),
        icon: Icon(ikon),
        label: Text(label),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text(AppStrings.appName)),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            _menu(context, AppStrings.jsonParse, Icons.data_object,
                const JsonParsePage()),
            _menu(context, AppStrings.jsonApi, Icons.cloud_download,
                const JsonApiPage()),
            _menu(context, AppStrings.movieDb, Icons.movie,
                const MovieDbPage()),
          ],
        ),
      ),
    );
  }
}
```

7. Sebagai latihan pengenalan, buat file `lib/latihan_json.dart` lalu panggil fungsinya dari tombol sementara, dan amati hasilnya di tab **Run**:

DART
```
import 'dart:convert';
```

```
void latihanJson() {
  const teks = '{"nim":"E32241001","nama":"Andi","nilai":[80,90,85]}';

  // String JSON -> Map
  final Map<String, dynamic> mhs = jsonDecode(teks);
  print('Nama  : ${mhs['nama']}');
  final List nilai = mhs['nilai'];
  final rata = nilai.reduce((a, b) => a + b) / nilai.length;
  print('Rata-rata: $rata');

  // Map -> String JSON
  final baru = {'nim': 'E32241002', 'nama': 'Budi', 'aktif': true};
  print(jsonEncode(baru));
}
```

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code pengenalan JSON.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa memahami konsep JSON dan menerapkan JSON pada Flutter.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code JSON	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code JSON	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code JSON	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code JSON	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 38 : PENGENALAN JSON

Pokok Bahasan	:	Pengambilan API
Acara Praktikum/Praktik	:	Minggu 11 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep JSON.
2. Mahasiswa mampu menerapkan JSON Parse dan JSON API.
3. Mahasiswa mampu menguji URL API melalui Postman.

b. Indikator Penilaian

Keberhasilan mahasiswa memahami konsep JSON Parse dan JSON API, menguji URL API melalui Postman, dan menerapkan JSON pada Flutter.

c. Dasar Teori

JSON Parse dan JSON API

JSON Parse adalah proses mengurai teks JSON (yang sudah ada di aplikasi, misalnya dari file asset) menjadi objek yang dapat ditampilkan. **JSON API** adalah proses mengambil teks JSON dari server melalui HTTP request, kemudian mengurainya.

Karena permintaan ke server membutuhkan waktu, pada Android native proses ini dijalankan di *background thread* (AsyncTask/Thread). Pada Flutter cukup menggunakan `async`/`await` dan `Future`, lalu menampilkan hasilnya dengan widget **FutureBuilder** yang otomatis menampilkan *loading*, data, atau error.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

JSON Parse

1. Buka kembali project `json_moviedb` yang dibuat di bagian sebelumnya.
2. Menambahkan file data JSON `assets/data/mahasiswa.json` (buat folder `assets/data/` di root project):

```JSON
{
  "mahasiswa": [
    { "nim": "E32241001", "nama": "Andi Pratama", "prodi": "Teknik Komputer" },
    { "nim": "E32241002", "nama": "Budi Santoso", "prodi": "Teknik Komputer" },
    { "nim": "E31241003", "nama": "Citra Lestari", "prodi": "Teknik Informatika" },
    { "nim": "E33241004", "nama": "Dewi Anggraini", "prodi": "Manajemen Informatika" }
  ]
```

```
}
```

Daftarkan asset pada `pubspec.yaml`:

```yaml
flutter:
  uses-material-design: true
  assets:
    - assets/data/
```

3. Menambahkan file `lib/json_parse_page.dart`. Widget item list dibuat di file yang sama (setara `json_parse.xml`, `list_row.xml` dan `JsonParseActivity.java`):

```dart
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart' show rootBundle;

class JsonParsePage extends StatefulWidget {
  const JsonParsePage({super.key});

  @override
  State<JsonParsePage> createState() => _JsonParsePageState();
}

class _JsonParsePageState extends State<JsonParsePage> {
  late final Future<List<dynamic>> _data = _bacaJson();

  Future<List<dynamic>> _bacaJson() async {
    // 1. Membaca teks JSON dari asset
    final teks = await rootBundle.loadString('assets/data/mahasiswa.json');
    // 2. Parse teks menjadi Map
    final Map<String, dynamic> obj = jsonDecode(teks);
    // 3. Mengambil array "mahasiswa"
    return obj['mahasiswa'] as List<dynamic>;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('JSON Parse')),
      body: FutureBuilder<List<dynamic>>(
        future: _data,
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          }
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final list = snapshot.data!;
          return ListView.builder(
            itemCount: list.length,
            itemBuilder: (_, i) => ListRow(item: list[i]),
          );
        },
      ),
    );
  }
}

// Setara list_row.xml
class ListRow extends StatelessWidget {
  final Map<String, dynamic> item;
  const ListRow({super.key, required this.item});

  @override
```

```
  Widget build(BuildContext context) {
    return ListTile(
      leading: const Icon(Icons.person),
      title: Text(item['nama']),
      subtitle: Text('${item['nim']} - ${item['prodi']}'),
    );
  }
}
```

JSON API

1. Halaman JSON API menampilkan daftar kontak dari URL https://api.androidhive.info/contacts/ yang memiliki struktur seperti berikut:

JSON
```
{
  "contacts": [
    {
      "id": "c200",
      "name": "Ravi Tamada",
      "email": "ravi@gmail.com",
      "address": "xx-xx-xxxx,x - street, x - country",
      "gender": "male",
      "phone": {
        "mobile": "+91 0000000000",
        "home": "00 000000",
        "office": "00 000000"
      }
    }
  ]
}
```

2. Menambahkan file `lib/http_handler.dart` (setara `HttpHandler.java`):

DART
```
import 'package:http/http.dart' as http;

class HttpHandler {
  // Melakukan GET request dan mengembalikan body sebagai String
  static Future<String?> makeServiceCall(String url) async {
    try {
      final response = await http
          .get(Uri.parse(url))
          .timeout(const Duration(seconds: 15));
      if (response.statusCode == 200) {
        return response.body;
      }
      return null;
    } catch (e) {
      return null;
    }
  }
}
```

3. Menambahkan file `lib/json_api_page.dart` (setara `json_api.xml`, `list_item.xml`, dan `JsonAPIActivity.java`):

DART
```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'http_handler.dart';

class JsonApiPage extends StatefulWidget {
  const JsonApiPage({super.key});

  @override
  State<JsonApiPage> createState() => _JsonApiPageState();
}
```

```
class _JsonApiPageState extends State<JsonApiPage> {
  static const String url = 'https://api.androidhive.info/contacts/';
  late Future<List<dynamic>> _contacts = _getContacts();

  Future<List<dynamic>> _getContacts() async {
    final jsonStr = await HttpHandler.makeServiceCall(url);
    if (jsonStr == null) {
      throw Exception("Couldn't get json from server.");
    }
    final Map<String, dynamic> jsonObj = jsonDecode(jsonStr);
    return jsonObj['contacts'] as List<dynamic>;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('JSON API')),
      body: RefreshIndicator(
        onRefresh: () async {
          setState(() => _contacts = _getContacts());
          await _contacts;
        },
        child: FutureBuilder<List<dynamic>>(
          future: _contacts,
          builder: (context, snapshot) {
            if (snapshot.connectionState != ConnectionState.done) {
              return const Center(child: CircularProgressIndicator());
            }
            if (snapshot.hasError) {
              return ListView(children: [
                Padding(
                  padding: const EdgeInsets.all(24),
                  child: Text('${snapshot.error}'),
                ),
              ]);
            }
            final contacts = snapshot.data!;
            return ListView.builder(
              itemCount: contacts.length,
              itemBuilder: (_, i) {
                final c = contacts[i];
                final phone = c['phone'] as Map<String, dynamic>;
                // Setara list_item.xml (name, email, mobile)
                return ListTile(
                  title: Text(c['name'],
                      style: const TextStyle(fontWeight: FontWeight.bold)),
                  subtitle: Text('${c['email']}\n${phone['mobile']}'),
                  isThreeLine: true,
                );
              },
            );
          },
        ),
      ),
    );
  }
}
```

4. Untuk sementara, buat file `lib/moviedb_page.dart` berisi kerangka halaman (isinya dilengkapi pada Acara 39):

DART

```
import 'package:flutter/material.dart';

class MovieDbPage extends StatelessWidget {
  const MovieDbPage({super.key});

  @override
```

```
  Widget build(BuildContext context) =>
      Scaffold(appBar: AppBar(title: const Text('MovieDB')));
}
```

5. Jalankan aplikasi, buka menu JSON Parse dan JSON API, lalu amati hasilnya.
6. **Menguji URL API melalui Postman.**

- Buka https://web.postman.co/home (atau aplikasi Postman desktop) dan login.
- Buat request baru dengan metode **GET** dan URL `https://api.androidhive.info/contacts/`.
- Klik **Send** dan amati struktur JSON pada bagian *Response* (status 200 OK, *body*, *headers*, waktu respons).
- Bandingkan struktur yang tampil di Postman dengan cara kode Dart mengambil `contacts`, `name`, `email`, dan `phone.mobile`.

> Jika URL contoh di atas tidak dapat diakses, gunakan API publik lain dengan struktur serupa, misalnya https://jsonplaceholder.typicode.com/users, lalu sesuaikan nama key yang diambil.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code JSON Parse dan JSON API, dengan berhasil menguji URL API melalui Postman.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.
3. Dokumentasi hasil pengujian URL API melalui Postman.

g. Kesimpulan

Mahasiswa mampu menerapkan JSON Parse dan JSON API serta menguji URL API melalui Postman.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code, dan pengujian melalui postman	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code, dan pengujian melalui postman	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code, dan pengujian melalui postman	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code, dan pengujian melalui postman	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 39 : PENGENALAN JSON

Pokok Bahasan	:	Implementasi JSON API MovieDB
Acara Praktikum/Praktik	:	Minggu 11 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep MovieDB.
2. Mahasiswa mampu menerapkan JSON API MovieDB pada Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa menerapkan JSON API MovieDB pada Flutter.

c. Dasar Teori

MovieDB

The Movie Database (TMDB, https://www.themoviedb.org) adalah website penyedia informasi tentang film yang juga menyediakan API secara gratis dengan syarat harus mendaftar (*register*) terlebih dahulu.

Untuk mendapatkan API key: masuk ke website tersebut lalu pilih **Join TMDB/Sign Up**. Setelah akun aktif, buka menu **Settings** pada profil, kemudian pilih menu **API** dan lakukan *request/generate* API key (jenis *Developer*). Isi formulir aplikasi; *Application URL* dapat menggunakan URL repositori GitHub project. Setelah disetujui akan tampil **API Key (v3 auth)** dan **API Read Access Token**.

Beberapa endpoint yang digunakan:

Endpoint	Keterangan
https://api.themoviedb.org/3/movie/now_playing?api_key=API_KEY	Daftar film yang sedang tayang
https://api.themoviedb.org/3/movie/upcoming?api_key=API_KEY	Daftar film yang akan tayang
https://image.tmdb.org/t/p/w500/{poster_path}	URL gambar poster film

> API key bersifat rahasia. Jangan mengunggah API key ke repositori publik; simpan di file terpisah yang dimasukkan ke `.gitignore`, atau kirim melalui `--dart-define`.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

JSON API MovieDB

1. Buka kembali project `json_moviedb` yang dibuat di bagian sebelumnya.

2. Tambahkan package untuk *caching* gambar (setara Picasso/Glide):

```bash
flutter pub add cached_network_image
```

3. Menambahkan file `lib/api_endpoint.dart` (setara `ApiEndpoint.java`):

```dart
class ApiEndpoint {
  static const String baseUrl = 'https://api.themoviedb.org/3/';
  static const String apiKey = 'ISI_DENGAN_API_KEY_ANDA';
  static const String nowPlaying = 'movie/now_playing?api_key=$apiKey&language=en-US';
  static const String upcoming = 'movie/upcoming?api_key=$apiKey&language=en-US';
  static const String urlImage = 'https://image.tmdb.org/t/p/w500';
}
```

4. Menambahkan file `lib/model_movie.dart` (setara `ModelMovie.java`):

```dart
class ModelMovie {
  final int id;
  final String title;
  final String overview;
  final String releaseDate;
  final double voteAverage;
  final String? posterPath;

  ModelMovie({
    required this.id,
    required this.title,
    required this.overview,
    required this.releaseDate,
    required this.voteAverage,
    this.posterPath,
  });

  factory ModelMovie.fromJson(Map<String, dynamic> json) => ModelMovie(
        id: json['id'],
        title: json['title'] ?? '-',
        overview: json['overview'] ?? '',
        releaseDate: json['release_date'] ?? '',
        voteAverage: (json['vote_average'] as num?)?.toDouble() ?? 0,
        posterPath: json['poster_path'],
      );
}
```

5. Menambahkan file `lib/movie_item.dart` sebagai tampilan satu item film (setara `list_item_film.xml` dan `MovieAdapter.java`):

```dart
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'api_endpoint.dart';
import 'model_movie.dart';

class MovieItem extends StatelessWidget {
  final ModelMovie movie;
  const MovieItem({super.key, required this.movie});

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      clipBehavior: Clip.antiAlias,
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          SizedBox(
```

```
            width: 100,
            height: 150,
            child: movie.posterPath == null
                ? const Icon(Icons.movie, size: 48)
                : CachedNetworkImage(
                    imageUrl: '${ApiEndpoint.urlImage}${movie.posterPath}',
                    fit: BoxFit.cover,
                    placeholder: (_, __) =>
                        const Center(child: CircularProgressIndicator()),
                    errorWidget: (_, __, ___) => const Icon(Icons.broken_image),
                  ),
          ),
          Expanded(
            child: Padding(
              padding: const EdgeInsets.all(10),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(movie.title,
                      style: const TextStyle(
                          fontSize: 16, fontWeight: FontWeight.bold)),
                  const SizedBox(height: 4),
                  Text('Rilis: ${movie.releaseDate}'),
                  Row(children: [
                    const Icon(Icons.star, size: 16, color: Colors.amber),
                    Text(' ${movie.voteAverage.toStringAsFixed(1)}'),
                  ]),
                  const SizedBox(height: 4),
                  Text(movie.overview,
                      maxLines: 3, overflow: TextOverflow.ellipsis),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

6. Menambahkan file `lib/fragment_movie.dart` yang memuat data film dari API (setara `fragmen_film.xml` dan `FragmentMovie.java`):

DART

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'api_endpoint.dart';
import 'model_movie.dart';
import 'movie_item.dart';

class FragmentMovie extends StatefulWidget {
  final String endpoint; // nowPlaying atau upcoming
  const FragmentMovie({super.key, required this.endpoint});

  @override
  State<FragmentMovie> createState() => _FragmentMovieState();
}

class _FragmentMovieState extends State<FragmentMovie>
    with AutomaticKeepAliveClientMixin {
  late Future<List<ModelMovie>> _movies = _getMovie();

  Future<List<ModelMovie>> _getMovie() async {
    final res = await http.get(
        Uri.parse('${ApiEndpoint.baseUrl}${widget.endpoint}'));
    if (res.statusCode != 200) {
      throw Exception('Gagal memuat data (${res.statusCode})');
    }
```

```
    final Map<String, dynamic> json = jsonDecode(res.body);
    final List results = json['results'];
    return results.map((e) => ModelMovie.fromJson(e)).toList();
  }

  @override
  bool get wantKeepAlive => true; // data tidak dimuat ulang saat pindah tab

  @override
  Widget build(BuildContext context) {
    super.build(context);
    return RefreshIndicator(
      onRefresh: () async {
        setState(() => _movies = _getMovie());
        await _movies;
      },
      child: FutureBuilder<List<ModelMovie>>(
        future: _movies,
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return ListView(children: [
              Padding(
                padding: const EdgeInsets.all(24),
                child: Text('${snapshot.error}'),
              ),
            ]);
          }
          final movies = snapshot.data!;
          return ListView.builder(
            itemCount: movies.length,
            itemBuilder: (_, i) => MovieItem(movie: movies[i]),
          );
        },
      ),
    );
  }
}
```

7. Mengubah file `lib/moviedb_page.dart` berisi AppBar dan TabBar (setara `moviedb.xml`, `toolbar_main.xml`, dan `MovieDBActivity.java`):

DART

```
import 'package:flutter/material.dart';
import 'api_endpoint.dart';
import 'fragment_movie.dart';

class MovieDbPage extends StatelessWidget {
  const MovieDbPage({super.key});

  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 2,
      child: Scaffold(
        appBar: AppBar(
          title: const Text('MovieDB'),
          bottom: const TabBar(tabs: [
            Tab(text: 'Now Playing'),
            Tab(text: 'Upcoming'),
          ]),
        ),
        body: const TabBarView(children: [
          FragmentMovie(endpoint: ApiEndpoint.nowPlaying),
          FragmentMovie(endpoint: ApiEndpoint.upcoming),
        ]),
      ),
```

```
      );
    }
}
```

8. Pastikan izin `INTERNET` sudah ada pada `AndroidManifest.xml` (Acara 37 langkah 4), lalu jalankan aplikasi dan buka menu **MovieDB**.

[Tempelkan screenshot hasil running aplikasi]

Gambar 1. Hasil implementasi JSON API MovieDB

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code JSON API MovieDB.
2. Dokumentasi hasil implementasi code dalam bentuk laporan.

g. Kesimpulan

Mahasiswa mampu menerapkan JSON API MovieDB pada Flutter.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code disertai menjelaskan setiap baris code JSON API movieDB	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code JSON API movieDB	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code JSON API movieDB	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code JSON API movieDB	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 40 : PENGENALAN JSON

Pokok Bahasan	:	Pengujian JSON API MovieDB melalui Postman
Acara Praktikum/Praktik	:	Minggu 11 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep MovieDB.
2. Mahasiswa mampu menguji hasil implementasi JSON API MovieDB melalui Postman dan menerapkannya pada Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam menguji hasil implementasi JSON API MovieDB melalui Postman.

c. Dasar Teori

Postman

Postman adalah aplikasi yang digunakan untuk pengembangan API. Aplikasi ini bekerja dengan cara menguji coba request API yang dikirim ke server dan response yang diterima dari server. Dengan begitu, pengembang aplikasi dapat mengetahui struktur data yang akan digunakan pada aplikasi yang tengah dibuat sehingga mampu mempercepat proses pengembangan aplikasi. Struktur yang terlihat di Postman menjadi acuan saat membuat kelas model Dart (`fromJson`).

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

1. Menguji URL API melalui Postman.
2. Buka halaman https://web.postman.co/home (atau aplikasi Postman desktop).
3. Buat request **GET** dan salin link berikut pada Postman (ganti `API_KEY_ANDA` dengan API key milik sendiri):

```
https://api.themoviedb.org/3/movie/now_playing?api_key=API_KEY_ANDA
```

4. Cara lain: isi URL `https://api.themoviedb.org/3/movie/now_playing`, lalu pada tab **Params** tambahkan key `api_key` dengan value API key Anda.
5. Klik **Send**, lalu amati hasil response: `page`, `results` (berisi `id`, `title`, `overview`, `poster_path`, `release_date`, `vote_average`), `total_pages`, dan `total_results`.

6. Ulangi untuk endpoint `movie/upcoming`, kemudian cocokkan key yang tampil dengan field pada kelas `ModelMovie` (Acara 39).
7. Jalankan kembali aplikasi Flutter dan bandingkan data yang tampil di aplikasi dengan hasil di Postman.

[Tempelkan screenshot hasil running aplikasi]

Gambar 1. Hasil pengujian API MovieDB pada Postman

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code JSON API MovieDB.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.
3. Dokumentasi hasil pengujian URL API melalui Postman.

g. Kesimpulan

Mahasiswa mampu menerapkan JSON API MovieDB serta menguji URL API melalui Postman.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code, dan pengujian melalui postman	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code, dan pengujian melalui postman	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code, dan pengujian melalui postman	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code, dan pengujian melalui postman	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 41 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST

Pokok Bahasan : Pengenalan Dio (padanan Retrofit)
Acara Praktikum/Praktik : Minggu 12 / 1
Tempat : Daring / Luring Politeknik Negeri Jember
Alokasi Waktu : 170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep library HTTP client Dio (padanan Retrofit).
2. Mahasiswa mampu menerapkan Dio pada Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa memahami serta menerapkan konsep Dio, dengan implementasi source code pada kasus tertentu.

c. Dasar Teori

Retrofit dan Volley pada Android merupakan library yang digunakan sebagai media komunikasi antara client (smartphone) dengan server berbasis *web service*. Kedua library seolah-olah menjadi penghubung antara smartphone dengan server: smartphone bertugas melakukan *request* kepada server, kemudian server memberikan *response* data sesuai yang diminta. Pada Flutter, peran yang sama dijalankan oleh package **dio** (padanan Retrofit + OkHttp) dan **http** (padanan Volley yang ringan).

Android	Flutter	Keterangan
Retrofit + OkHttp	dio	HTTP client lengkap: BaseOptions (base URL, timeout), interceptor, FormData, upload/download, pembatalan request.
Retrofit (anotasi @GET/@POST)	retrofit (package Dart) + retrofit_generator	Versi Dart dari Retrofit berbasis anotasi dan code generation, dibangun di atas Dio (pengayaan).
Volley	http	HTTP client resmi tim Dart, sederhana dan ringan.
Gson / converter-gson	dart:convert (jsonDecode) atau json_serializable	Mengubah response JSON menjadi objek model.
Picasso / Glide / Volley ImageLoader	Image.network atau cached_network_image	Menampilkan gambar dari URL.

Kedua library menyediakan 4 metode HTTP Request yang umum digunakan, antara lain:

- **GET**, digunakan untuk membaca data (*read only*) pada *resource*.
- **POST**, digunakan untuk mengirimkan atau membuat *resource* baru pada server.
- **PUT**, digunakan untuk memperbarui data pada *resource*.
- **DELETE**, digunakan untuk menghapus data pada *resource*.

Dio

Dio adalah HTTP client untuk Dart yang mendukung konfigurasi global, *interceptor*, *FormData*, pembatalan request, *timeout*, serta upload dan download file. Dio memudahkan komunikasi berbasis web service (REST client) dengan format pertukaran data **JSON** atau XML. Seperti Retrofit, request dan response dapat ditangani dengan mudah seperti pemanggilan fungsi biasa. Jika server mengirim header `Content-Type: application/json`, Dio otomatis mengubah response menjadi `Map`/`List` Dart.

Pada praktikum ini, format pertukaran data menggunakan JSON karena lebih cepat, ringan, dan umum digunakan oleh para pengembang aplikasi web maupun mobile.

Langkah pertama yaitu menambahkan izin **INTERNET** pada manifest agar aplikasi dapat terhubung ke server meskipun server terletak pada *localhost*. Selanjutnya tambahkan package `dio` pada project.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet
- XAMPP (Apache + MySQL) sebagai web server lokal

e. Prosedur Kerja

- Praktikum ini akan mengimplementasikan instalasi Dio dan http pada aplikasi Flutter.
- Instalasi meliputi package `dio` dan `http`; konversi JSON menggunakan `dart:convert` bawaan Dart.

Menyiapkan Web Service (API) di Localhost

1. Jalankan **Apache** dan **MySQL** pada XAMPP Control Panel.
2. Buka http://localhost/phpmyadmin, buat database `db_retrovolley`, lalu jalankan SQL berikut:

```sql
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_fullname VARCHAR(100) NOT NULL,
  user_email VARCHAR(100) NOT NULL UNIQUE,
  user_password VARCHAR(255) NOT NULL
);
```

3. Buat folder `C:\xampp\htdocs\retrovolley\` lalu buat file `koneksi.php`:

```php
<?php
header('Content-Type: application/json');
$conn = new mysqli('localhost', 'root', '', 'db_retrovolley');
if ($conn->connect_error) {
    echo json_encode(['code' => 500, 'status' => 'Koneksi database gagal']);
    exit;
}
```

4. Buat file `User_Registration.php` (GET = ambil semua user, POST = tambah user):

```php
<?php
require 'koneksi.php';

if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $sql = 'SELECT id, user_fullname, user_email FROM users ORDER BY id';
    $hasil = $conn->query($sql);
    $list = [];
    while ($row = $hasil->fetch_assoc()) {
        $row['id'] = (int) $row['id'];
        $list[] = $row;
    }
    echo json_encode(['code' => 200, 'status' => 'success', 'user_list' => $list]);
} else {
    $nama  = $_POST['user_fullname'] ?? '';
    $email = $_POST['user_email'] ?? '';
    $pass  = $_POST['user_password'] ?? '';
    if ($nama === '' || $email === '' || $pass === '') {
        echo json_encode(['code' => 400, 'status' => 'Data belum lengkap']);
        exit;
    }
    $hash = password_hash($pass, PASSWORD_DEFAULT);
    $stmt = $conn->prepare('INSERT INTO users (user_fullname, user_email, '
        . 'user_password) VALUES (?, ?, ?)');
    $stmt->bind_param('sss', $nama, $email, $hash);
    if ($stmt->execute()) {
        echo json_encode(['code' => 200, 'status' => 'Registrasi berhasil']);
    } else {
        echo json_encode(['code' => 409, 'status' => 'Email sudah terdaftar']);
    }
}
```

5. Buat file `User_Login.php`:

```php
<?php
require 'koneksi.php';

$email = $_POST['user_email'] ?? '';
$pass  = $_POST['user_password'] ?? '';
$stmt = $conn->prepare('SELECT id, user_fullname, user_email, user_password '
    . 'FROM users WHERE user_email = ?');
$stmt->bind_param('s', $email);
$stmt->execute();
$user = $stmt->get_result()->fetch_assoc();

if ($user && password_verify($pass, $user['user_password'])) {
    unset($user['user_password']);
    $user['id'] = (int) $user['id'];
    echo json_encode(['code' => 200, 'status' => 'Login berhasil', 'user' => $user]);
} else {
    echo json_encode(['code' => 401, 'status' => 'Email atau password salah']);
}
```

6. Uji API dengan Postman: `GET http://localhost/retrovolley/User_Registration.php`, dan `POST` dengan *Body* → *x-www-form-urlencoded* berisi `user_fullname`, `user_email`, `user_password`.

Dio

1. Buka Android Studio.
2. Buat project Flutter baru dan berikan nama `retrovolley`.
3. Pastikan target platform Android tercentang (minimal SDK mengikuti default Flutter, API 21 ke atas).

4. Tambahkan permission INTERNET dan izinkan koneksi HTTP (non-HTTPS) ke server lokal pada `android/app/src/main/AndroidManifest.xml`. Hal ini diperlukan meskipun web server berada pada localhost di laptop masing-masing:

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:label="retrovolley"
        android:usesCleartextTraffic="true"
        ...>
```

- Permission INTERNET menandakan bahwa aplikasi dapat melakukan koneksi internet.
- `usesCleartextTraffic="true"` diperlukan karena Android 9 ke atas memblokir koneksi `http://` tanpa enkripsi.

5. Selanjutnya, tambahkan dua package di bawah ini agar dapat mengimplementasikan Dio dan http pada project:

```bash
flutter pub add dio http
```

6. Perintah tersebut otomatis menjalankan `flutter pub get` sehingga library tersimpan pada project (setara *Sync Now*).
7. Buat struktur folder agar lebih rapi:

```
lib/
├── main.dart
├── global_variable.dart
├── model/          (user.dart, response_model.dart)
├── service/        (api_service.dart, http_service.dart)
└── pages/          (login_page.dart, home_page.dart, add_user_page.dart,
                     dio_page.dart, http_page.dart, update_user_page.dart)
```

8. Buat file `lib/global_variable.dart` untuk menampung BASE URL. Alamat `10.0.2.2` adalah alamat *localhost* laptop bila diakses dari emulator Android. Jika memakai HP, ganti dengan IP laptop (cek dengan `ipconfig`), misalnya `http://192.168.1.5/retrovolley/`.

```dart
class GlobalVariable {
  static String baseUrl = 'http://10.0.2.2/retrovolley/';
}
```

9. Buat file `lib/model/user.dart` dan `lib/model/response_model.dart`:

```dart
// lib/model/user.dart
class User {
  final int id;
  final String userFullname;
  final String userEmail;

  User({required this.id, required this.userFullname, required this.userEmail});

  factory User.fromJson(Map<String, dynamic> json) => User(
        id: int.parse('${json['id']}'),
        userFullname: json['user_fullname'] ?? '',
        userEmail: json['user_email'] ?? '',
```

```
      );
}
```

DART

```
// lib/model/response_model.dart
import 'user.dart';

// Response yang hanya berisi code dan status
class ResponseModel {
  final int code;
  final String status;
  final User? user;

  ResponseModel({required this.code, required this.status, this.user});

  factory ResponseModel.fromJson(Map<String, dynamic> json) => ResponseModel(
        code: int.parse('${json['code']}'),
        status: json['status'] ?? '',
        user: json['user'] == null ? null : User.fromJson(json['user']),
      );
}

// Response berisi code, status dan daftar user
class ResponseUser {
  final int code;
  final String status;
  final List<User> userList;

  ResponseUser({required this.code, required this.status, required this.userList});

  factory ResponseUser.fromJson(Map<String, dynamic> json) => ResponseUser(
        code: int.parse('${json['code']}'),
        status: json['status'] ?? '',
        userList: (json['user_list'] as List? ?? [])
            .map((e) => User.fromJson(e))
            .toList(),
      );
}
```

10. Buat file `lib/service/api_service.dart`. Kelas ini menyimpan seluruh request HTTP menggunakan Dio (setara interface `MethodHTTP` dan *RetrofitClient*):

DART

```
import 'dart:convert';
import 'package:dio/dio.dart';
import '../global_variable.dart';
import '../model/response_model.dart';

class ApiService {
  // Dibuat ulang setiap dipanggil agar mengikuti BASE URL terbaru
  static Dio get _dio => Dio(BaseOptions(
        baseUrl: GlobalVariable.baseUrl,
        connectTimeout: const Duration(seconds: 10),
        receiveTimeout: const Duration(seconds: 10),
        contentType: Headers.formUrlEncodedContentType,
      ));

  // Memastikan data berupa Map walaupun server tidak mengirim header JSON
  static Map<String, dynamic> _map(dynamic data) =>
      data is String ? jsonDecode(data) : Map<String, dynamic>.from(data);

  // POST User_Login.php
  static Future<ResponseModel> login(String email, String password) async {
    final res = await _dio.post('User_Login.php', data: {
      'user_email': email,
      'user_password': password,
    });
```

```
    return ResponseModel.fromJson(_map(res.data));
  }

  // POST User_Registration.php
  static Future<ResponseModel> register(
      String fullname, String email, String password) async {
    final res = await _dio.post('User_Registration.php', data: {
      'user_fullname': fullname,
      'user_email': email,
      'user_password': password,
    });
    return ResponseModel.fromJson(_map(res.data));
  }
}
```

11. Buat halaman menu utama `lib/pages/home_page.dart` yang menampilkan 2 ikon (Dio dan http), setara dua ImageView `ic_retrofit` dan `ic_volley` pada `activity_main.xml`. Ikon diambil dari kelas `Icons` bawaan Flutter (setara *Vector Asset*):

DART
```
import 'package:flutter/material.dart';
import 'dio_page.dart';
import 'http_page.dart';

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  Widget _ikon(BuildContext context, IconData ikon, String label,
      Widget tujuan) {
    return InkWell(
      // setara android:onClick="actionRetrofit" / "actionVolley"
      onTap: () => Navigator.push(
          context, MaterialPageRoute(builder: (_) => tujuan)),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(children: [
          Icon(ikon, size: 96),
          Text(label, style: const TextStyle(fontSize: 18)),
        ]),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Retrovolley')),
      body: Center(
        child: Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            _ikon(context, Icons.flash_on, 'Dio', const DioPage()),
            _ikon(context, Icons.http, 'http', const HttpPage()),
          ],
        ),
      ),
    );
  }
}
```

12. Buat kerangka halaman `lib/pages/dio_page.dart` dan `lib/pages/http_page.dart` (setara RetrofitActivity dan VolleyActivity; isinya dilengkapi pada Acara 43):

DART
```
import 'package:flutter/material.dart';
```

```
class DioPage extends StatelessWidget {
  const DioPage({super.key});

  @override
  Widget build(BuildContext context) =>
      Scaffold(appBar: AppBar(title: const Text('Dio')));
}
```

(buat juga `HttpPage` dengan cara yang sama).

13. Agar aplikasi lebih menarik, proses input BASE URL ditambahkan melalui ikon *link* yang terletak pada AppBar (setara menu ActionBar). Buat file `lib/pages/base_url_dialog.dart`:

DART
```
import 'package:flutter/material.dart';
import '../global_variable.dart';

Future<void> showBaseUrlDialog(BuildContext context) async {
  final c = TextEditingController(text: GlobalVariable.baseUrl);
  await showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('BASE URL'),
      content: TextField(
        controller: c,
        keyboardType: TextInputType.url,
        decoration: const InputDecoration(hintText: 'http://10.0.2.2/retrovolley/'),
      ),
      actions: [
        TextButton(onPressed: () => Navigator.pop(ctx), child: const Text('BATAL')),
        FilledButton(
          onPressed: () {
            var url = c.text.trim();
            if (!url.endsWith('/')) url = '$url/';
            GlobalVariable.baseUrl = url;
            Navigator.pop(ctx);
          },
          child: const Text('SIMPAN'),
        ),
      ],
    ),
  );
  c.dispose();
}
```

Membuat Halaman Login

- Fitur login merupakan langkah awal yang disediakan pada hampir seluruh aplikasi.
- Fitur ini juga akan diterapkan saat membuat proyek akhir.

1. Buatlah halaman login yang berisi kolom Email dan Password, serta tombol Login, dan teks **Register** untuk melakukan registrasi apabila pengguna belum memiliki akun.
2. Inisiasi controller email dan password, verifikasi input ketika pengguna menekan tombol Login (periksa apakah kolom sudah diisi), kemudian buat fungsi `loginUsingDio()`. Tuliskan pada `lib/pages/login_page.dart`:

DART
```
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import '../service/api_service.dart';
import 'add_user_page.dart';
import 'base_url_dialog.dart';
```

```
import 'home_page.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _email = TextEditingController();
  final _password = TextEditingController();
  bool _loading = false;

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  void _toast(String pesan) => ScaffoldMessenger.of(context)
      .showSnackBar(SnackBar(content: Text(pesan)));

  // Verifikasi input (setara actionLogin)
  void actionLogin() {
    if (_email.text.trim().isEmpty) {
      _toast('Email harus diisi');
      return;
    }
    if (_password.text.isEmpty) {
      _toast('Password harus diisi');
      return;
    }
    loginUsingDio();
  }

  Future<void> loginUsingDio() async {
    setState(() => _loading = true);
    try {
      final res = await ApiService.login(_email.text.trim(), _password.text);
      if (!mounted) return;
      if (res.code == 200) {
        _toast('Selamat datang, ${res.user?.userFullname}');
        Navigator.pushReplacement(
            context, MaterialPageRoute(builder: (_) => const HomePage()));
      } else {
        _toast(res.status);
      }
    } on DioException catch (e) {
      _toast('Gagal terhubung ke server: ${e.message}');
    } catch (e) {
      _toast('Terjadi kesalahan: $e');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Login'),
        actions: [
          IconButton(
            icon: const Icon(Icons.link),
            tooltip: 'Atur BASE URL',
            onPressed: () => showBaseUrlDialog(context),
          ),
```

```
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(24),
        children: [
          TextField(
            controller: _email,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
                labelText: 'Email', border: OutlineInputBorder()),
          ),
          const SizedBox(height: 12),
          TextField(
            controller: _password,
            obscureText: true,
            decoration: const InputDecoration(
                labelText: 'Password', border: OutlineInputBorder()),
          ),
          const SizedBox(height: 24),
          FilledButton(
            onPressed: _loading ? null : actionLogin,
            child: _loading
                ? const SizedBox(
                    width: 20, height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2))
                : const Text('LOGIN'),
          ),
          TextButton(
            onPressed: () => Navigator.push(context,
                MaterialPageRoute(builder: (_) => const AddUserPage())),
            child: const Text('Register'),
          ),
        ],
      ),
    );
  }
}
```

Membuat Halaman Registrasi

1. Buat halaman baru dengan nama `AddUserPage` yang berisi 3 kolom input: Nama Lengkap, Email, dan Password. ID tidak perlu ditambahkan karena sudah *auto increment* oleh DBMS. Tambahkan tombol untuk mengirim (*submit*) data ke web server.
2. Lengkapi kode `lib/pages/add_user_page.dart` seperti di bawah ini:

DART

```
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import '../service/api_service.dart';

class AddUserPage extends StatefulWidget {
  const AddUserPage({super.key});

  @override
  State<AddUserPage> createState() => _AddUserPageState();
}

class _AddUserPageState extends State<AddUserPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();
  final _password = TextEditingController();

  @override
  void dispose() {
    _nama.dispose();
    _email.dispose();
    _password.dispose();
```

```
    super.dispose();
  }

  void _toast(String pesan) => ScaffoldMessenger.of(context)
      .showSnackBar(SnackBar(content: Text(pesan)));

  Future<void> submitByDio() async {
    if (!_formKey.currentState!.validate()) return;
    try {
      final res = await ApiService.register(
          _nama.text.trim(), _email.text.trim(), _password.text);
      if (!mounted) return;
      _toast(res.status);
      if (res.code == 200) Navigator.pop(context, true);
    } on DioException catch (e) {
      _toast('Gagal terhubung ke server: ${e.message}');
    } catch (e) {
      _toast('Terjadi kesalahan: $e');
    }
  }

  String? _wajib(String? v) =>
      (v == null || v.trim().isEmpty) ? 'Wajib diisi' : null;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah User')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(labelText: 'Nama Lengkap'),
              validator: _wajib,
            ),
            TextFormField(
              controller: _email,
              keyboardType: TextInputType.emailAddress,
              decoration: const InputDecoration(labelText: 'Email'),
              validator: _wajib,
            ),
            TextFormField(
              controller: _password,
              obscureText: true,
              decoration: const InputDecoration(labelText: 'Password'),
              validator: _wajib,
            ),
            const SizedBox(height: 24),
            FilledButton(
              onPressed: submitByDio,
              child: const Text('SUBMIT (Dio)'),
            ),
          ],
        ),
      ),
    );
  }
}
```

3. Ubah `lib/main.dart` agar halaman pertama adalah halaman login:

DART

```
import 'package:flutter/material.dart';
import 'pages/login_page.dart';

void main() => runApp(MaterialApp(
      debugShowCheckedModeBanner: false,
```

```
      title: 'Retrovolley',
      theme: ThemeData(colorSchemeSeed: Colors.deepOrange),
      home: const LoginPage(),
    ));
```

4. Jalankan aplikasi. Atur BASE URL melalui ikon *link*, lakukan registrasi, kemudian login dengan akun yang baru dibuat.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code Dio dan UI-nya, halaman register, dan halaman login.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa memahami serta menerapkan konsep Dio (padanan Retrofit), dengan implementasi source code pada kasus tertentu.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code Dio	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code Dio	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code Dio	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code Dio	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 42 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST

Pokok Bahasan	:	Pengenalan http (padanan Volley)
Acara Praktikum/Praktik	:	Minggu 12 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep library http (padanan Volley).
2. Mahasiswa mampu menerapkan http pada Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa memahami serta menerapkan konsep http, dengan implementasi source code pada kasus tertentu.

c. Dasar Teori

Package http

Volley di Android adalah kerangka kerja HTTP yang menawarkan penjadwalan permintaan, penanganan permintaan secara bersamaan, *cache* respons, prioritas permintaan, dan pembatalan permintaan. Volley menyediakan penangan respons bawaan untuk string, gambar, dan JSON, serta cocok untuk mentransmisikan data aplikasi yang sederhana.

Padanannya pada Flutter adalah package **http** buatan tim Dart. Package ini menyediakan fungsi tingkat tinggi `http.get()`, `http.post()`, `http.put()`, dan `http.delete()` yang mengembalikan objek `Response` berisi `statusCode` dan `body` (String). Karena berbasis `Future`, banyak request dapat dijalankan bersamaan tanpa memblokir tampilan.

Perbedaan utama dengan Dio:

- `http` lebih ringan dan sederhana, tetapi tidak memiliki interceptor dan konfigurasi global bawaan.
- Response `http` selalu berupa String, sehingga harus diubah dengan `jsonDecode()` secara manual.
- Body berupa `Map<String, String>` otomatis dikirim sebagai `application/x-www-form-urlencoded` sehingga dapat dibaca PHP melalui `$_POST`.
- Untuk unduhan file besar sebaiknya menggunakan Dio (`download()`) atau package khusus download manager.

Package http ditambahkan dengan perintah `flutter pub add http`. Karena request dan response dikirim melalui jaringan, aplikasi memerlukan izin `android.permission.INTERNET` pada manifest.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

1. Buka kembali project `retrovolley` yang dibuat di bagian sebelumnya.
2. Pastikan package `http` sudah ditambahkan sehingga jumlah dependency yang ditambahkan menjadi 2 (dio dan http). Periksa pada `pubspec.yaml`:

```yaml
dependencies:
  flutter:
    sdk: flutter
  dio: ^5.0.0     # versi mengikuti hasil flutter pub add
  http: ^1.0.0
```

3. Buat file `lib/service/http_service.dart` (setara pemanggilan `Volley.newRequestQueue` dan `StringRequest`):

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../global_variable.dart';
import '../model/response_model.dart';

class HttpService {
  static Uri _url(String path) => Uri.parse('${GlobalVariable.baseUrl}$path');

  static Future<ResponseModel> login(String email, String password) async {
    final res = await http.post(_url('User_Login.php'), body: {
      'user_email': email,
      'user_password': password,
    }).timeout(const Duration(seconds: 10));
    return ResponseModel.fromJson(jsonDecode(res.body));
  }

  static Future<ResponseModel> register(
      String fullname, String email, String password) async {
    final res = await http.post(_url('User_Registration.php'), body: {
      'user_fullname': fullname,
      'user_email': email,
      'user_password': password,
    }).timeout(const Duration(seconds: 10));
    return ResponseModel.fromJson(jsonDecode(res.body));
  }
}
```

4. Buatlah fungsi `loginUsingHttp()` pada `LoginPage`. Tambahkan import `../service/http_service.dart`, lalu tambahkan fungsi berikut dan tombol kedua di bawah tombol LOGIN:

```dart
Future<void> loginUsingHttp() async {
  if (_email.text.trim().isEmpty || _password.text.isEmpty) {
    _toast('Email dan password harus diisi');
    return;
  }
  setState(() => _loading = true);
  try {
    final res = await HttpService.login(_email.text.trim(), _password.text);
    if (!mounted) return;
```

```
    if (res.code == 200) {
      Navigator.pushReplacement(
          context, MaterialPageRoute(builder: (_) => const HomePage()));
    } else {
      _toast(res.status);
    }
  } catch (e) {
    _toast('Gagal terhubung ke server: $e');
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}
```

DART
```
OutlinedButton(
  onPressed: _loading ? null : loginUsingHttp,
  child: const Text('LOGIN (http)'),
),
```

5. Buatlah fungsi `submitByHttp()` pada `AddUserPage` (tambahkan import `../service/http_service.dart`):

DART
```
Future<void> submitByHttp() async {
  if (!_formKey.currentState!.validate()) return;
  try {
    final res = await HttpService.register(
        _nama.text.trim(), _email.text.trim(), _password.text);
    if (!mounted) return;
    _toast(res.status);
    if (res.code == 200) Navigator.pop(context, true);
  } catch (e) {
    _toast('Gagal terhubung ke server: $e');
  }
}
```

Tambahkan tombol kedua di bawah tombol SUBMIT (Dio):

DART
```
OutlinedButton(
  onPressed: submitByHttp,
  child: const Text('SUBMIT (http)'),
),
```

6. Jalankan aplikasi, lalu bandingkan proses login dan registrasi menggunakan Dio dan http.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code http dan UI-nya.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa memahami serta menerapkan konsep http (padanan Volley), dengan implementasi source code pada kasus tertentu.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara detail setiap baris code http	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code serta menjelaskan secara singkat setiap baris code http	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code serta menjelaskan setiap baris code http	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code serta menjelaskan setiap baris code http	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 43 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST

Pokok Bahasan	:	Getting Data
Acara Praktikum/Praktik	:	Minggu 12 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu membuat program menyiapkan API.
2. Mahasiswa mampu membuat program untuk parsing data ke Flutter.
3. Mahasiswa mampu membuat program untuk menampilkan data dalam bentuk list view yang sudah di-custom.

b. Indikator Penilaian

1. Keberhasilan mahasiswa menyiapkan API untuk membuat program parsing data ke Flutter.
2. Keberhasilan mahasiswa membuat program untuk menampilkan data dalam bentuk list view yang sudah di-custom.

c. Dasar Teori

Getting Data

GET merupakan salah satu dari 4 metode HTTP Request yang umum digunakan. Metode ini digunakan untuk membaca data (*read only*) pada *resource*. Pada praktikum ini akan diimplementasikan GET menggunakan Dio dan http pada project `retrovolley` yang telah dibuat pada pertemuan sebelumnya.

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

Menyiapkan API dan Koneksi Dio serta http

1. Model dibuat menyesuaikan dengan response yang diberikan oleh server, oleh sebab itu perlu mengetahui format response. Format response GET `User_Registration.php` (Acara 41) adalah sebagai berikut:

```JSON
{
  "code": 200,
  "status": "success",
  "user_list": [
    { "id": 1, "user_fullname": "Andi Pratama", "user_email": "andi@polije.ac.id" },
    { "id": 2, "user_fullname": "Budi Santoso", "user_email": "budi@polije.ac.id" }
  ]
```

```
}
```

2. Kelas `User` berisi variabel `id`, `user_email`, dan `user_fullname` sudah dibuat pada `lib/model/user.dart`. Password tidak ikut dikirim oleh server demi keamanan.
3. Kelas `ResponseUser` (code, status, dan daftar user) dan `ResponseModel` (code dan status saja) sudah dibuat pada `lib/model/response_model.dart`.
4. Tambahkan fungsi GET pada `lib/service/api_service.dart` (setara menambahkan `@GET("User_Registration.php")` pada interface `MethodHTTP`):

DART
```
// Tambahkan di dalam class ApiService
static Future<ResponseUser> getUsers() async {
  final res = await _dio.get('User_Registration.php');
  return ResponseUser.fromJson(_map(res.data));
}
```

5. Tambahkan fungsi GET pada `lib/service/http_service.dart`:

DART
```
// Tambahkan di dalam class HttpService
static Future<ResponseUser> getUsers() async {
  final res = await http
      .get(_url('User_Registration.php'))
      .timeout(const Duration(seconds: 10));
  return ResponseUser.fromJson(jsonDecode(res.body));
}
```

6. BASE URL disimpan pada kelas `GlobalVariable` (Acara 41) sehingga dapat diubah dari dialog BASE URL.

Membuat Widget Item (setara Adapter Model)

Tampilan item dibuat agar sesuai kebutuhan. Dalam hal ini, user akan ditampilkan dalam bentuk list; tiap item berisi data nama user, email, dan ID user.

1. Buat file `lib/pages/user_item.dart` (setara `item_user.xml`, `cardview_user.xml`, dan `UserAdapter`):

DART
```
import 'package:flutter/material.dart';
import '../model/user.dart';

class UserItem extends StatelessWidget {
  final User user;
  final VoidCallback? onTap;
  const UserItem({super.key, required this.user, this.onTap});

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
      child: ListTile(
        leading: CircleAvatar(child: Text('${user.id}')),
        title: Text(user.userFullname,
            style: const TextStyle(fontWeight: FontWeight.bold)),
        subtitle: Text(user.userEmail),
        trailing: const Icon(Icons.edit),
        onTap: onTap,
      ),
    );
  }
}
```

2. Perhatikan nama properti yang ditampilkan (`id`, `userFullname`, `userEmail`) harus sesuai dengan model.

Implementasi GET pada Dio

1. Lengkapi `lib/pages/dio_page.dart`. Fungsi `getUserFromAPI()` dipanggil saat halaman dibuka (setara `onCreate`).
2. Proses berpindah dari DioPage ke AddUserPage ditambahkan melalui ikon *add* pada AppBar (setara menu `retrofit.xml` + `onCreateOptionsMenu` dan `onOptionsItemSelected`).
3. Ikon diambil dari `Icons.add` dan `Icons.refresh`; tidak perlu membuat folder menu.

DART
```
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import '../model/user.dart';
import '../service/api_service.dart';
import 'add_user_page.dart';
import 'update_user_page.dart';
import 'user_item.dart';

class DioPage extends StatefulWidget {
  const DioPage({super.key});

  @override
  State<DioPage> createState() => _DioPageState();
}

class _DioPageState extends State<DioPage> {
  List<User> _users = [];
  bool _loading = true;
  String? _error;

  @override
  void initState() {
    super.initState();
    getUserFromAPI();
  }

  Future<void> getUserFromAPI() async {
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      final res = await ApiService.getUsers();
      if (!mounted) return;
      setState(() => _users = res.userList);
    } on DioException catch (e) {
      if (mounted) setState(() => _error = 'Gagal memuat data: ${e.message}');
    } catch (e) {
      if (mounted) setState(() => _error = 'Terjadi kesalahan: $e');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _buka(Widget halaman) async {
    final berubah = await Navigator.push<bool>(
        context, MaterialPageRoute(builder: (_) => halaman));
    if (berubah == true) getUserFromAPI();
  }

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Dio - GET'),
        actions: [
          IconButton(
            icon: const Icon(Icons.add),
            onPressed: () => _buka(const AddUserPage()),
          ),
          IconButton(
            icon: const Icon(Icons.refresh),
            onPressed: getUserFromAPI,
          ),
        ],
      ),
      body: _loading
          ? const Center(child: CircularProgressIndicator())
          : _error != null
              ? Center(child: Text(_error!))
              : RefreshIndicator(
                  onRefresh: getUserFromAPI,
                  child: ListView.builder(
                    itemCount: _users.length,
                    itemBuilder: (_, i) => UserItem(
                      user: _users[i],
                      onTap: () => _buka(UpdateUserPage(user: _users[i])),
                    ),
                  ),
                ),
    );
  }
}
```

4. Halaman `UpdateUserPage` dibuat pada Acara 44. Sementara itu, buat kerangka `lib/pages/update_user_page.dart` berikut agar kode dapat dijalankan:

DART
```
import 'package:flutter/material.dart';
import '../model/user.dart';

class UpdateUserPage extends StatelessWidget {
  final User user;
  const UpdateUserPage({super.key, required this.user});

  @override
  Widget build(BuildContext context) =>
      Scaffold(appBar: AppBar(title: Text(user.userFullname)));
}
```

Implementasi GET pada http

1. Lengkapi `lib/pages/http_page.dart` dengan fungsi request data dari server dengan metode HTTP GET.
2. Panggil fungsi tersebut pada `initState()` (setara `onCreate`).
3. Panggil `getUserFromAPI()` pada aksi *refresh* agar data baru dapat ditampilkan, dan pada aksi *close* berikan perintah untuk menutup halaman.

DART
```
import 'package:flutter/material.dart';
import '../model/user.dart';
import '../service/http_service.dart';
import 'update_user_page.dart';
import 'user_item.dart';

class HttpPage extends StatefulWidget {
  const HttpPage({super.key});
```

```
  @override
  State<HttpPage> createState() => _HttpPageState();
}

class _HttpPageState extends State<HttpPage> {
  late Future<List<User>> _users = getUserFromAPI();

  Future<List<User>> getUserFromAPI() async {
    final res = await HttpService.getUsers();
    return res.userList;
  }

  void actionRefresh() => setState(() => _users = getUserFromAPI());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('http - GET'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh), onPressed: actionRefresh),
          IconButton(
            icon: const Icon(Icons.close),
            onPressed: () => Navigator.pop(context), // actionClose
          ),
        ],
      ),
      body: FutureBuilder<List<User>>(
        future: _users,
        builder: (context, snapshot) {
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Gagal memuat data: ${snapshot.error}'));
          }
          final users = snapshot.data!;
          return ListView.builder(
            itemCount: users.length,
            itemBuilder: (_, i) => UserItem(
              user: users[i],
              onTap: () async {
                final berubah = await Navigator.push<bool>(
                  context,
                  MaterialPageRoute(
                      builder: (_) => UpdateUserPage(user: users[i])),
                );
                if (berubah == true) actionRefresh();
              },
            ),
          );
        },
      ),
    );
  }
}
```

4. Jalankan aplikasi, login, lalu buka menu Dio dan http. Tambahkan beberapa user dan amati data yang tampil.

f. Hasil dan Pembahasan

1. API untuk parsing data.
2. Project aplikasi dari percobaan implementasi code program untuk parsing data ke Flutter dan menampilkan data dalam bentuk list view yang sudah di-custom.

3. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

1. Mahasiswa mampu membuat program menyiapkan API.
2. Mahasiswa mampu membuat program untuk menampilkan data dalam bentuk list view yang sudah di-custom.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam menyiapkan API, implementasi code untuk menampilkan data dalam bentuk list view, serta menjelaskan secara detail setiap baris code	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam menyiapkan API, implementasi code untuk menampilkan data dalam bentuk list view, serta menjelaskan singkat setiap baris code	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam menyiapkan API, implementasi code untuk menampilkan data dalam bentuk list view, serta penjelasannya	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan menyiapkan API, implementasi code untuk menampilkan data dalam bentuk list view, tanpa penjelasan	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 44 : PENGENALAN DIO (RETROFIT), HTTP (VOLLEY), GET, POST

Pokok Bahasan	:	Sending Data (Post)
Acara Praktikum/Praktik	:	Minggu 12 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep sending data (POST).
2. Mahasiswa mampu menerapkan sending data (POST) pada Flutter.

b. Indikator Penilaian

Keberhasilan mahasiswa memahami serta menerapkan konsep sending data (POST) dengan Dio dan http, dengan implementasi source code pada kasus tertentu.

c. Dasar Teori

Sending Data

Sending data menggunakan metode **POST** yang digunakan untuk mengirimkan atau membuat *resource* baru pada server. POST merupakan salah satu dari 4 metode HTTP Request yang umum diimplementasikan. Pada praktikum ini akan diimplementasikan POST menggunakan Dio dan http pada project `retrovolley`, dilanjutkan dengan operasi **Update** dan **Delete**.

Data POST dapat dikirim dalam beberapa format:

- `application/x-www-form-urlencoded` — pasangan key=value, dibaca PHP melalui `$_POST` (dipakai pada praktikum ini).
- `application/json` — body berupa teks JSON (`jsonEncode(map)`), dibaca PHP dengan `json_decode(file_get_contents('php://input'))`.
- `multipart/form-data` — untuk mengirim file/gambar (`FormData` pada Dio, `MultipartRequest` pada http).

d. Alat dan Bahan

- Kabel Data USB
- Jaringan Internet

e. Prosedur Kerja

- POST merupakan metode HTTP yang digunakan untuk mengirimkan data ke web server.
- Kemudian, web server menyimpan data ke DBMS sehingga hasilnya dapat ditampilkan menggunakan metode GET yang sudah dijelaskan sebelumnya.

Implementasi POST pada Dio

1. Kembali pada `AddUserPage`.

2. Fungsi `submitByDio()` (Acara 41) mengirim data dengan `_dio.post()` di dalam `ApiService.register()`. Perhatikan bahwa `BaseOptions.contentType` diatur ke `Headers.formUrlEncodedContentType` sehingga Map dikirim sebagai form. Setelah berhasil, halaman ditutup dengan `Navigator.pop(context, true)` sehingga DioPage memuat ulang daftar user.

Implementasi POST pada http

POST diimplementasikan dengan cara mengirimkan data user ke web server menggunakan `AddUserPage`. Jika sebelumnya menggunakan Dio, kali ini menggunakan package http melalui fungsi `submitByHttp()` (Acara 42) yang memanggil `http.post(url, body: {...})`.

Implementasi Update dan Delete

1. Tambahkan dua file PHP pada folder `htdocs/retrovolley/`:

PHP
```
<?php
// User_Update.php
require 'koneksi.php';
$id    = (int) ($_POST['id'] ?? 0);
$nama  = $_POST['user_fullname'] ?? '';
$email = $_POST['user_email'] ?? '';
$pass  = $_POST['user_password'] ?? '';

if ($pass !== '') {
    $hash = password_hash($pass, PASSWORD_DEFAULT);
    $stmt = $conn->prepare('UPDATE users SET user_fullname=?, user_email=?, '
        . 'user_password=? WHERE id=?');
    $stmt->bind_param('sssi', $nama, $email, $hash, $id);
} else {
    $stmt = $conn->prepare(
        'UPDATE users SET user_fullname=?, user_email=? WHERE id=?');
    $stmt->bind_param('ssi', $nama, $email, $id);
}
$ok = $stmt->execute();
echo json_encode([
    'code' => $ok ? 200 : 500,
    'status' => $ok ? 'Update berhasil' : 'Update gagal',
]);
```

PHP
```
<?php
// User_Delete.php
require 'koneksi.php';
$id = (int) ($_POST['id'] ?? 0);
$stmt = $conn->prepare('DELETE FROM users WHERE id=?');
$stmt->bind_param('i', $id);
$ok = $stmt->execute();
echo json_encode([
    'code' => $ok ? 200 : 500,
    'status' => $ok ? 'Data dihapus' : 'Hapus gagal',
]);
```

2. Tambahkan fungsi berikut di dalam kelas `ApiService` (Dio):

DART
```
static Future<ResponseModel> updateUser(
    int id, String fullname, String email, String password) async {
  final res = await _dio.post('User_Update.php', data: {
    'id': id,
    'user_fullname': fullname,
    'user_email': email,
    'user_password': password,
  });
```

```
  return ResponseModel.fromJson(_map(res.data));
}

static Future<ResponseModel> deleteUser(int id) async {
  final res = await _dio.post('User_Delete.php', data: {'id': id});
  return ResponseModel.fromJson(_map(res.data));
}
```

3. Tambahkan fungsi yang sama di dalam kelas `HttpService` (http). Perhatikan bahwa pada package http seluruh nilai body harus bertipe `String`:

DART
```
static Future<ResponseModel> updateUser(
    int id, String fullname, String email, String password) async {
  final res = await http.post(_url('User_Update.php'), body: {
    'id': '$id',
    'user_fullname': fullname,
    'user_email': email,
    'user_password': password,
  });
  return ResponseModel.fromJson(jsonDecode(res.body));
}

static Future<ResponseModel> deleteUser(int id) async {
  final res = await http.post(_url('User_Delete.php'), body: {'id': '$id'});
  return ResponseModel.fromJson(jsonDecode(res.body));
}
```

4. Buat halaman `UpdateUserPage`:

- Buat 3 kolom input: Nama Lengkap, Email, dan Password (password boleh dikosongkan bila tidak diganti).
- ID tidak dapat diubah karena *auto increment* oleh DBMS; ID dikirim dari halaman daftar user.
- Tambahkan tombol untuk Update dan Delete data pada web server.

Ganti isi `lib/pages/update_user_page.dart`:

DART
```
import 'package:flutter/material.dart';
import '../model/response_model.dart';
import '../model/user.dart';
import '../service/api_service.dart';
import '../service/http_service.dart';

class UpdateUserPage extends StatefulWidget {
  final User user;
  const UpdateUserPage({super.key, required this.user});

  @override
  State<UpdateUserPage> createState() => _UpdateUserPageState();
}

class _UpdateUserPageState extends State<UpdateUserPage> {
  late final _nama = TextEditingController(text: widget.user.userFullname);
  late final _email = TextEditingController(text: widget.user.userEmail);
  final _password = TextEditingController();
  bool _pakaiDio = true; // pilih library: Dio atau http

  @override
  void dispose() {
    _nama.dispose();
    _email.dispose();
    _password.dispose();
```

```
    super.dispose();
  }

  Future<void> _jalankan(Future<ResponseModel> Function() aksi) async {
    final messenger = ScaffoldMessenger.of(context);
    final navigator = Navigator.of(context);
    try {
      final res = await aksi();
      messenger.showSnackBar(SnackBar(content: Text(res.status)));
      if (res.code == 200) navigator.pop(true);
    } catch (e) {
      messenger.showSnackBar(
          SnackBar(content: Text('Gagal terhubung ke server: $e')));
    }
  }

  void _update() {
    final id = widget.user.id;
    final nama = _nama.text.trim();
    final email = _email.text.trim();
    final pass = _password.text;
    _jalankan(() => _pakaiDio
        ? ApiService.updateUser(id, nama, email, pass)
        : HttpService.updateUser(id, nama, email, pass));
  }

  Future<void> _delete() async {
    final yakin = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Hapus User'),
        content: Text('Hapus ${widget.user.userFullname}?'),
        actions: [
          TextButton(
              onPressed: () => Navigator.pop(ctx, false),
              child: const Text('BATAL')),
          FilledButton(
              onPressed: () => Navigator.pop(ctx, true),
              child: const Text('HAPUS')),
        ],
      ),
    );
    if (yakin != true) return;
    final id = widget.user.id;
    _jalankan(() => _pakaiDio
        ? ApiService.deleteUser(id)
        : HttpService.deleteUser(id));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Update User #${widget.user.id}')),
      body: ListView(
        padding: const EdgeInsets.all(24),
        children: [
          SegmentedButton<bool>(
            segments: const [
              ButtonSegment(value: true, label: Text('Dio')),
              ButtonSegment(value: false, label: Text('http')),
            ],
            selected: {_pakaiDio},
            onSelectionChanged: (s) => setState(() => _pakaiDio = s.first),
          ),
          const SizedBox(height: 16),
          TextField(
              controller: _nama,
              decoration: const InputDecoration(labelText: 'Nama Lengkap')),
          TextField(
              controller: _email,
              decoration: const InputDecoration(labelText: 'Email')),
```

```
          TextField(
              controller: _password,
              obscureText: true,
              decoration: const InputDecoration(
                  labelText: 'Password baru (kosongkan bila tetap)')),
          const SizedBox(height: 24),
          FilledButton.icon(
            onPressed: _update,
            icon: const Icon(Icons.save),
            label: const Text('UPDATE'),
          ),
          const SizedBox(height: 8),
          OutlinedButton.icon(
            onPressed: _delete,
            icon: const Icon(Icons.delete),
            label: const Text('DELETE'),
          ),
        ],
      ),
    );
  }
}
```

5. Build dan run project tersebut, kemudian analisa hasilnya: tambah user (POST), tampilkan daftar (GET), ubah data (Update), dan hapus data (Delete) menggunakan Dio maupun http.

f. Hasil dan Pembahasan

1. Project aplikasi dari percobaan implementasi code program sending data (POST), update, dan delete.
2. Dokumentasi hasil output aplikasi dalam bentuk laporan dilengkapi dengan analisa code pada implementasi source code.

g. Kesimpulan

Mahasiswa memahami konsep sending data (POST) dan menerapkannya pada Flutter.

h. Rubrik Penilaian

No	Indikator	Skor			
1	Ketepatan waktu dan ketepatan dalam implementasi code program sending data (post), serta menjelaskan secara detail setiap baris code	1	2	3	4
2	Ketepatan waktu dan ketepatan dalam implementasi code program sending data (post), serta penjelasan singkat setiap baris code	1	2	3	4
3	Ketepatan waktu akan tetapi kurang tepat dalam implementasi code program sending data (post), serta penjelasan setiap baris code	1	2	3	4
4	Keterlambatan pengumpulan tugas dan ketidaktepatan dalam implementasi code program sending data (post), serta tanpa penjelasan	1	2	3	4
Jumlah Skor					

Keterangan skor: 4 = sangat baik, 3 = baik, 2 = cukup, 1 = kurang.

Acara 45 : PROJECT FINAL

Pokok Bahasan	:	Laporan Studi Kasus
Acara Praktikum/Praktik	:	Minggu 13 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu menemukan studi kasus yang akan diselesaikan.
2. Mahasiswa mampu mengumpulkan informasi.
3. Mahasiswa mampu menemukan masalah inti dari studi kasus.
4. Mahasiswa mampu menemukan ide untuk solusi dari permasalahan.

b. Indikator Penilaian

1. Keberhasilan menemukan studi kasus yang akan diselesaikan.
2. Keberhasilan mahasiswa mengumpulkan informasi dan menemukan masalah inti dari studi kasus.
3. Keberhasilan menemukan ide untuk solusi dari permasalahan.

c. Dasar Teori

Design Thinking

Design Thinking adalah proses berulang di mana kita berusaha memahami pengguna, menantang asumsi, dan mendefinisikan kembali masalah dalam upaya mengidentifikasi strategi dan solusi alternatif yang mungkin tidak langsung terlihat dengan tingkat awal pemahaman kita. Pada saat yang sama, Design Thinking menyediakan pendekatan berbasis solusi untuk menyelesaikan masalah. Ini adalah cara berpikir dan bekerja serta kumpulan metode langsung.

Design Thinking berputar di sekitar minat yang mendalam dalam mengembangkan pemahaman tentang orang-orang yang menjadi tujuan perancangan produk atau layanan. Hal ini membantu kita mengamati dan mengembangkan empati dengan target pengguna. Design Thinking membantu kita dalam proses bertanya: mempertanyakan masalah, mempertanyakan asumsi, dan mempertanyakan keterkaitannya.

Design Thinking sangat berguna dalam mengatasi masalah-masalah yang tidak jelas atau tidak dikenal, dengan melakukan *reframing* masalah dengan cara-cara yang berpusat pada manusia, menciptakan banyak ide dalam *brainstorming*, dan mengadopsi pendekatan langsung dalam pembuatan prototype dan testing. Design Thinking juga melibatkan eksperimen yang sedang berjalan: membuat sketsa, membuat prototype, testing, dan mencoba berbagai konsep dan ide.

Tahapan dalam Proses Design Thinking

Empathise

Tahap pertama dari proses Design Thinking adalah mendapatkan pemahaman empatik tentang masalah yang dicoba untuk diselesaikan. Ini melibatkan konsultasi dengan para ahli untuk mencari tahu lebih banyak tentang bidang yang menjadi perhatian melalui pengamatan, keterlibatan, dan empati dengan orang-orang untuk memahami pengalaman dan motivasi mereka. Empati memungkinkan pemikir desain untuk mengesampingkan asumsi mereka sendiri tentang dunia untuk mendapatkan wawasan tentang pengguna dan kebutuhan mereka.

Define

Selama tahap Define, kita mengumpulkan informasi yang telah dibuat dan dikumpulkan selama tahap Empathise. Di sinilah kita menganalisis pengamatan dan menyintesisnya untuk menentukan masalah inti yang telah diidentifikasi. Kita harus berusaha mengidentifikasi masalah sebagai pernyataan masalah dengan cara yang berpusat pada manusia. Sebagai ilustrasi, alih-alih mengidentifikasi masalah sebagai keinginan perusahaan seperti "Kita perlu meningkatkan pangsa pasar produk makanan di antara remaja perempuan sebesar 5%", cara yang lebih baik untuk mendefinisikan masalah adalah "Remaja perempuan perlu makan makanan bergizi agar dapat berkembang, menjadi sehat dan tumbuh."

Tahap Define akan membantu para desainer dalam sebuah tim untuk mengumpulkan ide-ide hebat untuk membangun fitur, fungsi, dan elemen lain yang memungkinkan mereka menyelesaikan masalah, atau paling tidak memungkinkan pengguna menyelesaikan masalah sendiri dengan tingkat kesulitan minimal.

Ideate

Selama tahap ketiga dari proses Design Thinking, desainer siap untuk mulai menghasilkan ide. Kita telah memahami pengguna dan kebutuhan mereka di tahap Empathise, telah menganalisis dan menyintesis pengamatan di tahap Define, dan berakhir dengan pernyataan masalah yang berpusat pada manusia. Dengan latar belakang yang kuat, kita dan anggota tim dapat mulai "berpikir di luar kotak" untuk mengidentifikasi solusi baru.

Ada ratusan teknik Ideation seperti Brainstorm, Brainwrite, Worst Possible Idea, dan SCAMPER. Sesi Brainstorm dan Worst Possible Idea biasanya digunakan untuk merangsang pemikiran bebas dan memperluas ruang masalah. Penting untuk mendapatkan sebanyak mungkin ide atau solusi masalah, lalu memilih beberapa teknik Ideation lainnya pada akhir fase untuk menyelidiki dan menguji ide-ide tersebut sehingga ditemukan cara terbaik untuk memecahkan masalah.

Sumber: https://sis.binus.ac.id/2020/03/17/design-thinking-pengertian-tahapan-dan-contoh-penerapannya/

d. Alat dan Bahan

- Kertas A4
- Kertas Folio

- Sticky Note
- Spidol

e. Prosedur Kerja

1. Mahasiswa berdiskusi dengan kelompok untuk mencari studi kasus di lingkungan sekitar, menggali informasi, serta menemukan inti masalah dari studi kasus.
2. Dari studi kasus yang sudah ditemukan, setiap kelompok menganalisa permasalahan inti yang terdapat dalam studi kasus.
3. Setiap kelompok menyusun pertanyaan untuk pengumpulan data.
4. Setiap kelompok mempresentasikan di depan kelas.
5. Kelompok lain mengoreksi dan mengomentari hasil ide yang dikemukakan.

f. Hasil dan Pembahasan

1. Dokumen hasil diskusi kelompok berupa beberapa studi kasus yang diajukan setiap anggota kelompok beserta alasannya sampai dengan studi kasus yang dipilih, serta beberapa pertanyaan yang dipersiapkan untuk pengumpulan data.
2. Dokumen laporan untuk disusun dalam laporan akhir.

g. Kesimpulan

Mahasiswa menemukan studi kasus yang akan diselesaikan dari hasil diskusi dalam 1 kelompok, dan susunan pertanyaan yang dipersiapkan untuk pengumpulan data.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 46 : PROJECT FINAL

Pokok Bahasan	:	Laporan Hasil Observasi
Acara Praktikum/Praktik	:	Minggu 13 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu menemukan studi kasus yang akan diselesaikan.
2. Mahasiswa mampu mengumpulkan informasi.
3. Mahasiswa mampu menemukan masalah inti dari studi kasus.
4. Mahasiswa mampu menemukan ide untuk solusi dari permasalahan.

b. Indikator Penilaian

1. Keberhasilan menemukan studi kasus yang akan diselesaikan.
2. Keberhasilan mahasiswa mengumpulkan informasi dan menemukan masalah inti dari studi kasus.
3. Keberhasilan menemukan ide untuk solusi dari permasalahan.

c. Dasar Teori

Proses Bisnis

Definisi proses bisnis menurut ahli:

1) Weske (2007): proses bisnis adalah serangkaian instrumen untuk mengorganisir suatu kegiatan dan untuk meningkatkan pemahaman atas keterkaitan suatu kegiatan.
2) Sparx Systems (2004): sekumpulan kegiatan atau aktivitas yang dirancang untuk menghasilkan suatu keluaran tertentu bagi pelanggan tertentu.
3) Hammer dan Champy dalam Weske (2007): proses bisnis adalah sekumpulan kegiatan yang mengambil satu atau banyak masukan dan menciptakan sebuah keluaran yang berguna bagi pelanggan.

Contoh:

- Penerimaan Karyawan oleh HRD
- Penjualan Toko oleh Kasir
- Pendaftaran Mahasiswa oleh Staf Administrasi
- Peminjaman Buku oleh Mahasiswa

Detail contoh — proses bisnis peminjaman buku dapat diuraikan sebagai berikut: memasuki perpustakaan → menunjukkan ID member → memilih buku → membawa buku ke pustakawan → memasukkan ID member dan kode buku → melakukan transaksi peminjaman.

Teknik Pengumpulan Data

1) **Wawancara**: teknik pengumpulan data dengan mengadakan komunikasi dengan sumber data melalui dialog (tanya jawab) secara lisan, baik langsung maupun tidak langsung.
2) **Observasi**: pengamatan langsung pada lingkungan dan/atau pengamatan langsung suatu kegiatan yang akan dibuat sistemnya.
3) **Kuesioner dan Literatur**: memberikan kuesioner kepada pengguna yang telah dipilih, sedangkan literatur dilakukan dengan mencari berbagai sumber tertulis (buku, arsip, majalah, artikel, jurnal, atau dokumen yang relevan dengan permasalahan yang dikaji). Informasi yang didapat dijadikan rujukan untuk memperkuat sistem yang akan dibuat.

d. Alat dan Bahan

- Kertas A4
- Kertas Folio
- Sticky Note
- Spidol

e. Prosedur Kerja

Seluruh anggota kelompok melakukan pengambilan data dan menggali informasi kepada objek, dalam bentuk: (1) observasi; (2) wawancara; (3) kuesioner.

1. Diskusikan dengan masing-masing kelompok hasil pengumpulan data.
2. Uraikan setiap informasi yang didapatkan dengan bahasa hasil analisa.
3. Uraikan proses bisnis dari objek studi kasus.
4. Temukan beberapa masalah yang urgent dan butuh penyelesaian.
5. Tentukan masalah yang akan diberikan solusi berupa aplikasi mobile berbasis Flutter dari hasil diskusi analisa masalah.

f. Hasil dan Pembahasan

1. Dokumen hasil pengumpulan data proses bisnis, permasalahan yang terdapat dalam objek, serta solusi yang diberikan terhadap masalah yang urgent.
2. Dokumen laporan untuk disusun dalam laporan akhir.

g. Kesimpulan

Mahasiswa menemukan studi kasus yang akan diselesaikan dari hasil diskusi dalam 1 kelompok.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %

No	Aspek Penilaian	Bobot
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 47 : PROJECT FINAL

Pokok Bahasan	:	Laporan Prototype
Acara Praktikum/Praktik	:	Minggu 13 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu merancang prototype untuk project yang akan dikerjakan.
2. Mahasiswa mampu melakukan validasi terhadap prototype.

b. Indikator Penilaian

Keberhasilan mahasiswa merancang prototype untuk project dan melakukan validasi.

c. Dasar Teori

Prototype

Tim desain akan menghasilkan sejumlah versi produk yang murah dan diperkecil, atau fitur spesifik yang ditemukan dalam produk, sehingga mereka dapat menyelidiki solusi masalah yang dihasilkan pada tahap sebelumnya. Prototype dapat dibagikan dan diuji dalam tim itu sendiri, di departemen lain, atau pada sekelompok kecil orang di luar tim desain. Ini adalah fase eksperimental, dan tujuannya adalah mengidentifikasi solusi terbaik untuk setiap masalah yang diidentifikasi selama tiga tahap pertama. Solusi diimplementasikan dalam prototype, dan satu per satu diselidiki dan diterima, diperbaiki dan diperiksa ulang, atau ditolak berdasarkan pengalaman pengguna.

Pada akhir tahap ini, tim desain akan memiliki gagasan yang lebih baik tentang kendala yang melekat pada produk dan masalah yang ada, serta pandangan yang lebih jelas tentang bagaimana pengguna sebenarnya akan berperilaku, berpikir, dan merasa ketika berinteraksi dengan produk akhir.

Prototype aplikasi Flutter dapat dibuat dengan perangkat lunak desain UI seperti **Figma**, kemudian diterjemahkan ke dalam widget Flutter. Material 3 Design Kit pada Figma membantu agar desain sesuai dengan widget Material yang tersedia di Flutter.

Test

Desainer menguji produk lengkap secara ketat menggunakan solusi terbaik yang diidentifikasi selama fase prototyping. Ini adalah tahap akhir dari design thinking, tetapi dalam proses berulang, hasil yang dihasilkan selama fase testing sering digunakan untuk mendefinisikan kembali satu atau lebih masalah dan menginformasikan pemahaman pengguna, kondisi penggunaan, bagaimana orang berpikir, berperilaku, merasakan, dan berempati. Bahkan selama fase ini, perubahan dan penyempurnaan dilakukan untuk menyingkirkan solusi masalah dan memperoleh pemahaman sedalam mungkin terhadap produk dan penggunanya.

Sumber: https://sis.binus.ac.id/2020/03/17/design-thinking-pengertian-tahapan-dan-contoh-penerapannya/

d. Alat dan Bahan

- Kertas A4
- Kertas Folio
- Sticky Note
- Spidol

e. Prosedur Kerja

1. Berdiskusi dengan masing-masing kelompok untuk merancang prototype project.
2. Membuat sketsa prototype untuk setiap halaman (form) project yang akan dikerjakan.
3. Implementasikan sketsa ke dalam software (misalnya Figma) untuk menghasilkan prototype yang mudah dipahami.

f. Hasil dan Pembahasan

1. Dokumen hasil pembuatan prototype project akhir menggunakan software yang telah dipilih.
2. Dokumen laporan untuk disusun dalam laporan akhir.

g. Kesimpulan

Mahasiswa dalam 1 kelompok menyelesaikan prototype project akhir yang telah dikerjakan.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 48 : PROJECT FINAL

Pokok Bahasan	:	Laporan Validasi
Acara Praktikum/Praktik	:	Minggu 13 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu melakukan validasi prototype project yang telah dikerjakan.
2. Mahasiswa mampu memperbaiki hasil validasi prototype project.

b. Indikator Penilaian

Keberhasilan mahasiswa melakukan validasi prototype project dan memperbaikinya.

c. Dasar Teori

Prototype

Prototype adalah versi produk yang murah dan diperkecil untuk menyelidiki solusi masalah yang dihasilkan pada tahap sebelumnya. Solusi diimplementasikan dalam prototype, dan satu per satu diselidiki dan diterima, diperbaiki dan diperiksa ulang, atau ditolak berdasarkan pengalaman pengguna. Pada akhir tahap ini, tim desain memiliki gagasan yang lebih baik tentang kendala produk dan bagaimana pengguna akan berperilaku ketika berinteraksi dengan produk akhir.

Test

Desainer menguji produk lengkap secara ketat menggunakan solusi terbaik yang diidentifikasi selama fase prototyping. Dalam proses berulang, hasil fase testing sering digunakan untuk mendefinisikan kembali satu atau lebih masalah, sehingga perubahan dan penyempurnaan tetap dilakukan untuk memperoleh pemahaman sedalam mungkin terhadap produk dan penggunanya.

Sumber: https://sis.binus.ac.id/2020/03/17/design-thinking-pengertian-tahapan-dan-contoh-penerapannya/

d. Alat dan Bahan

- Kertas A4
- Kertas Folio
- Sticky Note
- Spidol

e. Prosedur Kerja

1. Diskusi dengan anggota kelompok untuk melakukan validasi prototype.
2. Melakukan validasi kepada objek studi kasus.

3. Diskusi hasil validasi dan memperbaiki prototype jika terdapat ketidaksesuaian dengan keinginan objek.
4. Mempresentasikan di depan kelas hasil prototype dan hasil perbaikannya.

f. Hasil dan Pembahasan

Dokumentasi hasil validasi prototype dan hasil perbaikan prototype untuk disusun dalam laporan akhir.

g. Kesimpulan

Mahasiswa berhasil melakukan validasi dan perbaikan prototype.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 49 : PROJECT FINAL

Pokok Bahasan	:	Laporan Use Case Diagram
Acara Praktikum/Praktik	:	Minggu 14 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu membuat use case secara kelompok dari studi kasus project akhir.
2. Mahasiswa mampu merancang database secara kelompok dari studi kasus project akhir.

b. Indikator Penilaian

Keberhasilan mahasiswa membuat use case, merancang dan menyusun database secara berkelompok dari studi kasus project akhir.

c. Dasar Teori

UML Diagram: Use Case Diagram

Use case diagram merupakan diagram yang menggambarkan hubungan antara aktor dengan sistem. Use case diagram dapat mendeskripsikan interaksi antara satu atau lebih aktor dengan sistem yang akan dibuat, digunakan untuk mengetahui fungsi apa saja yang ada di dalam sebuah sistem, dan merepresentasikan interaksi aktor dengan sistem. Dengan demikian, use case dapat dipresentasikan dengan urutan yang sederhana dan mudah dipahami oleh konsumen. Manfaat use case adalah memudahkan komunikasi dengan *domain expert* dan *end user*, serta memberikan kepastian pemahaman tentang *requirement* atau kebutuhan sebuah sistem.

Use case diagram mempunyai 3 komponen, yaitu:

- **Sistem** — menyatakan batasan sistem dalam relasi dengan aktor-aktor yang menggunakannya (di luar sistem) dan fitur-fitur yang harus disediakan (dalam sistem).
- **Aktor** — segala hal di luar sistem yang akan menggunakan sistem tersebut untuk melakukan sesuatu. Bisa berupa manusia, sistem, atau perangkat yang memiliki peranan dalam keberhasilan operasi sistem.
- **Use Case** — gambaran fungsional dari sebuah sistem, sehingga konsumen dan pengguna sistem memahami fungsi sistem yang tengah dibangun.

Use case diagram juga mempunyai beberapa relasi, yaitu:

- **Association** — mengidentifikasi interaksi yang dilakukan oleh aktor tertentu dengan use case tertentu, digambarkan dengan garis antara aktor dan use case.
- **Generalization** — mendefinisikan relasi antara dua aktor atau dua use case di mana salah satunya mewarisi (*inherit*) dan menambahkan atau meng-*override* sifat dari yang lainnya.
- **Dependency** — terbagi menjadi 2 macam: **include** (use case yang satu memanggil use case yang lain) dan **extend** (pemanggilan hanya terjadi pada kondisi tertentu).

Sumber: https://socs.binus.ac.id/2019/11/26/uml-diagram-use-case-diagram/

d. Alat dan Bahan

- Kertas A4
- Kertas Folio
- Sticky Note
- Spidol

e. Prosedur Kerja

Diskusikan dengan kelompok untuk membuat use case diagram project akhir, dan mulai menyusunnya ke dalam laporan akhir.

f. Hasil dan Pembahasan

Dokumentasi use case diagram untuk disusun dalam laporan akhir.

g. Kesimpulan

Mahasiswa membuat use case diagram untuk disusun ke dalam project akhir.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 50 : PROJECT FINAL

Pokok Bahasan	:	Laporan Database
Acara Praktikum/Praktik	:	Minggu 14 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu membuat use case secara kelompok dari studi kasus project akhir.
2. Mahasiswa mampu merancang database secara kelompok dari studi kasus project akhir.

b. Indikator Penilaian

Keberhasilan mahasiswa membuat use case, merancang dan menyusun database secara berkelompok dari studi kasus project akhir.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model
4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

- Kertas A4
- Kertas Folio
- Sticky Note
- Spidol

e. Prosedur Kerja

Diskusikan dengan kelompok untuk membuat perancangan database project akhir dan mengimplementasikan hasil perancangan (SQLite lokal dengan sqflite, atau MySQL melalui API PHP seperti pada Acara 41–44), dan mulai menyusunnya ke dalam laporan akhir.

f. Hasil dan Pembahasan

Dokumentasi perancangan dan implementasi database untuk disusun dalam laporan akhir.

g. Kesimpulan

Mahasiswa membuat perancangan dan implementasi database untuk disusun ke dalam project akhir.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 51 : PROJECT FINAL

Pokok Bahasan	:	Progres Implementasi Project
Acara Praktikum/Praktik	:	Minggu 14 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu mengimplementasikan aplikasi berbasis Flutter dari kasus tertentu.
2. Mahasiswa mampu menerapkan konsep dasar Flutter yang telah dipelajari.
3. Mahasiswa mampu melaporkan pencapaian progres dari project yang dikerjakan.
4. Mahasiswa mampu mempresentasikan progres masing-masing kelompok.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam kelompok mengimplementasikan aplikasi berbasis Flutter dari kasus tertentu.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model

4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

- Kertas A4
- Kertas Folio
- Sticky Note
- Spidol

e. Prosedur Kerja

1. Diskusikan dengan kelompok untuk membuat implementasi aplikasi berbasis Flutter dari kasus tertentu.
2. Push setiap kegiatan dalam membangun aplikasi pada akun GitHub (lihat Acara 1–4).
3. Invite seluruh anggota kelompok dan dosen pengampu pada repositori GitHub.
4. Melakukan laporan pencapaian dari project yang dikerjakan.
5. Menyampaikan target untuk monitoring selanjutnya.
6. Presentasi progres.

f. Hasil dan Pembahasan

Dokumentasi implementasi project akhir dalam membangun aplikasi pada akun GitHub.

g. Kesimpulan

Mahasiswa setiap kelompok melaporkan pencapaian progres dari project yang dikerjakan yang di-push ke GitHub.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 52 : PROJECT FINAL

Pokok Bahasan	:	Dokumentasi Project
Acara Praktikum/Praktik	:	Minggu 14 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu mengimplementasikan aplikasi berbasis Flutter dari kasus tertentu.
2. Mahasiswa mampu menerapkan konsep dasar Flutter yang telah dipelajari.
3. Mahasiswa mampu melaporkan pencapaian progres dari project yang dikerjakan.
4. Mahasiswa mampu mempresentasikan progres masing-masing kelompok.

b. Indikator Penilaian

Keberhasilan mahasiswa dalam kelompok mengimplementasikan aplikasi berbasis Flutter dari kasus tertentu.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model

4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

1. Laptop
2. Jaringan Internet
3. Kabel Data USB
4. Perangkat keras pendukung pengambilan video
5. Perangkat lunak pendukung pengeditan video

e. Prosedur Kerja

Diskusikan dengan kelompok untuk membuat sebuah video dokumentasi pada aplikasi yang telah dibuat, serta dokumentasi dalam bentuk laporan akhir dan manual book. Sertakan file APK hasil `flutter build apk --release`.

f. Hasil dan Pembahasan

1. Video dokumentasi penggunaan aplikasi dengan durasi maksimal 5 menit.
2. Laporan akhir dan manual book.

g. Kesimpulan

Mahasiswa setiap kelompok membuat dokumentasi final project berupa video, laporan, dan manual book.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 53 : PROJECT FINAL

Pokok Bahasan	:	Dokumentasi Project
Acara Praktikum/Praktik	:	Minggu 15 / 1
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mahasiswa mampu mempresentasikan hasil project-nya.

b. Indikator Penilaian

Keberhasilan mahasiswa menjelaskan hasil final project, dan penguasaan materi.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model
4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

1. Laptop
2. Jaringan Internet
3. Kabel Data USB
4. Perangkat keras pendukung pengambilan video
5. Perangkat lunak pendukung pengeditan video

e. Prosedur Kerja

1. Secara bergantian presentasikan hasil pengerjaan project aplikasi Flutter, setiap sesi 30 menit (demo aplikasi pada HP/emulator, penjelasan arsitektur kode, dan repositori GitHub).
2. Lanjutkan dengan diskusi bersama dosen penguji.

f. Hasil dan Pembahasan

Setiap masukan dari dosen penguji menjadi saran perbaikan project yang dikerjakan.

g. Kesimpulan

Presentasi adalah proses pemaparan hasil penyelesaian final project.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 54 : PROJECT FINAL

Pokok Bahasan	:	Dokumentasi Project
Acara Praktikum/Praktik	:	Minggu 15 / 2
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mahasiswa mampu mempresentasikan hasil project-nya.

b. Indikator Penilaian

Keberhasilan mahasiswa menjelaskan hasil final project, dan penguasaan materi.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model
4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

1. Laptop
2. Jaringan Internet
3. Kabel Data USB
4. Perangkat keras pendukung pengambilan video
5. Perangkat lunak pendukung pengeditan video

e. Prosedur Kerja

1. Secara bergantian presentasikan hasil pengerjaan project aplikasi Flutter, setiap sesi 30 menit (demo aplikasi pada HP/emulator, penjelasan arsitektur kode, dan repositori GitHub).
2. Lanjutkan dengan diskusi bersama dosen penguji.

f. Hasil dan Pembahasan

Setiap masukan dari dosen penguji menjadi saran perbaikan project yang dikerjakan.

g. Kesimpulan

Presentasi adalah proses pemaparan hasil penyelesaian final project.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 55 : PROJECT FINAL

Pokok Bahasan	:	Presentasi Project
Acara Praktikum/Praktik	:	Minggu 15 / 3
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mahasiswa mampu mempresentasikan hasil project-nya.

b. Indikator Penilaian

Keberhasilan mahasiswa menjelaskan hasil final project, dan penguasaan materi.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model
4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

1. Laptop
2. Jaringan Internet
3. Kabel Data USB
4. Perangkat keras pendukung pengambilan video
5. Perangkat lunak pendukung pengeditan video

e. Prosedur Kerja

1. Secara bergantian presentasikan hasil pengerjaan project aplikasi Flutter, setiap sesi 30 menit (demo aplikasi pada HP/emulator, penjelasan arsitektur kode, dan repositori GitHub).
2. Lanjutkan dengan diskusi bersama dosen penguji.

f. Hasil dan Pembahasan

Setiap masukan dari dosen penguji menjadi saran perbaikan project yang dikerjakan.

g. Kesimpulan

Presentasi adalah proses pemaparan hasil penyelesaian final project.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**

Acara 56 : PROJECT FINAL

Pokok Bahasan	:	Presentasi Project
Acara Praktikum/Praktik	:	Minggu 15 / 4
Tempat	:	Daring / Luring Politeknik Negeri Jember
Alokasi Waktu	:	170 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

Mahasiswa mampu mempresentasikan hasil project-nya.

b. Indikator Penilaian

Keberhasilan mahasiswa menjelaskan hasil final project, dan penguasaan materi.

c. Dasar Teori

Definisi Database

Database atau dalam bahasa Indonesia disebut basis data adalah sekumpulan data berupa informasi yang saling berhubungan antara satu dengan lainnya yang disimpan di dalam komputer secara sistematis sehingga dapat diakses untuk digunakan kembali.

Pengguna database terbagi menjadi 2 jenis, yaitu *database administrator* dan *database user*. Database administrator (DBA) berwenang dalam pengawasan database, serta berfungsi mendefinisikan pola struktur database dan memodifikasi pola dan organisasi fisik. Sementara database user adalah pengguna dari database tersebut.

Sumber: https://sis.binus.ac.id/2018/12/18/database-secara-umum/

Conceptual Design Database

Conceptual Design Database adalah tahapan pertama dalam desain database. Tujuan tahapan ini adalah merancang database yang independen dari *database software* dan detail fisik. Tahapan ini menghasilkan *conceptual data model* yang menjelaskan entitas data, atribut, hubungan antar tabel, dan *constraints* di suatu database. Desain ini bersifat deskriptif dan naratif.

Semua elemen data yang dibutuhkan dalam suatu transaksi database harus dijabarkan di modelnya, dan semua elemen data yang ada harus digunakan setidaknya satu kali di suatu transaksi database. Conceptual database memiliki 4 tahapan yaitu:

1. Analisis dan persyaratan data
2. Normalisasi dan perancangan hubungan antar entitas
3. Verifikasi data model
4. Distributed database design

Sumber: https://sis.binus.ac.id/2020/07/07/database-design-conceptual-design/

d. Alat dan Bahan

1. Laptop
2. Jaringan Internet
3. Kabel Data USB
4. Perangkat keras pendukung pengambilan video
5. Perangkat lunak pendukung pengeditan video

e. Prosedur Kerja

1. Secara bergantian presentasikan hasil pengerjaan project aplikasi Flutter, setiap sesi 30 menit (demo aplikasi pada HP/emulator, penjelasan arsitektur kode, dan repositori GitHub).
2. Lanjutkan dengan diskusi bersama dosen penguji.

f. Hasil dan Pembahasan

Setiap masukan dari dosen penguji menjadi saran perbaikan project yang dikerjakan.

g. Kesimpulan

Presentasi adalah proses pemaparan hasil penyelesaian final project.

h. Rubrik Penilaian

No	Aspek Penilaian	Bobot
1	Kelengkapan	25 %
2	Kesesuaian	25 %
3	Kedalaman materi	25 %
4	Kreativitas	25 %
Jumlah Skor		**100 %**